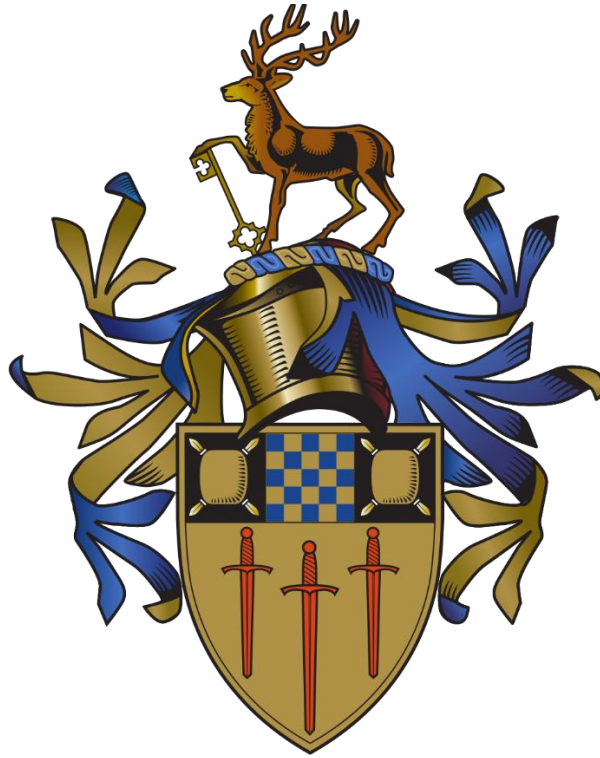




Development and Implementation of a Localisation Module for Autonomous Agile Robot Navigation

FINAL DISSERTATION – YEAR 3 PROJECT
THURSDAY, 7 MAY 2026

Department of Computer Science and Electronic Engineering
Faculty of Engineering and Physical Sciences
University of Surrey

Author: Jude John Burton

Author's Student URN: 677t9503

Project Supervisor: Professor S. Fallah

Module Name: Year 3 Project

Module Code: EEE3017

Course: UFA12-5, (BEng) Electronic Engineering with Computer Systems (with PTY)

Abstract

This project focuses on the development and deployment of a modular LiDAR–IMU localisation and SLAM framework for an autonomous robotic platform using ROS 2 and embedded NVIDIA hardware. The system integrates LiDAR odometry, IMU fusion, keyframe mapping, and configurable runtime workflows to support real-time localisation and mapping within indoor environments.

Development was carried out using both simulation-first testing and live hardware deployment on a modified Traxxas X-Maxx robotic platform equipped with a Livox MID-360 LiDAR sensor, Microstrain IMU, and NVIDIA Jetson embedded systems. Multiple localisation approaches were implemented throughout development, including both a custom NDT-based LiDAR odometry pipeline and FAST-LIO integration. Significant refinement was also carried out surrounding LiDAR deskewing, sensor calibration, runtime optimisation, deployment automation, networking, and overall system stability during real-world operation.

A major focus of the project was not only developing the localisation framework itself, but also understanding the practical engineering problems involved in deploying SLAM systems onto physical robotic hardware. As a result, a large amount of development time was spent solving issues relating to timing consistency, TF alignment, vibration, hardware integration, power delivery, unstable networking, and embedded-system reliability.

The final system successfully demonstrated real-time localisation and keyframe mapping during live robot operation while supporting multiple deployment configurations across TX2i and Jetson Orin Nano platforms. Although full standalone integration with the perception framework was not completed within the project scope, simultaneous runtime operation of both systems on the same embedded device was successfully demonstrated as a proof of concept.

Overall, the project successfully produced a modular and deployment-ready LiDAR–IMU SLAM framework while also providing substantial practical understanding of real-world robotic deployment, ROS 2 integration, embedded systems, and sensor-fusion engineering.

Glossary

Term	Meaning
Bias	A systematic offset in sensor measurements that causes consistent error accumulation over time if not modelled or corrected.
Drift	The gradual accumulation of error in estimated position or orientation over time, typically caused by sensor noise or bias.
EKF (Extended Kalman Filter)	A nonlinear state estimation algorithm that combines a predictive motion model with sensor measurements to estimate a system's state while accounting for uncertainty.
Ground Truth	A reference representation of the true system state, used to evaluate estimation accuracy and quantify error in simulations or experiments.
IMU (Inertial Measurement Unit)	A sensor module that measures linear acceleration and angular velocity, and optionally magnetic field data, is used to estimate motion and orientation over short time periods.
LiDAR (Light Detection and Ranging)	A sensing technology that measures distances by emitting laser pulses and timing their reflections, producing a 2D or 3D representation of the surrounding environment.
Localisation	The process of estimating a robot's position and orientation within a known or partially known environment.
Measurement Model	A mathematical model that relates sensor measurements to the system state within a state estimation framework.
Multi-Modal Sensor Fusion	The integration of data from multiple sensor types, such as LiDAR, IMU, and wheel encoders, improves robustness and estimation accuracy.
Odometry	The estimation of a robot's change in position and orientation over time using motion-related sensor data, such as wheel encoders, IMUs, or visual information.
Pose	A representation of a robot's position and orientation, typically expressed as spatial coordinates and rotational angles.

DDS (Data Distribution Service)	The middleware communication protocol used by ROS 2 for message passing, discovery, and distributed communication between nodes.
FAST-LIO	A LiDAR–Inertial Odometry framework that tightly couples LiDAR and IMU measurements for high-frequency localisation and mapping.
Keyframe Mapping	A mapping approach where selected LiDAR scans are stored and accumulated based on motion or time thresholds to reduce redundancy and computational cost.
NDT (Normal Distributions Transform)	A scan-matching algorithm used for LiDAR odometry that aligns point clouds using probabilistic voxel representations.
Point Cloud	A collection of spatial points representing the surrounding environment, commonly generated by LiDAR sensors.
ROS 2 (Robot Operating System 2)	A robotics middleware framework providing communication, package management, hardware abstraction, and distributed processing tools for robotic systems.
Scan Matching	The process of aligning sequential LiDAR scans to estimate relative robot motion between observations.
SLAM (Simultaneous Localisation and Mapping)	The process of simultaneously estimating a robot's pose while constructing a map of an unknown environment.
TF (Transform Framework)	The ROS 2 coordinate transform system used to maintain spatial relationships between robot and sensor reference frames.
Deskewing	A LiDAR preprocessing technique that compensates for sensor motion occurring during scan acquisition to reduce distortion in point-cloud data.

Contents

Abstract.....	1
Glossary	2
Contents	4
List of Figures	8
List of Tables.....	10
1 Introduction.....	11
2 Project Overview.....	11
2.1 Motivation	11
2.2 Objectives and Deliverables	12
2.3 Specification.....	12
2.4 Constraints.....	12
3 Literature Review	13
3.1 Concepts	13
3.1.1 <i>Localisation</i>	13
3.1.1.1 Bayes Filter	14
3.1.1.2 Challenges	14
3.1.2 <i>Odometry</i>	15
3.1.2.1 Dead Reckoning.....	15
3.1.3 <i>Sensor Fusion</i>	16
3.1.4 <i>SLAM</i>	16
3.1.5 <i>System Types</i>	18
3.2 Sensors.....	18
3.2.1 <i>2D - LiDAR</i>	18
3.2.2 <i>IMU</i>	20
3.2.3 <i>Wheel Encoders</i>	21
3.3 Algorithms.....	22
3.3.1 <i>State Estimation and Sensor Fusion</i>	22
3.3.1.1 Extended Kalman Filter	22
3.3.1.2 Error-State Kalman Filter.....	23
3.3.2 <i>LiDAR Scan Matching</i>	23
3.3.2.1 Normal Distributions Transform.....	24
3.3.3 <i>SLAM Algorithms</i>	24
3.3.3.1 Graph-Based SLAM.....	24
3.3.4 <i>Map-Based Localisation</i>	25

3.3.4.1 Continuous Scan-to-Map Matching	25
3.4 Software - ROS 2	25
3.4.1 Overview.....	25
3.4.2 Architecture	25
3.4.3 Deployment.....	26
3.5 Hardware	26
3.5.1 Nvidia Jetson Nano.....	26
3.6 Summary	26
4 Preliminary Investigation and Technology Evaluation	27
4.1 Initial Direction and Strategy	27
4.2 Investigatory Work.....	28
4.2.1 Correction Investigation.....	28
4.2.2 Filter Investigation	29
4.2.3 Resulting SLAM Framework	32
4.3 FAST-LIO	33
4.4 Hardware Platform Evaluation	33
4.4.1 Jetson Platforms	33
4.4.2 ROS Versions.....	34
4.5 Summary	34
5 System Design	34
5.1 Specifications	34
5.2 High-Level System Overview	35
5.3 Hardware Design	36
5.3.1 Overall Hardware Architecture	36
5.3.2 Circuit Design	38
5.3.3 Existing Mounting Structure.....	39
5.3.4 Mechanical Measurements	41
5.3.4.1 Original	41
5.3.4.2 Alteration Design	43
5.3.5 3D Enclosure Design.....	46
5.4 Software Design	49
5.4.1 Software Architecture Overview.....	49
5.4.2 Processing Pipeline.....	50
5.4.3 Modular Software Approach	51
5.4.4 ROS 2 Framework and Node Architecture.....	53

5.4.4.1 Sensor Drivers	54
5.4.4.2 Localisation Node	55
5.4.4.3 Mapping Node.....	55
5.4.4.4 State Estimation Node.....	55
5.4.4.5 Processing Nodes	55
5.4.5 <i>ROS Communication Architecture</i>	55
5.4.6 <i>Coordinate Frame and TF Architecture</i>	57
5.4.8 <i>Launch System Design and Configuration</i>	59
5.5 Procurement and Cost Analysis	59
5.6 Summary	61
6 Methodology	61
6.1 Implementation Strategy	61
6.2 Staged Development Process	62
6.2.1 <i>ROS 2 Environment and Workspace Setup</i>	62
6.2.2 <i>Sensor Integration and Validation</i>	63
6.2.2.1 Microstrain GX5-3DM-25 IMU	63
6.2.2.2 Livox MID-360 LiDAR.....	63
6.2.3 <i>EKF State Estimator Development</i>	64
6.2.4 <i>LiDAR Odometry Development</i>	66
6.2.5 <i>Sensor Fusion Refinement</i>	67
6.2.6 <i>Keyframe Mapping Integration</i>	68
6.2.7 <i>Feature Expansion and System Refinement</i>	69
6.2.8 <i>Playback-Based Validation and Iterative Testing</i>	69
6.3 Hardware Setup	70
6.3.1 <i>Assembly and Integration</i>	70
6.3.2 <i>Hardware Layout Grid</i>	81
6.3.3 <i>Coordinate Offsets From Origin</i>	83
6.4 Software Setup and Deployment.....	84
6.4.1 <i>Environment Setup</i>	85
6.4.1.1 Ubuntu 22.04, ROS 2 Humble	85
6.4.1.2 Ubuntu 18.04, ROS 2 Foxy	85
6.4.2 <i>Package Installation and Deployment</i>	86
6.4.3 <i>Runtime Procedure and Launch Sequence</i>	87
6.4.3.1 Full SLAM Deployment	87
6.4.3.2 Sensor Deployment	88

6.4.3.3 Sensor Deployment	88
6.5 Summary	88
7 System Development and Integration	89
7.1 DEVELOPMENT Overview	89
7.2 Hardware	89
7.2.1 <i>Challenges and Solutions</i>	89
7.2.1.1 Jetson Orin Nano Mount	89
7.2.1.2 Batteries and correct storage	90
7.2.1.3 IMU USB Port	91
7.2.1.4 Wi-Fi Adapter	91
7.2.1.5 Jetson Nano Failure	93
7.2.1.6 Robot Speed	94
7.3 Software	94
7.3.1 <i>Challenges and Solutions</i>	94
7.3.1.1 TX2i Driver Compatiblity	94
7.3.1.2 LiDAR Setup	95
7.3.1.3 IMU Setup	96
7.3.1.4 Auto-Starting Program	96
7.3.1.5 Data Disruption	97
7.4 Localisation and SLAM	98
7.4.1 <i>Challenges and Solutions</i>	98
7.4.1.1 Keyframe Smear	98
7.4.1.2 Sensor Relative Pose Tuning	98
7.4.1.3 Fixing NDT	98
7.4.1.4 Object Obstruction	99
7.5 System Optimisation and Engineering Judgement	100
8 Results and Evaluation	101
8.1 Results	101
8.2 Assessment	105
8.3 Advantages and Limitations	106
8.4 Lessons Learned	106
8.6 Summary	107
9 Conclusion and Further Work	108
References	110
Appendices	113

Appendix A Planning	113
Appendix B: Additional Project Information.....	116
Appendix C: Links	121

LIST OF FIGURES

Figure 1 - 3D Axis ^[23]	14
Figure 2 - SLAM Mapping Result ^[26]	17
Figure 3 - IMU Accelerometer, Gyroscope Drift ^[24]	20
Figure 4 - ROS Architecture ^[17]	26
Figure 5 - Experiment Environment	28
Figure 6 - IMU Only Simulation	29
Figure 7 - Wheel Odometry Enabled Simulation	29
Figure 8 - Filter Position Error, No Odom.....	30
Figure 9 - Filter Orientation Error, No Odom.....	31
Figure 10 - Filter Position Error, Odom.....	31
Figure 11 - Filter Orientation Error, Odom	32
Figure 12 - Proposed Framework	32
Figure 13 - FAST-LIO Pipeline ^[27]	33
Figure 14 - System Design High-Level Overview	36
Figure 15 - Hardware Architecture Design.....	38
Figure 16 - System Circuit Design.....	39
Figure 17 – Original Inner Enclosure	40
Figure 18 – Original Outer Enclosure.....	41
Figure 19 - Robot Dimensions	42
Figure 20 - Outer Enclosure Dimensions, Sagittal Plane	42
Figure 21 - Outer Enclosure Dimensions, Frontal Plane	43
Figure 22 - IMU Mount Design	44
Figure 23 - LiDAR Mount Transversal Plane Measurements.....	45
Figure 24 - LiDAR Mount Frontal Plane Measurements	45
Figure 25 - Jetson Orin Nano Mount Location Dimensions.....	46
Figure 26 - Jetson Orin Nano Mount	47
Figure 27 - Jetson Orin Nano Mount, Full Schematic	48
Figure 28 - Jetson Orin Nano Enclosure Displacement Analysis.....	49
Figure 29 - Jetson Orin Nano Factor Analysis.....	49
Figure 30 - Software Overall Architecture	50
Figure 31 - Primary Processing Pipeline	51
Figure 32 - Primary ROS Topics	51
Figure 33 - Package Dependency Tree	52
Figure 34 - Workspace Structure	53
Figure 35 - Full System Pipeline Graph	53
Figure 36 - TF Architecture	58
Figure 37 - Launch and Configuration Hierarchy.....	59
Figure 38 - Development Strategy	62
Figure 39 - Microstrain IMU Message	63

Figure 40 - Raw LiDAR Point-Cloud.....	64
Figure 41 - Pure IMU Estimation	65
Figure 42 - EKF Performance Comparison	66
Figure 43 - NDT Before Tuning	67
Figure 44 - NDT After Tuning.....	67
Figure 45 - EKF vs LiDAR Odom Path.....	68
Figure 46 - Keyframe Map	69
Figure 47 - Refinement Workflow.....	70
Figure 48 - TX2i + LiDAR Power Cable Procurement	71
Figure 49 - TX2i Power Cable Installation.....	71
Figure 50 - Jetson Orin Nano Power Cable Procurement.....	71
Figure 51 - IMU Mount	72
Figure 52 - IMU Mounted.....	72
Figure 53 - Expected Result 1	72
Figure 54 - Power Station Integration.....	73
Figure 55 - Extra Fan Removal.....	73
Figure 56 - Inner Enclosure Integration.....	74
Figure 57 - Inner Enclosure Fixture + Battery Strap	74
Figure 58 - Wi-Fi Adapter	75
Figure 59 - USB Hub Connection.....	75
Figure 60 - Camera Integration.....	76
Figure 61 - Jetson Power Cable Connection.....	76
Figure 62 - Jetson Nano Mounted.....	77
Figure 63 - Jetson Nano Power Cable Insertion	77
Figure 64 - Expected Result 2.....	77
Figure 65 - LiDAR Mount Integration.....	78
Figure 66 - LiDAR Cable Connection	78
Figure 67 - LiDAR Cable Wiring	79
Figure 68 - LiDAR Cable Organisation.....	79
Figure 69 - Battery Connection.....	80
Figure 70 - LiDAR Ethernet Connection.....	80
Figure 71 - Final Integration	81
Figure 72 - Expected Result 3.....	81
Figure 73 - Final Result, Top-Down View	82
Figure 74 - Exposed Final Result, Top-Down View	82
Figure 75 - Axis Relative to Robot.....	84
Figure 76 - Robot Origin Visual Identification.....	84
Figure 77 - Incorrect Jetson Mount.....	90
Figure 78 - Final Jetson Mount.....	90
Figure 79 – Wi-Fi Antenna Implementation.....	92
Figure 80 - Broken Wi-Fi Adapter.....	92
Figure 81 - Wi-Fi Adapter Repair Process	93
Figure 82 - Livox Config File.....	95
Figure 83 - Example Failed LiDAR Initialisation	95
Figure 84 - LiDAR Datastream.....	95
Figure 85 - Microstrain Configuration File Location	96

Figure 86 - IMU Data-Stream.....	96
Figure 87 - Bad Data Info	97
Figure 88 - Keyframe Smear.....	98
Figure 89 - Tilted Keyframe Map.....	99
Figure 90 - Correct Keyframe map.....	99
Figure 91 - RTK GPS Obstruction.....	100
Figure 92 - Keyframe Map with Path Visible.....	102
Figure 93 - Loop Closure Logs.....	102
Figure 94 - Keyframe vs Image	103
Figure 95 - FAST-LIO Comparison	103
Figure 96 - Keyframe Map Views	104
Figure 97 - Runtime with Perception Model	104
Figure 98 - 60% Throttle Keyframe Map	105
Figure 9 - Complete Gantt Chart	113

LIST OF TABLES

Table 1 - Project Objectives.....	12
Table 2 - Project Main Deliverables	12
Table 3 – High-Level Specification.....	12
Table 4 - Localisation Constraints ^[2]	14
Table 5 - LiDAR Use-Cases ^[6]	19
Table 6 - 2D LiDAR Constraints ^[3]	19
Table 7 - 6-Axis IMU Updates Rates ^[4]	20
Table 8 - 6-Axis IMU Constraints ^[1]	20
Table 4 - System Specification	34
Table 5 -Existing Hardware Mount Identification.....	40
Table 6 - Packages Detail	52
Table 7 - Active Nodes Functionality.....	54
Table 8 - Primary ROS Topics.....	56
Table 9 - Primary TF Transforms	57
Table 10 - Procurement Table.....	60
Table 11 - SLAM Basic Launch Arguments	87
Table 1 - SLAM Component Optimisation	101
Table 18 - Risk Assessment Matrix	114

1 | Introduction

Autonomous robotic systems require reliable localisation and environmental awareness to operate effectively within real-world environments. One of the most common methods for achieving this is through Simultaneous Localisation and Mapping (SLAM), where a robot estimates its own position while simultaneously generating a map of its surroundings. However, although many SLAM systems perform well in controlled demonstrations or simulation environments, deploying them onto real robotic hardware introduces many additional practical engineering challenges relating to timing consistency, sensor fusion, calibration, networking, computational limitations, and hardware reliability.

This project focuses on the development and implementation of a modular LiDAR–IMU localisation and SLAM framework using ROS 2 and embedded NVIDIA compute platforms. The system integrates LiDAR odometry, IMU fusion, keyframe mapping, and configurable deployment workflows to support real-time localisation on an autonomous robotic platform. Development was carried out using both simulation-first testing and live hardware deployment, allowing iterative refinement of the localisation pipeline throughout the project.

A major focus of this project was not only developing the localisation framework itself, but also understanding the practical engineering challenges involved in deploying SLAM systems onto real hardware. As a result, a large portion of the project involved refinement of sensor calibration, LiDAR deskewing, runtime optimisation, hardware integration, deployment tooling, network communication, and debugging infrastructure. Multiple runtime configurations were also implemented, including both a custom NDT-based localisation pipeline and FAST-LIO integration.

This report discusses the full development process of the system, beginning with background research into localisation and SLAM technologies before moving into the proposed system design, hardware architecture, software implementation, and deployment workflow. The report then covers the iterative development process, optimisation stages, hardware and software challenges encountered throughout the project, final system results, evaluation of the completed framework, lessons learned, and recommendations for future work.

Overall, the final system demonstrates a modular and deployment-ready LiDAR–IMU localisation framework capable of operating on embedded robotic hardware while also providing a practical understanding of the challenges associated with real-world robotic SLAM deployment.

2 | Project Overview

2.1 | MOTIVATION

Accurate localisation is essential for agile indoor mobile robots, as navigation and control depend on reliable estimates of position and orientation. Indoor environments are particularly challenging due to the absence of GPS, dynamic obstacles, and geometric ambiguity, making

onboard sensor-based localisation necessary. While SLAM is widely used in unknown environments, its computational cost can limit real-time performance for fast-moving robots. This project is therefore motivated by the need for a lightweight, real-time localisation system that combines an IMU and LiDAR sensor to achieve robust indoor navigation.

2.2 | OBJECTIVES AND DELIVERABLES

The Objectives and Deliverables are displayed in in *Table 1* & *2*, respectively. These objectives outline the core tasks of the project, from literature survey to deployment of the final prototype localisation system. The items within the Deliverables have been directly integrated to the Gantt chart.

Table 1 - Project Objectives

Objective / Deliverable	Description
<i>Literature survey on localisation systems</i>	Review probabilistic localisation, odometry, SLAM, sensor fusion, and LiDAR/IMU-based methods
<i>Learning ROS 2</i>	Gain familiarity with ROS 2 concepts, nodes, topics, and deployment
<i>Develop a localisation algorithm using multi-modal sensor information</i>	Design and justify IMU–LiDAR fusion using probabilistic filters (EKF/ESKF)
<i>Implement the algorithm on a robot computing system</i>	Deploy localisation module on NVIDIA Jetson Nano using ROS 2
<i>Real-time performance analysis and demonstration</i>	Evaluate latency, update rate, drift correction, and robustness
<i>Report writing</i>	Document methodology, results, and evaluation clearly and professionally

Table 2 - Project Main Deliverables

Item	Contribution	Due Date
<i>Project Agreement & Logbook</i>	Pass/Fail requirement	End of Week 2 (Autumn Semester)
<i>Mid-Term Report</i>	10%	Friday 2 January, 4:00 pm (Week 12)
<i>Final Project Report</i>	45%	Wednesday 7 May, 4:00 pm (Week 11)
<i>Final Presentation & Viva</i>	45%	Weeks 14–15 (June)

2.3 | SPECIFICATION

From the objectives in *Table 1* as well as the initial project meetings, the high-level system specification in *Table 3* was developed.

Table 3 – High-Level Specification

Category	Specification
Robot type	4-wheeled indoor mobile robot
State estimate	3D pose
Sensors	9-Axis IMU GX5-3DM-25, 3D LiDAR Livox MID-360
Target behaviour	Low-latency, high-rate pose output suitable for agile motion
Software stack	ROS 2, containerised deployment (Docker)
Deployment platform	NVIDIA Jetson Nano (embedded), Nvidia TX2i
Outputs	Corrected pose estimate
Validation approach	Simulation first (including optional wheel-encoder), deployment, real-time performance testing

2.4 | CONSTRAINTS

The key constraints of the project are:

1. Only IMU and 2D LiDAR Sensor
2. Deployment must be on Jetson Nano

3. ROS 2 version mismatch with Jetson Nano requires containerization
4. Operation is for indoor, agile motion
5. Time constraints; October 2025 – June 2026

These constraints set boundaries on the approaches available for achieving a final solution, influencing algorithm choice and limiting the feasible computational and sensing strategies. The literature survey serves as the main method for developing an in-depth understanding of these limitations and the potential approaches for a solution available.

3 | Literature Review

In order to efficiently approach the decomposition and planning of the project ([Section 2](#)), it was essential to first develop a clear and structured understanding of the problem domain. As localisation systems are inherently multidisciplinary, this required examining several closely related areas rather than treating localisation as an isolated task.

To achieve this, a range of relevant literature was reviewed, focusing on the most significant components in robotic localisation systems: localisation theory, odometry, sensor fusion, SLAM, applicable sensors, the algorithms to bring these together and the implementation method. Particular emphasis was placed on approaches compatible with 2D LiDAR and inertial sensing, aligning with the hardware specifications of the target platform.

By studying these topics, this review establishes the theoretical basis required to justify algorithm selection, system design, and implementation decisions made later in the project.

3.1 | CONCEPTS

Please note that each sub-section generally follows the following rough format(if applicable):

1. Definition/Introduction
2. Theoretical background
3. Challenges/Drawbacks
4. Use-cases
5. *Engineering judgement*

3.1.1 | Localisation

Localisation is the core objective of the project, and the items following it in this literature review are merely components or methods of it. Specifically, ‘localisation’ is the fundamental process of determining a robot's pose (state vector), comprising its position (x, y, z) and orientation (*roll, pitch, yaw*) relative to a [global or local](#) reference frame^[1]. For this project, the state vector is simply (x, y, θ) given its 2D nature^[2]. Blatantly, localisation is the math and the final ‘goal’; its job is to answer the question “*Where am I?*”^[1]

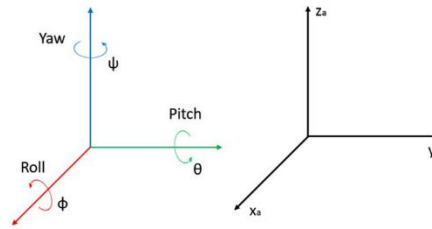


Figure 1 - 3D Axis^[23]

The ‘localisation module’ refers to the software component responsible for estimating the robot’s pose^[3]. Accurate localisation is essential, as without it path planning and control are not possible^[4]. In this project, localisation must operate in real time and is treated as a recursive estimation problem, continuously updating the robot’s state using noisy sensor data.^[5]

The theoretical basis of localisation relies heavily on Probabilistic Robotics, specifically the Bayes Filter framework^[1]. Probabilistic Robotics refers to using probability distributions instead of exact measurement values; it models the uncertainty in the sensor data^[1].

3.1.1.1 | Bayes Filter

The Bayes Filter’s objective is to estimate the ‘posterior’ distribution, which is the range of potential values that the pose can take on, each with its own likelihood (modelled by a PDF), given the previous state and motion, as well as the current sensor data^[1]. The pose with the highest probability is selected, and then the cycle repeats^[1]. Mathematically, this is the probability of the post x_t at time t , given the past measurements $z_{1:t}$ and control inputs $u_{1:t}$; (1.1).

$$p(x_t | z_{1:t}, u_{1:t}) \quad (1.1)^{[1]}$$

This process is split into two phases: the ‘Prediction’ phase and the ‘Correction’ phase.^[1]

- *Prediction Phase*: estimates the robot’s current state by advancing the previous state forward in time using a motion model and the information on its motion since the last timestamp, which is obtained from internal sensor data^[1].
- *Correction Phase*: refines the predicted state by using sensor data that relates the robot to its environment (environmental sensors, e.g. LiDAR scans, camera features, or range measurements) to correct the predicted pose^[1].

3.1.1.2 | Challenges

Despite the robustness of probabilistic frameworks, physical localisation is hindered by several systemic limitations, displayed in Table 4^[2].

Table 4 - Localisation Constraints^[2]

Limitation	Description
Sensor noise	A primary challenge as every hardware component introduces a degree of stochastic(random) error in the measurement data
Computational latency	Introduces a time mismatch between the robot’s actual physical state and its calculated pose. If the localisation algorithm is computationally expensive, processing delays can introduce control instability, as the robot processes with outdated state estimates. Consequently, high-speed or agile operation requires the localisation module to deliver low-latency, high-frequency updates.

<i>Sensor update mismatch</i>	Occurs when the robot is moving at increased velocities, and the environment-based sensors update too slowly to smoothly track motion on their own. Thus, the solution is to use motion-based sensors in dead-reckoning to estimate motion between the environment-sensor updates, using those slower sensor updates to correct drift.
<i>Dynamic environments</i>	When the robot is present in a dynamic environment (e.g. people walking past or in front of the robot), it may misinterpret that as robot motion.

Based on this analysis, reliable real-time localisation for an agile autonomous ground robot requires a probabilistic approach that can manage sensor noise, latency, and environmental uncertainty. The Bayes filter framework is well-suited to this task, enabling high-frequency motion data from an IMU to provide continuous dead-reckoned state propagation, while lower-rate environment-based sensors such as 2D LiDAR are used to periodically correct accumulated drift. At higher velocities, this fusion becomes essential to achieve a smooth and temporally consistent pose estimate. Consequently, the localisation module must be optimised for low latency, effective sensor fusion, and suitable for dynamic environments.

3.1.2 | Odometry

Odometry is one of the main parts of localisation and is closely related to dead reckoning; it is the process of estimating a robot's incremental change in position and orientation over time using internal motion sensors^[1]. Odometry provides continuous relative motion information that (basis of dead reckoning), used to estimate the robot's new state^[1]. Within the localisation module, odometry serves as the primary source of relative motion estimation; in this project, it is purely inertial, provided by the 6-axis IMU^[1]. While this approach offers high-frequency state updates between external corrections, it suffers from cumulative drift due to sensor noise and integration errors^[1].

By modelling the robot as moving in a 2D plane(project specification), its state (typically) can be represented by (x, y, θ) where x and y represent the horizontal and vertical position, respectively, and θ is the heading/angle^[2]. By measuring linear velocity v and angular velocity, ω . The new pose obtained by independently calculating x , y , θ : (2.2), (2.3), (2.4). In an inertial-only system, this involves double-integrating the linear acceleration data (2.1) from the IMU to derive position, and single-integrating the angular velocity (gyroscope) to derive orientation.^[2]

$$a = \frac{dv}{dt} \rightarrow v = \int a dt \quad (2.1)^{[2]}$$

$$x_{t+1} = x_t + v_t \cos(\theta_t) \Delta t \quad (2.2)^{[2]}$$

$$y_{t+1} = y_t + v_t \sin(\theta_t) \Delta t \quad (2.3)^{[2]}$$

$$\theta_{t+1} = \theta_t + \omega_t \Delta t \quad (2.4)^{[2]}$$

3.1.2.1 | Dead Reckoning

Estimating the robot's pose purely by motion measurements (from Odometry) and adding to the starting pose^[1]. IMU-only localisation is a form of dead-reckoning; it measures and evaluates the motion of the robot, and uses that to determine the new position^[1]. The major

drawback of this localisation approach is that the result is unavoidable drift (position error will significantly increase with time), as there is no environmental information to anchor the position estimations^[1].

During development, it is essential to observe accumulated drift, the impact of errors, and idealised odometry behaviour, as well as to assess how effectively these errors can be mitigated without external sensors.

3.1.3 | Sensor Fusion

Sensor fusion is the process of combining data from multiple sensors. Within the project context, it is done to produce a state estimate that is more accurate and reliable than any single sensor could provide in isolation^[1]. Specifically, to this localisation module, sensor fusion serves as a layer that bridges IMU-based odometry with LiDAR observations ([Bayes' filter](#)). Consequently, weaknesses are mitigated; gain LiDAR's spatial stability and IMU's responsiveness.^[1]

Optimal estimation forms the theoretical basis for sensor fusion and is commonly implemented using Bayesian filters such as the Kalman Filter, where the objective is to estimate the system state as accurately as possible given noisy and incomplete measurements^[1]. Within this framework, each sensor measurement carries uncertainty, and predicted states are combined with new measurements by weighting them according to their respective uncertainties to produce a more reliable estimate of the pose^[1].

The primary constraints with sensor fusion are ensuring noise matrices (Q and R) are valued correctly to prevent the filter to over-trust a drifting sensor or ignoring valid correction data, leading to inaccurate state estimates^[1]. Additionally, poor sensor synchronisation can result in measurements from different time instances being fused, causing pose discontinuities or estimation lag^[1].

Noise covariance values must be carefully selected for Sensor Fusion to ensure accurate estimates; identifying filters that can optimise this process will be crucial.

3.1.4 | SLAM

Simultaneous Localisation and Mapping (SLAM) is the process by which a robot constructs a map of an unknown environment while simultaneously estimating its own position within that map^[1]. While localisation assumes a pre-existing map is available, and odometry provides only relative motion, SLAM is the combined process used when a robot enters an unknown space^[1].

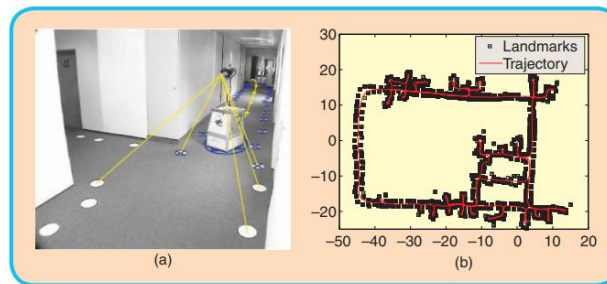


Figure 2 - SLAM Mapping Result^[26]

Theoretically, the robot maintains a probabilistic belief over both its state and the environment. SLAM can be expressed as the Bayes Filter Framework + Mapping, thus the resulting process is exactly as described mentioned [previously](#), with the addition of mapping^[1]. As the robot moves through the environment, the corresponding poses are associated with previously observed features or spatial locations. When the robot revisits a known area, these repeated observations provide a strong reference that allows accumulated drift to be reduced; this is called loop closure^[1]. Through repeated prediction and correction, the robot incrementally refines both its pose estimate and the map, improving consistency over time^[1].

The challenge with SLAM algorithmically is the co-dependency between the map and the robot's pose: to get a precise map, you need to have an accurate pose estimation; to get an accurate pose estimation, you need a precise map. A solution to this is [Graph-Based SLAM](#)^[3].

The primary limitation of SLAM is its computational complexity. As the robot explores larger environments, the number of stored poses and map elements increases, leading to higher memory usage and computational demand as more relationships must be evaluated, making real-time performance increasingly difficult to maintain^[1]. Additionally, 2D LiDAR are susceptible to geometric ambiguity: In environments where sensor observations appear indistinguishable across different locations, aliasing can occur, compromising map consistency and pose estimation^[1]. This can be corrected by comparing motion history to validate location^[1].

For high-speed scenarios, SLAM must be executed asynchronously to maintain real-time control performance^[3]:

- *Front-End*: High-frequency sensor (e.g. IMU) must run fast enough to provide poses for control
- *Back-End*: low-frequency global map optimisation and drift correction using accumulated environmental sensor (e.g. LiDAR) data and pose constraints

Implementing SLAM in our localisation module is essential for generating the environment reference data that is utilised to correct the IMU drift. In a real-time system restricted to 2D LiDAR and a 6-axis IMU, the SLAM approach will have to be carefully [selected](#) to match their sensor limitations; we will also have to consider the computing platform: [Nvidia Jetson Nano](#) and the computational power constraints that come with it.

3.1.5 | System Types

Localisation systems can be characterised by whether state estimation is driven by relative motion sensing or corrected using environment-referenced observations^[2]. For indoor robots using a 2D LiDAR and IMU, high-frequency local motion estimates are combined with lower-frequency environmental corrections to produce a single pose estimate in a common reference frame^[1].

The distinction between global and local systems is defined by the coordinate frames used to represent the robot's state^[3].

- *Local Systems* (Relative) – Also called “ego-motion”/ “odometry”, track the robot's displacement relative to a starting pose, e.g. (0, 0, 0) on a global map; rely on internal sensors like the 6-axis IMU to measure acceleration and angular velocity^[2].
- *Global Systems* (Absolute) – Determine the robot's pose within a fixed, external reference frame, such as a map of a maze; rely on environmental sensors like 2D LiDAR to match environmental features against known landmarks^[2].

The other main characteristic which defines a localisation system is its linearity, whether the control input to the resulting robot state can be described by a linear model^[2].

- *Linear Systems*: Assumes the motion follows a straight line.^[2]
- *Non-Linear Systems*: When a robot's motion depends on its orientation, such that changes in heading affect position through trigonometric relationships^[2].

For this project, the localisation system is designed as a locally driven estimator with environment-based correction. IMU data provides continuous short-term motion estimation, and LiDAR observations are used to correct accumulated drift. The system being moderately linear is a significant component when considering appropriate state estimation filters.

3.2 | SENSORS

3.2.1 | 2D - LiDAR

LiDAR (Light Detection and Ranging) serves as the primary environmental sensor for the localisation module, providing the global reference required to bound the unavoidable drift in the IMU^[3]. Systems use LiDAR sensors at a fixed rate, typical use cases can be seen in *Table 5*.^[4]

The sensor operates on the Time of Flight principle, transmitting infrared laser pulses and measuring the time between emission and reflection^[4]. Using a rotating mirror assembly, the environment is sampled in a 2D plane^[4]. The distance, d , is obtained by evaluating half the speed of light against the measured time delay (3.1), after which the point location is computed using (3.2) and (3.4) to obtain (x, y, θ) , where $r = d$ and θ is the laser beam angle at the moment of transmission^[4].

$$d = \frac{c\Delta t}{2} \quad (3.1)^{[4]}$$

$$x = r\cos(\theta) \quad (3.2)^{[4]}$$

$$y = r \sin(\theta) \quad (3.3)^{[4]}$$

$$f_{Hz} = \frac{RPM}{60} \quad (3.4)^{[4]}$$

Table 5 - LiDAR Use-Cases^[6]

Update Rate (RPM, Hz)	Use-case
~ 300, 5	Slow robots or mapping purposes
~ 600, 10	Indoor SLAM
~ 900 – 1200, 15 - 20	High-speed navigation

Table 6 below shows the range of constraints LiDAR are susceptible to; it provides the considerations contributing to selecting the optimal algorithm^[3].

Table 6 - 2D LiDAR Constraints^[3]

Constraint/Challenge	Description	Engineering Judgement
<i>Measurement error</i>	Error due to resolution at a distance of 10m with a resolution of 1° , $s = r\Delta\theta = \frac{10\pi}{180} = 17.5cm$ (error)	This means the algorithm cannot depend on the LiDAR for fine details and thus must instead use scan matching to detect landmarks such as walls and corners
<i>Motion Skew (Temporal Distortion)</i>	A LiDAR scan is acquired over time; robot motion during scanning distorts the point cloud	Use high-rate IMU angular velocity to deskew scans into a common reference frame before processing
<i>Corridor Problem</i>	Feature-poor corridors provide weak constraints along the direction of travel	Allow uncertainty to grow along the unconstrained axis and rely on inertial motion estimates
<i>Material Interaction</i>	Specular or absorptive surfaces cause missing or invalid LiDAR returns	Detect sparse scans and temporarily rely on inertial odometry
<i>Dynamic Obstacles</i>	Moving objects introduce measurements inconsistent with the static environment	Apply point-cloud filtering to suppress dynamic elements before localisation.
<i>Gaussian Range Noise</i>	High-frequency noise	Model measurement uncertainty conservatively
<i>Angular Uncertainty</i>	Encoder or timing errors cause bearing jitter in scan points	Minimise through calibration and cautious handling of angular measurements
<i>Systematic Range Bias</i>	Constant offsets shift all range measurements	Correct through calibration to prevent persistent localisation error

LiDAR is the most computationally demanding sensor in this project due to the volume of data points processed per scan^[6]. While the trigonometry for a single pulse is basic, the algorithm is required to map thousands of points against a map, typically limiting its update rate to 5–10Hz on embedded hardware^[6].

Overall, while a 2D LiDAR provides strong information for localisation, it is clear that it cannot be relied upon as a standalone sensing modality for this project. Finite angular resolution leads to increasing errors between points at range, whilst environmental factors such as feature-poor corridors, and dynamic obstacles reduce the observability and

reliability of LiDAR measurements. Consequently, the localisation approach must be designed to treat LiDAR primarily as a correction source.

3.2.2 | IMU

The IMU (Inertial Measurement Unit) is the primary internal sensor in the project's system: it provides high-frequency data on the linear and angular acceleration to model the motion of the robot; this high frequency provides the short-term updates between the LiDAR pulses that are required to make a more accurate estimation of the pose^[2].

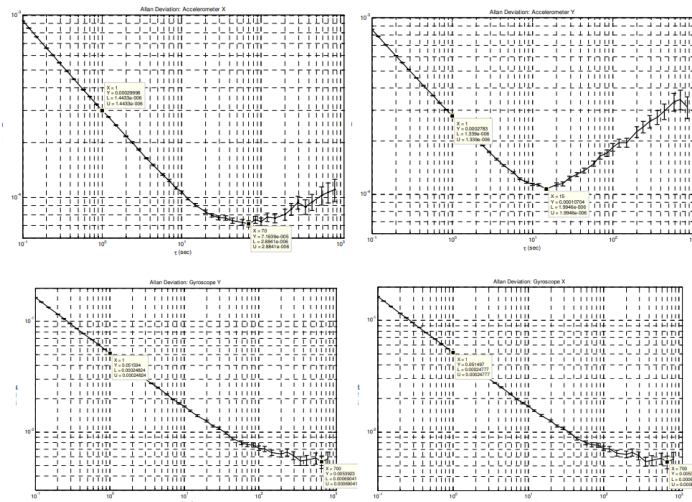


Figure 3 - IMU Accelerometer, Gyroscope Drift^[24]

IMUs are typically categorised as 6-axis or 9-axis. A 6-axis IMU, used in this project, combines a 3-axis accelerometer and a 3-axis gyroscope, while a 9-axis IMU additionally includes a magnetometer to reference the Earth's magnetic field for yaw correction^[4]. However, magnetometer measurements are highly unreliable indoors due to magnetic interference from structural steel, wiring, and electronic equipment, making 9-axis IMUs unsuitable for this project^[3].

Typical update rates for a 6-axis can be seen in *Table 7*; *Table 8* outlines the common sources of error in IMU systems.

Table 7 - 6-Axis IMU Updates Rates^[4]

Component	Update Rate (Hz)	Result
Gyroscope	200–1000	Captures fast rotations smoothly; higher rates reduce aliasing but increase noise.
Accelerometer	200–1000	Resolves short-term motion; high rates amplify vibration and bias effects.
IMU (overall)	100–1000	Provides continuous motion updates between slower external sensors (e.g. LiDAR).

Table 8 - 6-Axis IMU Constraints^[1]

Constraint/Challenge	Description	Engineering Judgement
Gaussian White Noise	High-frequency electronic jitter	Apply low-pass filtering and model noise in the process covariance $\mathbf{Q}$.

<i>Constant Bias</i>	Constant offset when stationary	Perform startup zero-calibration to estimate and remove bias.
<i>Bias Instability (Drift)</i>	Slow bias drift (temperature/vibration)	Estimate sensor bias online and compensate continuously.
<i>Inertial Drift ($E \propto t^2$)</i>	Position drift from double integration	Use IMU for short-term motion only; rely on LiDAR to correct drift.
<i>Unbounded Yaw Drift</i>	Unbounded heading drift (no magnetometer)	Use LiDAR scan matching as a virtual heading reference.
<i>Vibration Interference</i>	Mechanical vibration from motors	Physically isolate the IMU and reject acceleration outliers in software.
<i>Centripetal Acceleration</i>	Centripetal acceleration during turning	Use robot kinematics to separate turning forces from linear motion.

The IMU is the most computationally efficient sensor to model, as its state updates rely on simple integration^[2]. The mathematical simplicity allows for extremely quick processing (200Hz+), making it the primary source for high-frequency motion tracking with insignificant processing demand^[2].

The IMU provides a significantly higher update rate than the LiDAR, enabling continuous short-term motion estimation between slower LiDAR updates that are used to correct accumulated drift. This makes the IMU essential for maintaining smooth and responsive localisation during motion. However, reliable performance depends on properly mitigating error sources such as bias and vibration, which are key considerations in the system design.

3.2.3 | Wheel Encoders

Whilst this project does not contain any wheel encoders, it is still a suitable point of research to benefit the simulation stage^[2].

Most wheel encoders are implemented using quadrature encoders, which employ two phase-shifted channels (A and B) to measure wheel rotation^[2]. By counting amount of ticks the encoder generated per revolution and observing the phase difference between the channels, both wheel speed and direction can be determined^[2]. These are then converted into left and right wheel velocities; these velocities are compared to determine the robot's forward speed, and how fast it is turning can be calculated (4.1), (4.2)^[2].

$$v = \frac{v_R + v_L}{2} \quad (4.1)^{[2]}$$

$$\omega = \frac{V_R - V_L}{L} \quad (4.2)^{[2]}$$

Encoders can correct IMUs in the short term, but also suffer from long-term drift due to wheel slip and surface irregularities, which can cause both noisy measurements and systematic drift in position and heading^[3].

In summary, although the project robot does not include wheel encoders, modelling simple wheel-based odometry during the design and simulation phase provides a lightweight way to test and optimise the IMU approach before integrating LiDAR-based localisation^[2].

3.3 | ALGORITHMS

After reviewing the [Concepts](#) and [Sensors](#) sections of the research, along with the provided specifications, the various components of the project's localisation module were identified. The objective at this stage of the research was to understand how these concepts could be realised in a manner that aligns with the project objectives.

3.3.1 | State Estimation and Sensor Fusion

As mentioned in the [first sub-section](#), the core goal of the localisation module is to determine, as close as possible, the robot's state vector, $\mathbf{x}^{[1]}$. Through the Bayes' framework, measurements are assumed to have a degree of uncertainty about their value, modelled by a Gaussian probability distribution, covariance $\mathbf{P}^{[1]}$. During the sensor fusion process, the algorithm calculates a weighting matrix, $\mathbf{K}$ (Kalman Gain), which determines how much to 'trust' the new measurements^[4]. The various Kalman Filters are essentially just different methods for attaining $\mathbf{K}^{[4]}$. Some different filters taken into consideration can be viewed below:

The Unscented Kalman Filter (UKF) – for non-linear systems that estimates the state by propagating a small set of representative states through the true system equations, avoiding linearization^[5]. High accuracy for highly nonlinear systems, but extensive processing cost makes it unsuitable for the project due to real-time constraints and high-frequency IMU^[5].

Particle Filter (PF) – a state estimation method that is non-parametric (uncertainty does not have a fixed mathematical shape), representing the system state using a large set of weighted samples of the robot's state (particles)^[1]. Excellent at handling non-linear motion and non-Gaussian noise; however, again, at high computational cost^[1].

Complementary Filter (CF) - A simple fusion method that combines sensor data based on frequency characteristics, typically using gyroscope measurements for short-term motion and accelerometer data for long-term stability^[6]. While computationally light, it does not explicitly model uncertainty, making it unsuitable for full robot localisation; however, it may be used for basic orientation estimation in this project^[6].

3.3.1.1 | Extended Kalman Filter

Unlike the standard Kalman Filter, which assumes linear system models, the EKF is designed to handle moderately non-linear systems by locally approximating the non-linear motion (5.1) and observation models (5.2) with linear models about the current state estimate^[4].

$$\mathbf{x}_k = f(\mathbf{x}_{k-1}, \mathbf{u}_k) + \mathbf{w}_k \quad (5.1)^{[4]}$$

$$\mathbf{z}_k = h(\mathbf{x}_k) + \mathbf{v}_k \quad (5.2)^{[4]}$$

where $f()$ and $h()$ represent the non-linear motion and measurement models, respectively, and $\mathbf{w}_k$ and $\mathbf{v}_k$ represent the process noise (uncertainty in how the robot actually moves compared to the motion model) and measurement noise^[4]. These models are then approximated using first-order Taylor series expansions, producing Jacobian matrices that are used to propagate uncertainty and update the pose estimate during measurement correction^[4]. The full formulas can be found in [Appendix C](#).

3.3.1.2 | Error-State Kalman Filter

The Error-State Kalman Filter (ESKF) estimates small differences between a predicted reference state and the robot's actual state, rather than directly estimating the full state^[7]. The robot's nominal state, $\hat{\mathbf{x}}$, is propagated using IMU data, while a separate error state, $\delta\mathbf{x}$, represents small deviations from this prediction^[7].

ESKF Cycle Process:

1. First, (6.1) defines how the true state is formed by taking the predicted state $\hat{\mathbf{x}}$ and applying a small correction $\delta\mathbf{x}$; the $\oplus$ operator means adding a small position correction and applying a small heading correction^[8].
2. (6.2) propagates the reference state using the non-linear motion model $f()$ and the IMU data^[8].
3. In parallel, (6.3) propagates the small error state using a linear approximation, so standard Kalman filtering can be applied to estimate $\delta\mathbf{x}$ when a measurement $\mathbf{z}_k$ becomes available^[8].
4. Finally, the estimated error is injected back via ($\oplus$) to correct the nominal pose and continuously offset bias over time^[8].

$$\mathbf{x} = \hat{\mathbf{x}} \oplus \delta\mathbf{x} \quad (6.1)^{[8]}$$

$$\hat{\mathbf{x}}_k = f(\hat{\mathbf{x}}_{k-1}, \mathbf{u}_k) \quad (6.2)^{[8]}$$

$$\delta\mathbf{x} = \mathbf{F}_k \delta\mathbf{x}_{k-1} + \mathbf{G}_k \mathbf{w}_k \quad (6.3)^{[8]}$$

For this project, both the EKF and ESKF are suitable for fusing IMU data with 2D LiDAR measurements. The EKF estimates the full state directly, while the ESKF estimates small errors about a nominal trajectory, providing improved stability during high-rate inertial integration. Including IMU bias terms allows accelerometer and gyroscope errors to be estimated and corrected over time, improving the reliability of local motion estimates, with LiDAR used to correct accumulated drift. Given the project's moderately linear motion model and real-time constraints, an EKF-based approach (with ESKF as a refinement) is a promising component of the final solution.

3.3.2 | LiDAR Scan Matching

LiDAR scan matching refers to the methods of estimating the robot's motion by comparing consecutive scans to each other or to a reference map^[9]. The aim is to determine the global change in position and orientation between scans, to be used for inertial drift correction^[9].

Below are 2 scan-matching approaches considered:

- *Iterative Closest Point (ICP)* – Aligns scans by continuously matching corresponding points to estimate relative motion^[10]. It is computationally expensive and heavily influenced by the initial pose estimate, making it unsuitable for real-time use in this project^[10].
- *Correlative Scan Matching (CSM)* – Estimates motion by searching over a grid of possible transformations and determining the one that aligns most with the measurement^[11]. Very effective, but the exhaustive search process is computationally intensive and therefore is not ideal for the project's real-time requirements^[11].

3.3.2.1 | Normal Distributions Transform

The Normal Distributions Transform (NDT) estimates the relative pose between LiDAR scans by representing the environment as a set of local normal distributions rather than discrete points (6.1)^[12]. The reference scan is divided into a grid where each box is modelled by a mean and a covariance^[12]. This means that the problem is now simply a likelihood maximisation problem (6.2)^[12]. Consequently, it is less affected by noise and partial overlap (when 2 LiDAR scans only share some of the same landmarks). Computational cost is also reduced as the alignment comparison is done against cell distributions rather than individual points^[12].

$$p(\mathbf{x}) \sim N(\mu_i, \Sigma_i) \quad (6.1)^{[12]}$$

$$L(\mathbf{T}) = \sum_j \log(p(\mathbf{T}\mathbf{x}_j)) \quad (6.2)^{[12]}$$

NDT is appropriate for this project; it is effective for indoor localisation in structured environments and offers a good mix of accuracy and computational efficiency. It is more suitable for real-time deployment than ICP or CSM.

3.3.3 | SLAM Algorithms

Simultaneous Localisation and Mapping (SLAM) algorithms estimate the robot's pose while building a map of the environment at the same time^[1]. Early SLAM systems often rely only on LiDAR scan matching to measure motion; however, purely LiDAR-based approaches are sensitive to motion distortion, sparse features(cause of [aliasing](#)), and poor initial alignment during fast manoeuvres^[13]. As a result, modern SLAM systems also utilise internal sensors to correct this^[13]. A couple of the unsuitable SLAM methods that were considered:

- *Hector* SLAM performs scan matching using only LiDAR data against an occupancy grid without requiring an internal sensor^[14]. While lightweight, it assumes small motion between scans and lacks suitable loop closure; it is not appropriate for agile motion and the long-term consistency required in this project^[14].
- *Cartographer* aligns each new LiDAR scan to a local map to estimate motion and uses loop closures to globally optimise the robot's trajectory^[15]. Highly accurate mapping, but the continuous optimisation required is computationally intensive, making it unsuitable for real-time operation on the project's embedded platform during agile motion^[15].

3.3.3.1 | Graph-Based SLAM

Graph-based SLAM expresses the robot's path as a graph of pose nodes linked by constraints from sensor measurements^[13]. As the robot moves, new poses are added using local motion estimates, while loop closures connect previously visited poses, allowing global optimisation to correct accumulated drift and ensure map consistency^[13].

The primary limitation of graph-based SLAM is computational scalability; a greater map size results in more memory utilised^[13]. This can be problematic for hardware platforms such as the Jetson Nano with limited memory^[16]. Can be mitigated by using sub-mapping and asynchronous optimisation^[13].

For this project, graph-based SLAM aligns well with the sensing and computational restrictions. Motion calculated from IMU data in between LiDAR updates improves scan matching during agile motion and allows for real-time responsiveness.

3.3.4 | Map-Based Localisation

Map-based localisation estimates the robot's pose by matching environmental sensor data against a pre-existing map of the environment^[1]. Unlike SLAM, the map is assumed to be fixed and exists before runtime, allowing the localisation problem to focus solely on estimating the robot's position and orientation within a known reference frame^[2]. This approach avoids the high computational cost that comes with continuous map construction and [global optimisation](#), making it appropriate for real-time operation once a reliable map has been created^[3].

3.3.4.1 | Continuous Scan-to-Map Matching

For this project, the most suitable map-based localisation method is continuous scan-to-map matching using 2D LiDAR, supported by IMU motion estimation^[12]. LiDAR scans are aligned against a pre-built map to provide environment-referenced pose corrections, while the IMU supplies a smooth motion in between updates, allowing for stability in agile motion^[12]. This combination avoids the computational demand of grid-based or particle-based methods; it provides reliable, low-latency localisation^[12].

3.4 | SOFTWARE - ROS 2

3.4.1 | Overview

In modern robotics, the Robot Operating System 2 (ROS 2) serves as the main middleware (layer that sits between application code and operating system) that bridges the software and hardware^[17]. For the project's localisation module, ROS 2 serves not just as a tool but also as an architecture that ensures it can be reproduced and scaled^[17]. ROS 2 removes the central master used in ROS 1 by allowing nodes to discover each other in a distributed manner^[18]. One significant advantage of using ROS 2 in this project is its Data Distribution Service Standard (DDS), which allows for real-time support and cross-platform portability^[19].

3.4.2 | Architecture

ROS program architecture is based on nodes, topics, services and actions^[17]. Each provides a unique function; ultimately, it is a way to complete processes independently and in parallel^[17]. The data pipeline is formed through Publishers and Subscribers, as illustrated in *Figure 1*. [Appendix D](#) displays the ROS architecture^[17].

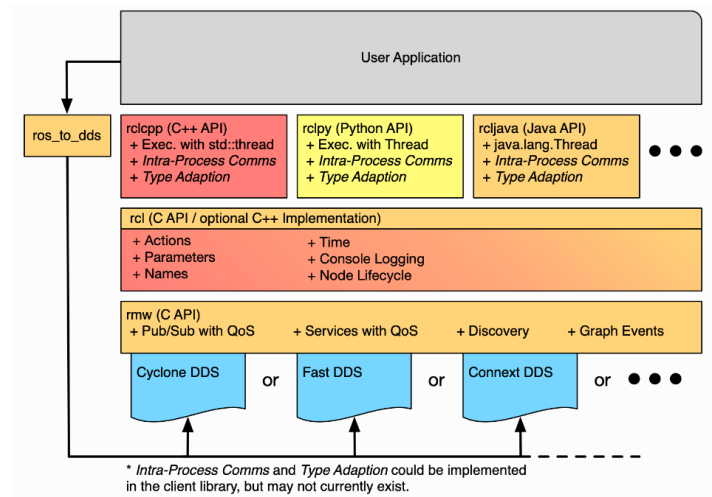


Figure 4 - ROS Architecture^[17]

3.4.3 | Deployment

A significant challenge in the deployment phase of this project is the software compatibility gap between the development environment and the physical hardware^[20]. The NVIDIA Jetson Nano officially supports Ubuntu 18.04, while ROS 2 Humble requires Ubuntu 22.04, making native installation on the Nano impossible^[20]. To resolve this, the project will be deployed using a Docker container^[21].

Although containerisation introduces an extra layer, it achieves almost identical performance (less than 1% difference) by sharing the host Linux kernel, and, with the NVIDIA Container Toolkit, meets the real-time requirements of the 2D LiDAR and IMU^[22].

3.5 | HARDWARE

3.5.1 | Nvidia Jetson Nano

The NVIDIA Jetson Nano serves as the onboard localisation host, selected for its ability to process high-frequency IMU and LiDAR data in real time^[20]. Its parallel processing capability allows simultaneous inertial integration and LiDAR-based correction, minimising latency between the robot's physical motion and estimated pose^[20]. In addition, the Jetson's I/O bandwidth supports multi-modal sensor fusion without CPU bottlenecks^[20], and its compatibility with ROS 2 enables straightforward deployment from the development environment^[17].

3.6 | SUMMARY

This literature review focused on key localisation concepts such as sensing modalities and algorithms for indoor robot localisation. The most appropriate, and common, approach to this localisation problem is a recursive Bayesian state-estimation problem requiring fusion of internal motion data with environment-referenced corrections. Individual sensor limitations were identified, with inertial odometry prone to drift and LiDAR-only approaches vulnerable to sparsity, dynamics, and geometric ambiguity.

Kalman filter-based methods were found to provide an effective balance between accuracy, computational efficiency, and real-time performance. EKF and ESKF approaches were

identified as suitable for IMU–LiDAR fusion, while more computationally intensive methods were deemed impractical. Probabilistic LiDAR scan matching, such as Normal Distributions Transform, was identified as a suitable source of geometric correction.

Overall, the literature supports a localisation architecture driven by IMU prediction, anchored by lower-rate LiDAR corrections within a Kalman filtering framework, forming a justified basis for the localisation module design.

4 | Preliminary Investigation and Technology Evaluation

4.1 | INITIAL DIRECTION AND STRATEGY

We initially planned to evaluate each major component of the localisation pipeline independently, allowing different state estimation, correction, and mapping approaches to be tested in an isolated simulation environment before integration into a final framework. The purpose of this strategy was to determine which methods were most suitable for agile indoor robot navigation whilst maintaining real-time performance and low computational overhead. To support this, a custom ROS 2 simulation environment was developed, where synthetic IMU measurements were generated from a predefined ground-truth motion path and then corrupted with configurable noise and drift characteristics to emulate the behaviour of a real sensor. The robot motion consisted of controlled trajectories such as square traversal, circular motion, and return-to-origin movement, allowing different estimator behaviours to be observed under varying motion conditions. Figure 5 displays a snippet of this simulation environment, showing the generated path and estimated trajectories.

During these tests, wheel odometry was also temporarily integrated to observe the correction characteristics of the filters and compare the behaviour of raw inertial integration against probabilistic filtering approaches. However, due to the overall project time constraints and the increasing complexity of implementing a complete SLAM framework entirely from first principles, the project direction was partially altered after completion of the state estimation analysis. Whilst the filter comparison stage was completed internally, the state correction and mapping stages were then aligned more closely with approaches consistently recommended throughout the literature review and existing high-performance SLAM frameworks.

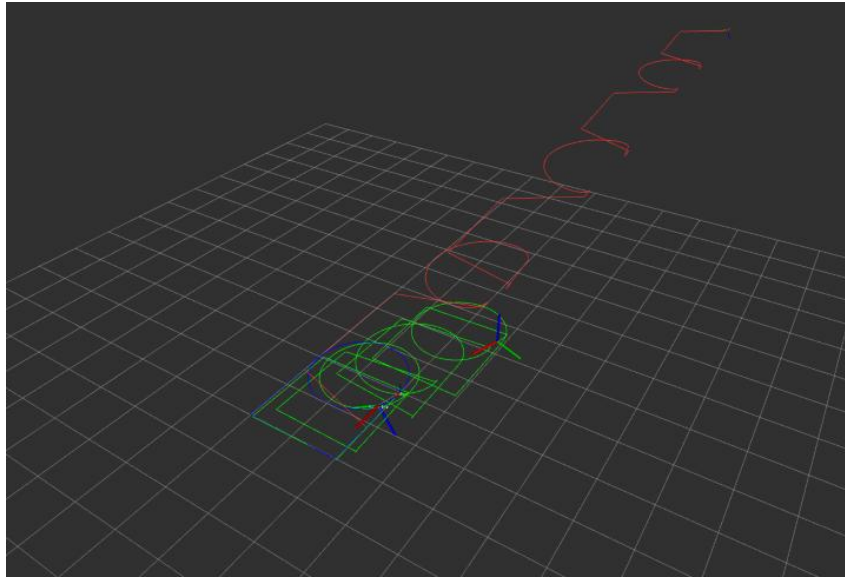


Figure 5 - Experiment Environment

4.2 | INVESTIGATORY WORK

4.2.1 | Correction Investigation

As the sensor data was generated entirely within simulation, a complete ground-truth trajectory was available throughout testing. This allowed direct comparison between the estimated pose and the true robot motion, making it possible to clearly observe positional drift and orientation error over time. Configurable noise and bias were also added to the generated IMU data to better emulate the behaviour of a real inertial sensor.

The initial focus was placed on evaluating the behaviour of the Extended Kalman Filter under two configurations: IMU-only localisation and IMU localisation with wheel odometry correction. As wheel odometry was relatively simple to integrate within the existing simulation framework, it was used as a temporary correction source before progressing towards LiDAR-based correction. Figures 6 and 7 display the resulting trajectories and error plots for both configurations.

During IMU-only operation, the EKF still produced a considerably smoother and more stable estimate than direct raw inertial integration; however, significant positional drift accumulated over time due to inertial error propagation. After wheel odometry correction was introduced, the estimated trajectory remained substantially closer to the ground-truth path, with noticeably reduced long-term drift. These results aligned strongly with the literature review, validating that whilst probabilistic filtering improves localisation stability, an external correction source is still required to constrain long-term inertial drift.

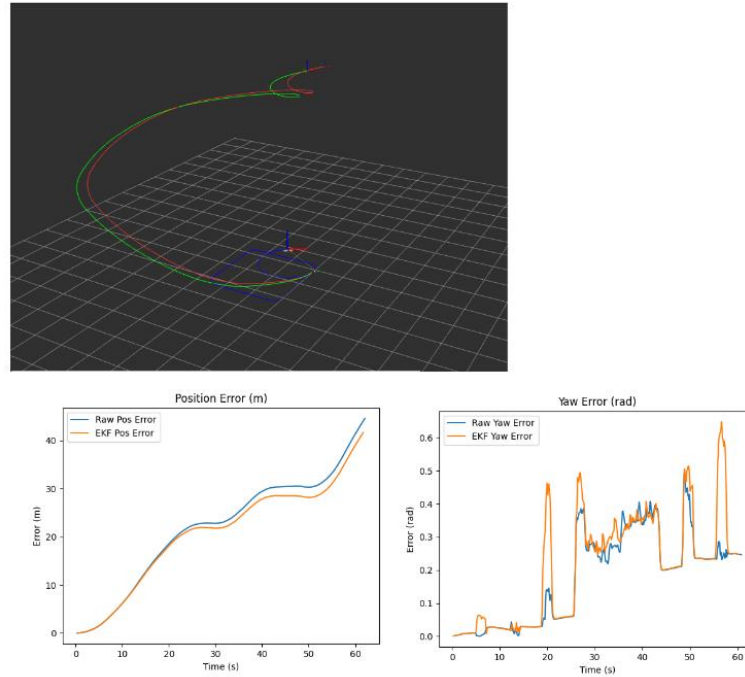


Figure 6 - IMU Only Simulation

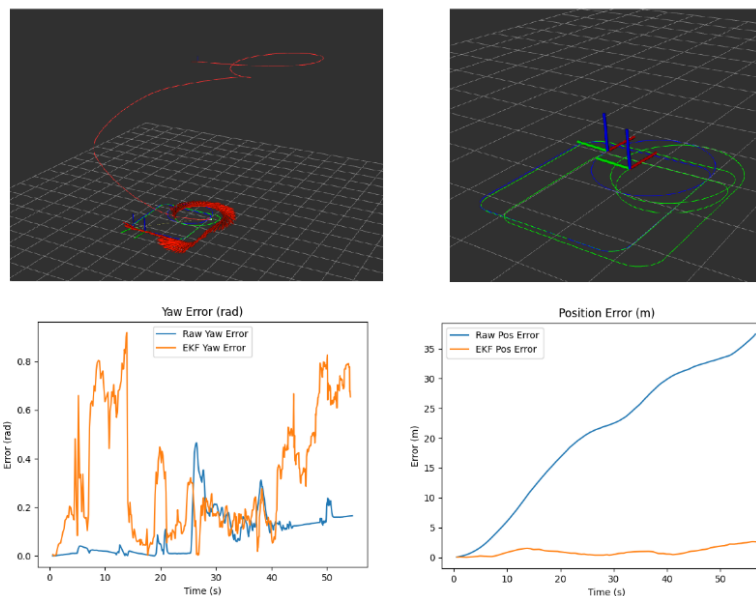


Figure 7 - Wheel Odometry Enabled Simulation

4.2.2 | Filter Investigation

Following the initial EKF validation, additional comparisons were carried out using several built-in filtering approaches under the same simulated motion conditions. The resulting localisation and error data for each filter was exported into CSV format and analysed within MATLAB, as this provided more detailed graphical comparison and error visualisation. The filters compared included the EKF, ESKF, UKF, Complementary Filter (CF), Particle Filter (PF), and direct raw inertial integration. Figures 10 and 11 display the resulting positional and yaw-error comparisons with wheel odometry correction enabled. Overall, all probabilistic filtering approaches performed substantially better than raw inertial integration

and the Complementary Filter, both of which accumulated significant long-term positional drift throughout the trajectory. This aligned strongly with the literature review, where inertial-only approaches were consistently identified as unsuitable for stable long-term localisation. Whilst the UKF and ESKF maintained relatively low positional drift, noticeably noisier yaw behaviour was observed during more aggressive motion segments. The Particle Filter also produced stable results, although with slightly increased noise compared to the EKF. Across both graphs, the EKF consistently produced the most balanced overall behaviour, maintaining the lowest yaw error whilst also heavily constraining positional drift.

These observations aligned closely with the literature survey, where EKF-based localisation was repeatedly identified as a robust and computationally efficient solution for embedded robotic localisation systems. Based on these results, the EKF was selected as the primary state estimation filter moving forward, whilst the final SLAM framework was aligned towards NDT-based LiDAR odometry and keyframe mapping.

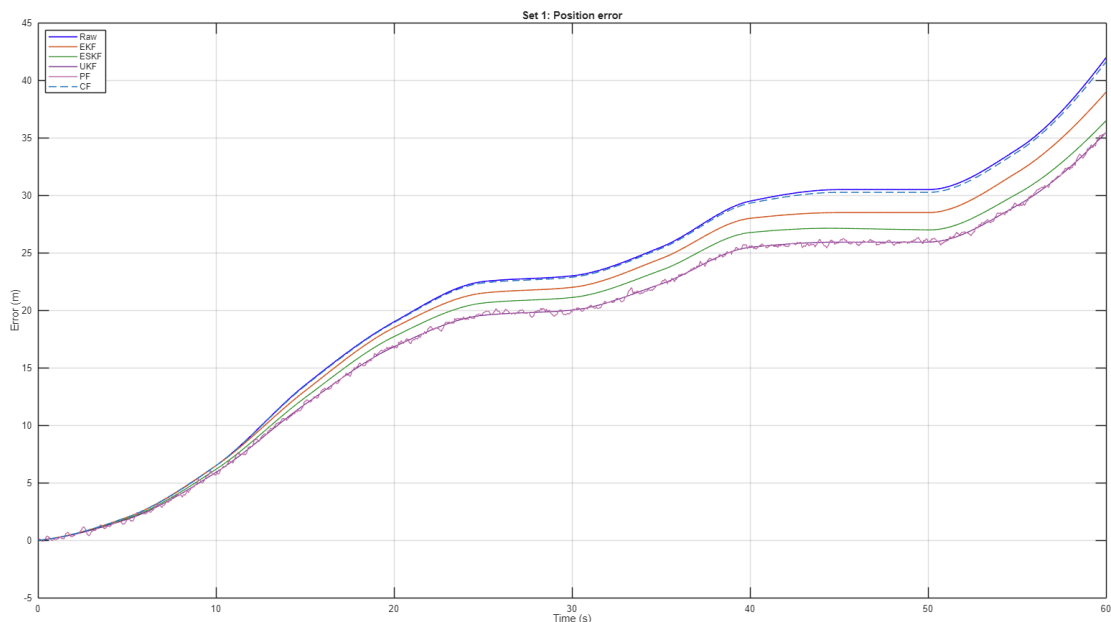


Figure 8 - Filter Position Error, No Odom

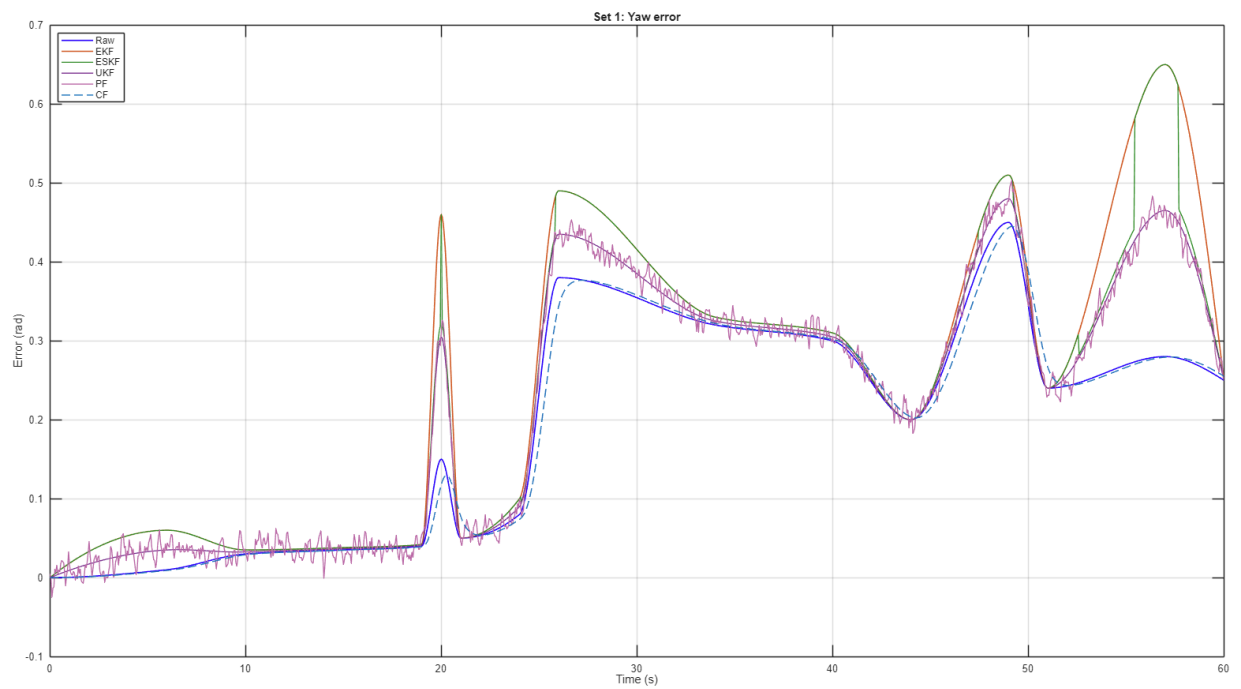


Figure 9 - Filter Orientation Error, No Odom

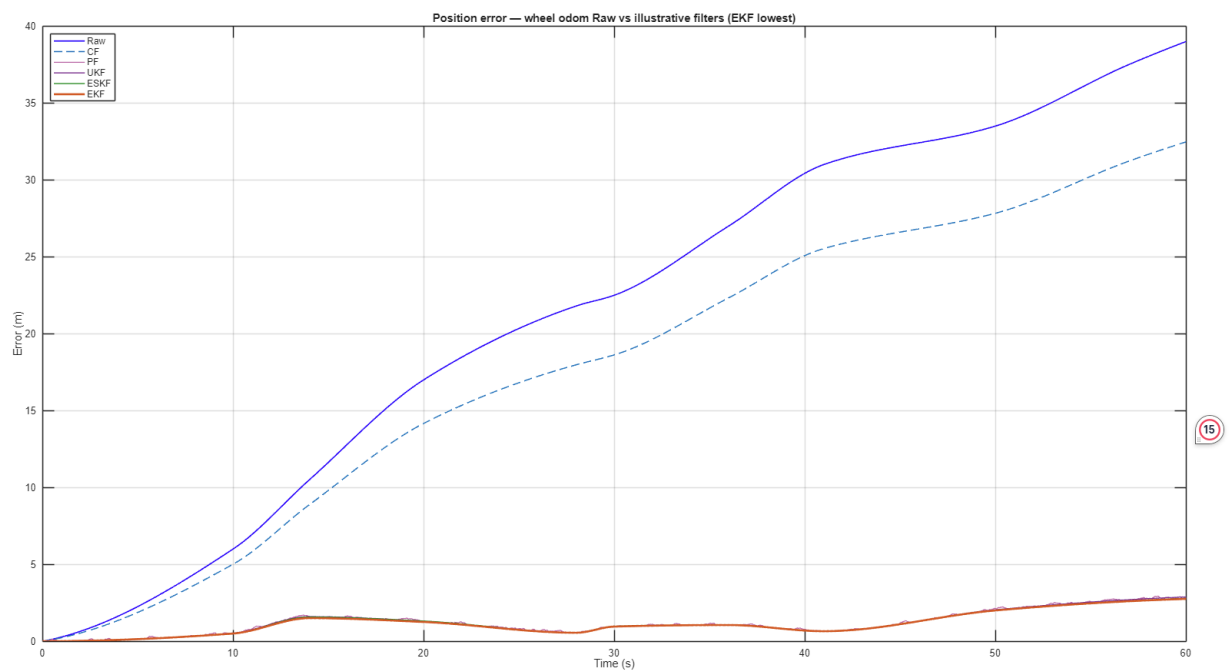


Figure 10 - Filter Position Error, Odom

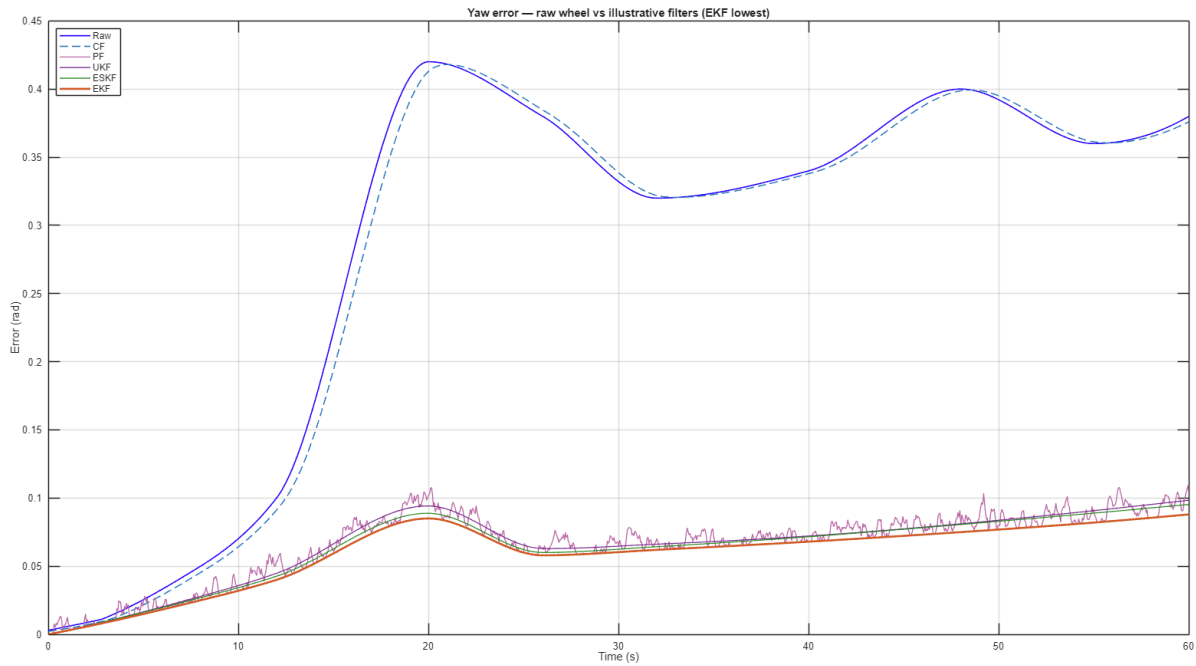


Figure 11 - Filter Orientation Error, Odom

4.2.3 | Resulting SLAM Framework

Following completion of the preliminary filter evaluation stage, the resulting SLAM framework was selected primarily from the findings and recommendations identified throughout the literature review, alongside the EKF selection determined during the previous simulation testing. The final proposed framework therefore consisted of an EKF-based state estimation pipeline combined with NDT LiDAR odometry and keyframe-based map accumulation; referred to throughout the remainder of the project as the EKF-NDT SLAM framework.

Within this architecture, high-rate IMU measurements are first used for continuous EKF prediction, whilst lower-rate LiDAR scans are processed through NDT scan matching to generate odometry corrections. These corrected pose estimates are then used for keyframe-based map accumulation. Figure 12 displays the resulting high-level SLAM workflow selected for further implementation and deployment.

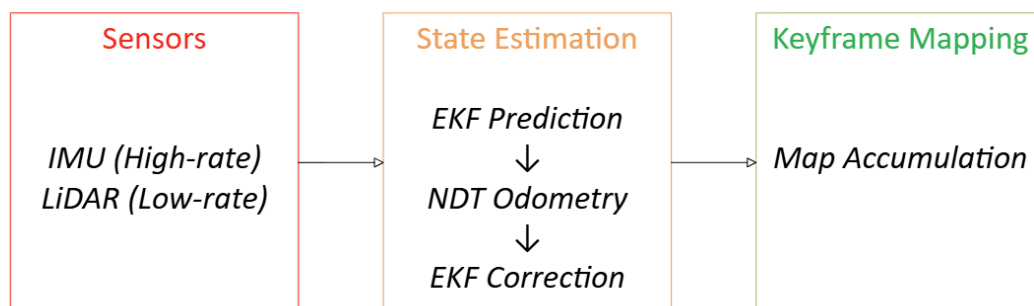


Figure 12 - Proposed Framework

4.3 | FAST-LIO

After researching potential frameworks that could provide a suitable baseline during development, we identified the FAST-LIO repository developed by HKU-MARS as a strong candidate for rapid SLAM deployment and evaluation. FAST-LIO is a tightly-coupled LiDAR-Inertial Odometry framework that integrates high-rate IMU propagation with LiDAR-based correction and mapping within a complete real-time SLAM pipeline. Due to its strong performance, minimal initial tuning requirements, and compatibility with the selected Livox LiDAR hardware, the framework performed reliably almost immediately after deployment within the ROS 2 environment. As a result, FAST-LIO was adopted as a practical baseline system throughout development, allowing comparison between the custom localisation framework and an already established high-performance SLAM solution. Figure 13 displays a high-level overview of the FAST-LIO processing architecture. It's GitHub repository can be found [here](#).

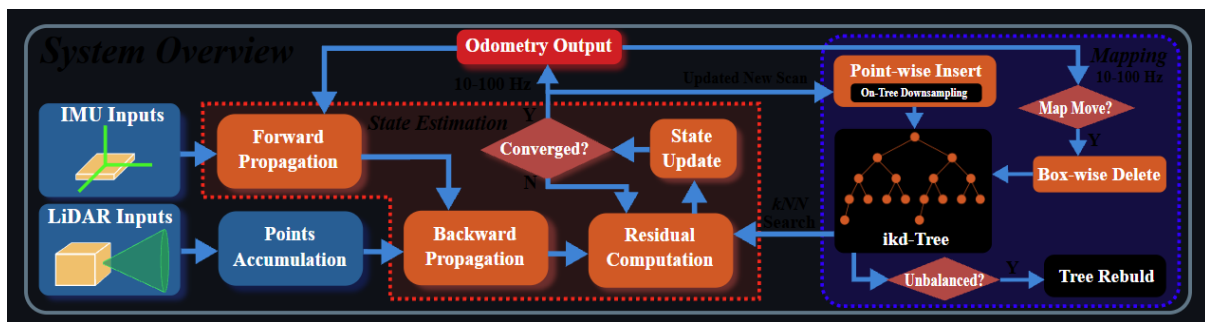


Figure 13 - FAST-LIO Pipeline^[27]

4.4 | HARDWARE PLATFORM EVALUATION

4.4.1 | Jetson Platforms

A key requirement of the project was that the localisation framework had to remain deployable across both older and newer embedded robotic compute platforms. As a result, two NVIDIA Jetson platforms were targeted throughout development: the Jetson TX2i and the Jetson Orin Nano.

The Jetson TX2i represented the legacy embedded hardware already integrated within the robotic platform and therefore formed an important deployment requirement for backwards compatibility. However, due to platform support limitations, the TX2i is primarily constrained to Ubuntu 18.04 and older ROS 2 distributions. Alongside this, the Jetson Orin Nano was selected as the modern deployment target, supporting Ubuntu 22.04 and ROS 2 Humble directly whilst also providing substantially greater computational performance.

Supporting both platforms throughout development therefore acted as a proof of concept that the localisation framework could remain functional across both legacy and modern robotic infrastructure.

4.4.2 | ROS Versions

The ROS 2 distribution used throughout the project was dependent on the operating-system and driver support constraints of the selected embedded platforms. Due to the Ubuntu 18.04 requirement of the Jetson TX2i, deployment on this platform required the use of an older ROS 2 installation, dependent on final driver compatibility. In comparison, the Jetson Orin Nano supported ROS 2 Humble under Ubuntu 22.04, which became the primary development environment during later implementation stages.

As a result, another project requirement became maintaining compatibility across multiple ROS 2 environments, ensuring that the localisation framework remained portable between both old and new deployment systems.

4.5 | SUMMARY

Overall, this section established the primary development direction for the remainder of the project. The preliminary testing, literature analysis, and platform evaluation collectively guided the selection of the resulting EKF-NDT SLAM framework, following the probability-based Bayesian localisation framework discussed throughout the literature review. Within this architecture, the EKF performs high-rate state prediction using IMU measurements, whilst NDT LiDAR odometry provides external correction updates to reduce long-term drift accumulation. These corrected pose estimates are then used within a keyframe-based mapping framework for environment accumulation. The investigations within this section also defined the deployment constraints associated with the embedded hardware and ROS 2 environments, providing both the technical justification and practical foundation for the final system architecture and implementation approach used throughout the remainder of the project.

5 | System Design

5.1 | SPECIFICATIONS

Table 4 displays the high-level system specifications established for the proposed localisation and SLAM framework. These specifications define the intended operating environment, sensing configuration, software architecture, deployment platform, and overall system requirements used throughout development.

Table 9 - System Specification

Category	Specification
Robot Platform	Traxxas X-Maxx autonomous RC platform
Operating Environment	Indoor agile navigation
Localisation Method	LiDAR-IMU sensor fusion
LiDAR Sensor	Livox MID-360
IMU Configuration	External IMU + integrated LiDAR IMU
State Estimation	Extended Kalman Filter (EKF)
LiDAR Odometry	NDT-based scan matching
Mapping Method	Keyframe-based point cloud mapping

Middleware	ROS 2 Humble
Embedded Compute Platform	NVIDIA Jetson Orin Nano
Secondary Platform	NVIDIA Jetson TX2i
Runtime Requirement	Real-time onboard operation
Visualisation	RViz 2
Communication	DDS / FastRTPS
Development Workflow	Simulation-first deployment approach
Software Architecture	Modular ROS 2 node structure
Deployment Target	Fully onboard SLAM operation
Primary Outputs	Pose estimate, odometry, keyframe map
Validation Method	Simulation and real-world testing
Integration	Create standalone package with another 3 rd year student

5.2 | HIGH-LEVEL SYSTEM OVERVIEW

The resulting localisation framework follows a modular ROS 2 based SLAM architecture consisting of sensor acquisition, probabilistic state estimation, LiDAR odometry, mapping, and visualisation layers. Figure 14 displays the high-level processing architecture of the final system.

The system primarily operates from two sensor inputs: the Livox MID-360 LiDAR and the Microstrain GX5-3DM-25 IMU. These sensors interface with the ROS 2 middleware through their respective driver nodes, which publish the required sensor streams and TF information into the ROS ecosystem. The IMU measurements provide the primary high-rate motion information for state propagation, whilst the LiDAR data provides lower-rate environmental measurements used for external correction.

Following the Bayesian localisation framework discussed previously, the EKF state filter performs the prediction stage of the localisation pipeline using inertial measurements from the IMU. Due to the high update rate of the IMU, this allows continuous estimation of the robot pose during aggressive motion and between LiDAR updates. However, as inertial prediction alone accumulates drift over time, external correction is required to constrain long-term positional and orientation error.

This correction stage is performed through NDT LiDAR odometry, where consecutive LiDAR scans are matched against the environment to estimate relative motion updates. These odometry estimates are then fused within the EKF framework to provide probabilistic correction of the predicted state estimate. The resulting corrected pose estimates are subsequently passed into the keyframe mapping system, where scans are accumulated into a persistent environment representation for SLAM visualisation and localisation consistency.

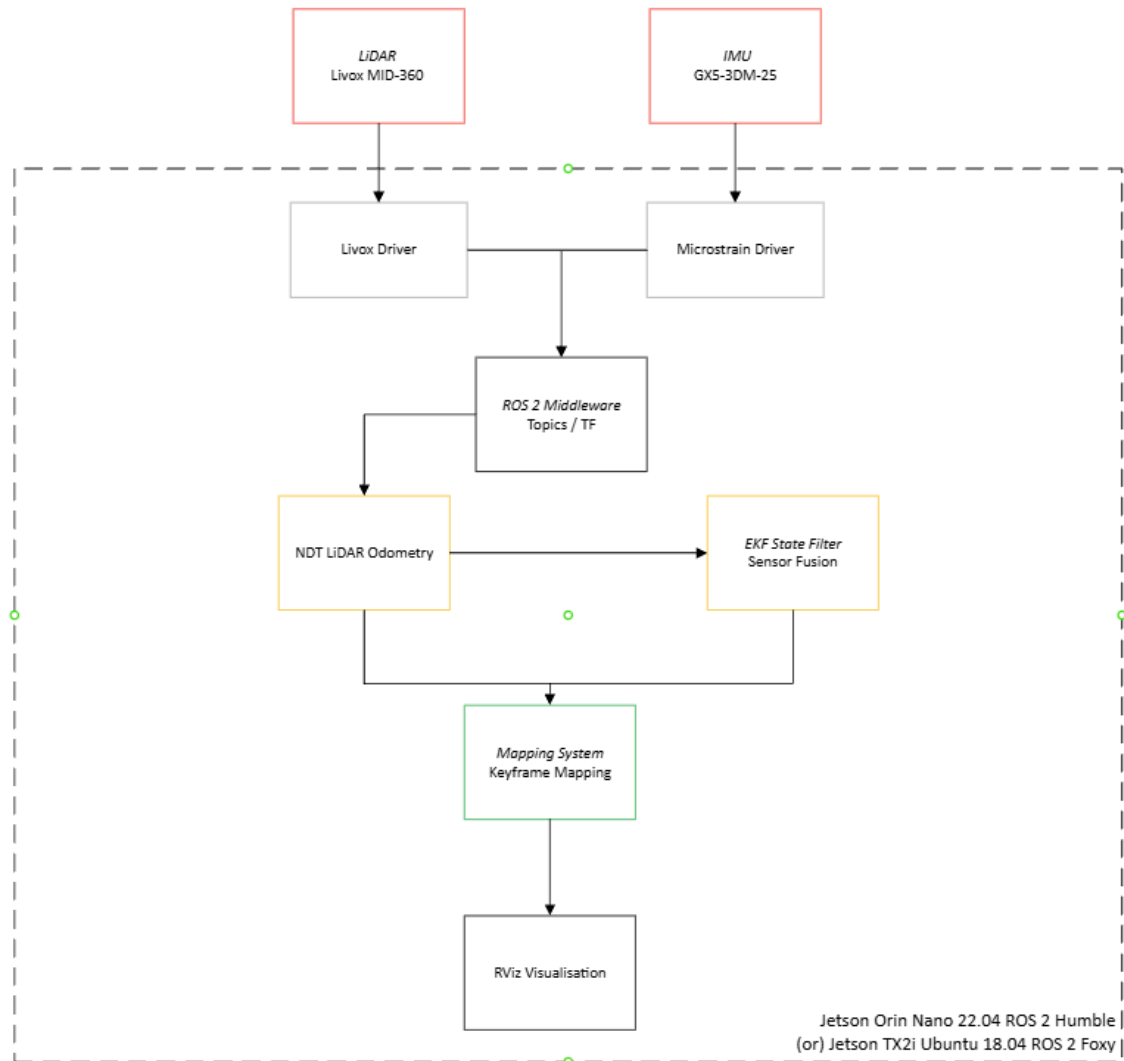


Figure 14 - System Design High-Level Overview

Alongside the custom EKF-NDT framework, the FAST-LIO package was also integrated within the workspace as an alternative tightly-coupled LiDAR–Inertial SLAM solution. FAST-LIO was primarily utilised as a baseline and fallback deployment framework for systems where implementing and tuning the custom NDT pipeline would be impractical due to hardware limitations, deployment constraints, or time restrictions. The inclusion of both frameworks also improved overall system modularity, allowing different localisation pipelines to be enabled or substituted depending on the deployment requirements whilst maintaining the same overall ROS 2 architecture and sensor interfaces.

5.3 | HARDWARE DESIGN

5.3.1 | Overall Hardware Architecture

Figure 15 displays the overall hardware architecture of the developed localisation and mapping subsystem. The system was designed around an existing Traxxas mobile robot platform, which already provided the primary chassis, drive system, and base power infrastructure. Rather than permanently modifying the entire robotic platform, the localisation

framework was engineered as a semi-independent subsystem consisting of the embedded compute hardware, sensing hardware, power distribution, and communication interfaces.

The primary processing unit consisted of either the Jetson Orin Nano running Ubuntu 22.04 with ROS 2 Humble, or alternatively the Jetson TX2i operating under Ubuntu 18.04 with ROS 2 Foxy depending on deployment requirements. The embedded system interfaced directly with the Livox MID-360 LiDAR through Ethernet communication and the Microstrain GX5-3DM-25 IMU through USB connectivity. Both sensors were powered through the portable onboard power system, alongside the wireless communication adapter used for remote access and debugging during development.

A major design objective of the hardware architecture was modularity and ease of deployment. As a result, all localisation and mapping components were mounted directly onto a dedicated localisation subsystem frame rather than being permanently integrated throughout the entire robot chassis. This created an effective “drop-in” subsystem, where only the localisation frame itself required securing to the target robotic platform. This significantly reduced implementation complexity whilst also allowing the complete sensing and compute stack to be removed, serviced, upgraded, or transferred between robotic systems with minimal modification.

This semi-permanent architecture also proved highly beneficial during initial software development and testing stages, as the entire localisation subsystem could remain fully assembled and operational outside of the final robot platform. This simplified debugging, sensor calibration, wiring management, and deployment iteration throughout development whilst maintaining a cleaner and more maintainable hardware integration overall.

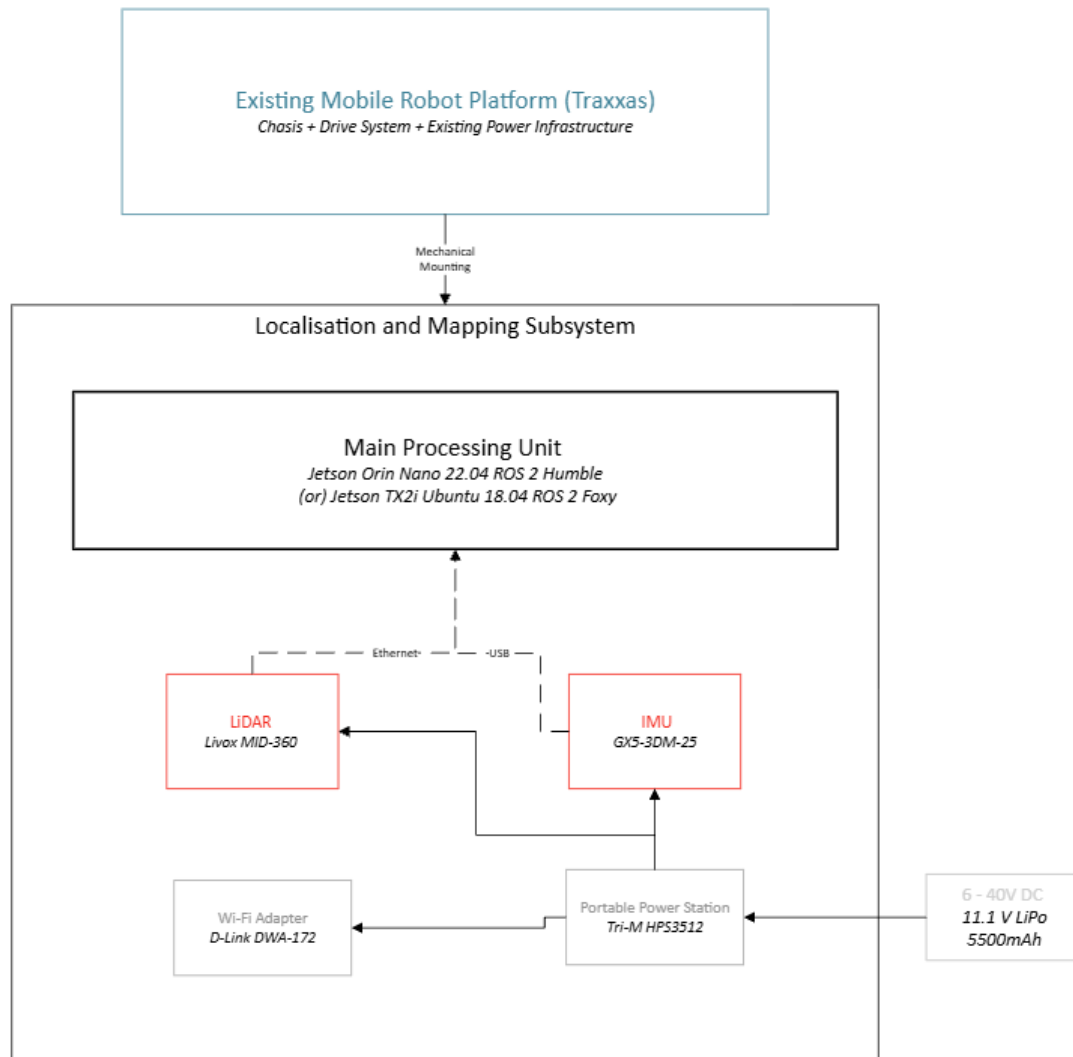


Figure 15 - Hardware Architecture Design

5.3.2 | Circuit Design

Figure 16 displays the electrical and communication architecture used throughout the localisation and mapping subsystem. The circuit design was intentionally kept relatively simple, with the primary objective being reliable power distribution and stable communication between all sensing and processing devices. Despite the simplicity of the architecture, correct implementation was still critical, as instability within either the power or communication system would directly affect localisation reliability during operation.

The Tri-M HPS3512 portable power station acted as the centralised power distribution hub, supplying regulated power to the selected embedded processing unit, LiDAR sensor, and supporting peripherals. Both the Jetson TX2i and Jetson Orin Nano configurations were supported through a switched power arrangement, allowing either platform to be enabled depending on deployment requirements whilst maintaining the same subsystem layout.

Communication interfaces were separated depending on device requirements. The Livox MID-360 LiDAR interfaced directly through Ethernet due to the high-rate point cloud throughput required for real-time SLAM processing, whilst lower-bandwidth devices such as the Microstrain IMU and wireless adapter interfaced through the USB hub architecture.

A major engineering consideration throughout the design was maintaining modularity and reducing implementation complexity. Centralising both power distribution and communication routing simplified cable management, improved serviceability, and allowed easier debugging and subsystem reconfiguration throughout development.

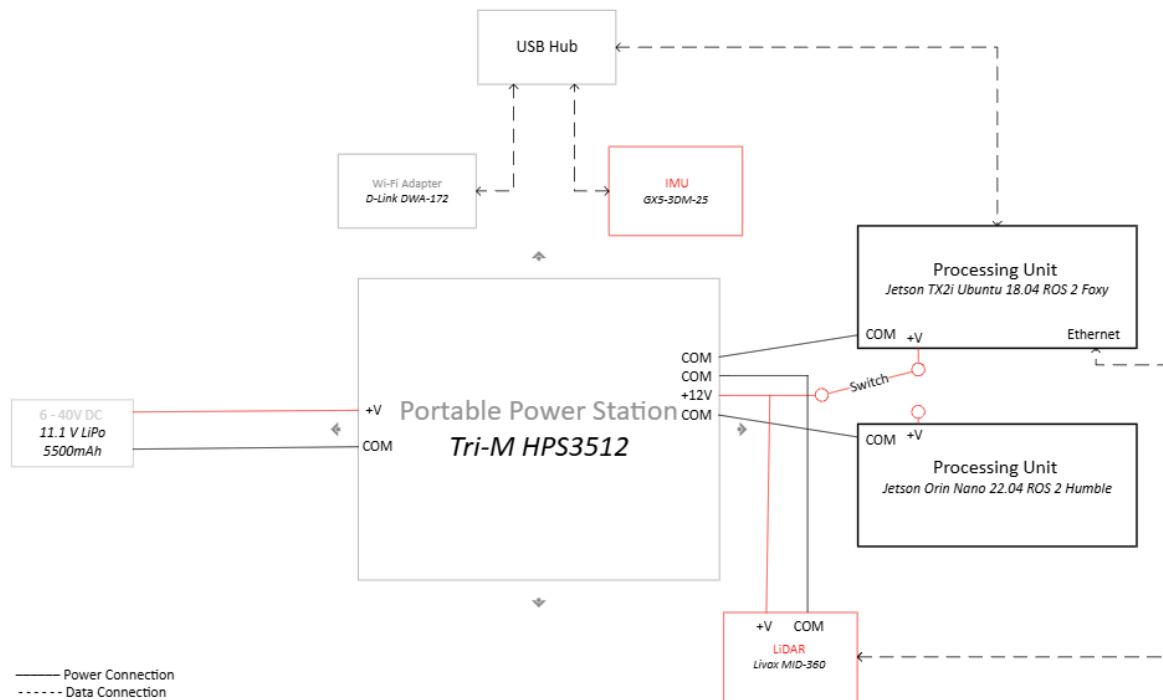


Figure 16 - System Circuit Design

5.3.3 | Existing Mounting Structure

Figure 17 displays the electrical and communication architecture used throughout the localisation and mapping subsystem. The circuit design was intentionally kept relatively simple, with the primary objective being reliable power distribution and stable communication between all sensing and processing devices. Despite the simplicity of the architecture, correct implementation was still critical, as instability within either the power or communication system would directly affect localisation reliability during operation.

The Tri-M HPS3512 portable power station acted as the centralised power distribution hub, supplying regulated power to the selected embedded processing unit, LiDAR sensor, and supporting peripherals. Both the Jetson TX2i and Jetson Orin Nano configurations were supported through a switched power arrangement, allowing either platform to be enabled depending on deployment requirements whilst maintaining the same subsystem layout. We researched the dimensions and mounting structure for the IMU and LiDAR Sensor.^{[50][51]}

Communication interfaces were separated depending on device requirements. The Livox MID-360 LiDAR interfaced directly through Ethernet due to the high-rate point cloud

throughput required for real-time SLAM processing, whilst lower-bandwidth devices such as the Microstrain IMU and wireless adapter interfaced through the USB hub architecture.

A major engineering consideration throughout the design was maintaining modularity and reducing implementation complexity. Centralising both power distribution and communication routing simplified cable management, improved serviceability, and allowed easier debugging and subsystem reconfiguration throughout development.

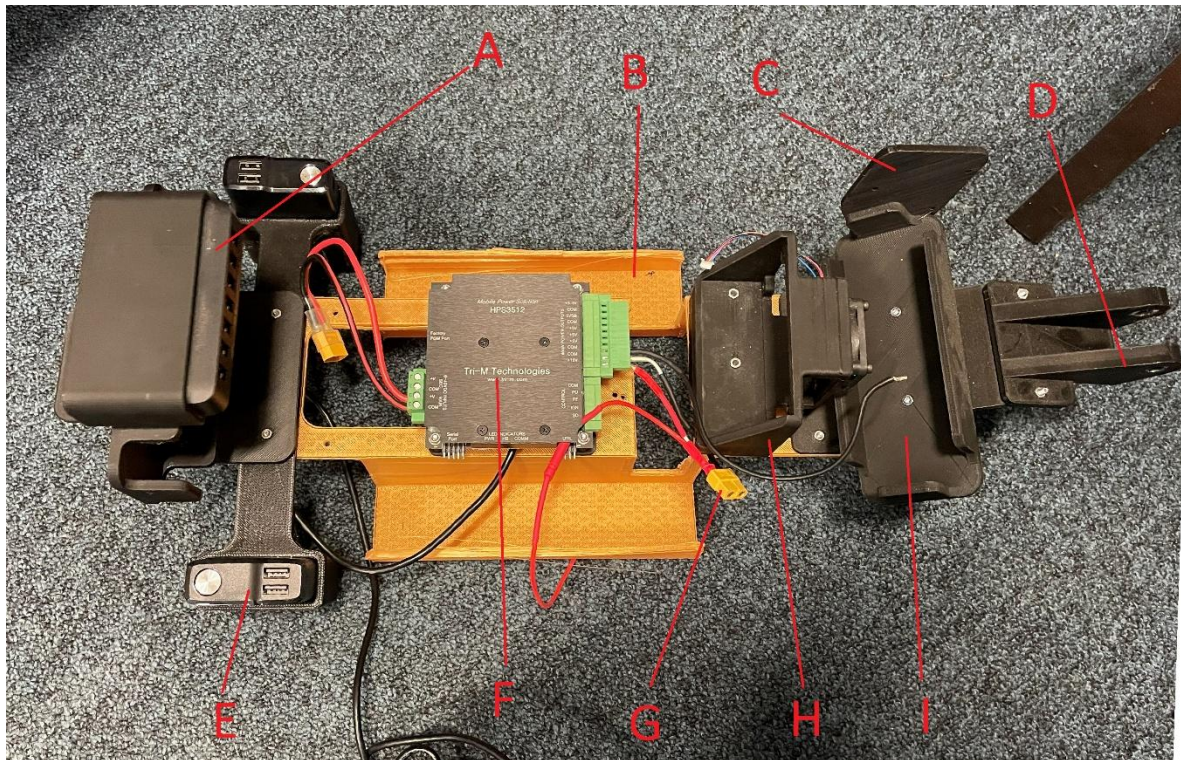


Figure 17 – Original Inner Enclosure

Table 10 -Existing Hardware Mount Identification

Label	Component Description
A	USB Hub + Mount + Battery Mount
B	Battery Mount and Core
C	PWM Driver Mount
D	Camera Mount
E	Power Pack + Mount
F	Power Station
G	Power Connectors
H	TX2i Mount + Fan
I	Additional Battery Mount

An additional component of the original mounting structure was a large 3D-printed outer enclosure which unified the sensing and mounting assemblies into a cleaner integrated structure whilst also improving cable management and subsystem protection. As shown in Figure 18, the enclosure had originally been designed around a different sensing configuration consisting of a 2D LiDAR mount and RTK-GPS antenna positioned on the

upper surface of the robot. The figure also displays proposed alteration areas identified during redesign for integration of the revised localisation hardware.

The enclosure itself was secured to the robot using only four primary mounting screws, with two positioned on either side of the chassis. This proved completely sufficient for normal operation and development use throughout the project. However, from an engineering perspective, more substantial securing methods would likely be required if the platform were to be deployed within more aggressive or demanding operational environments in the future.

Overall, reusing the existing enclosure significantly reduced fabrication complexity, as the majority of the large-scale structural integration had already been completed before development began.



Figure 18 – Original Outer Enclosure

5.3.4 | Mechanical Measurements

5.3.4.1 | *Original*

Figures 20 and 21 display the primary dimensions of the original robotic platform and outer enclosure structure. These measurements were primarily included for scale and reference purposes throughout the mechanical integration stage. As the existing mounting subsystem already provided more than sufficient space for the required sensing and compute hardware, the overall robot dimensions themselves did not significantly influence the engineering judgement or design decisions throughout development. Instead, development focused primarily on appropriately modifying and extending the existing mounting structure to support the revised localisation framework components.

The dimensions of the internal mounting assembly were omitted at this stage, as the structure remained fully contained within the overall footprint of the robot platform itself and therefore did not introduce additional spatial constraints.

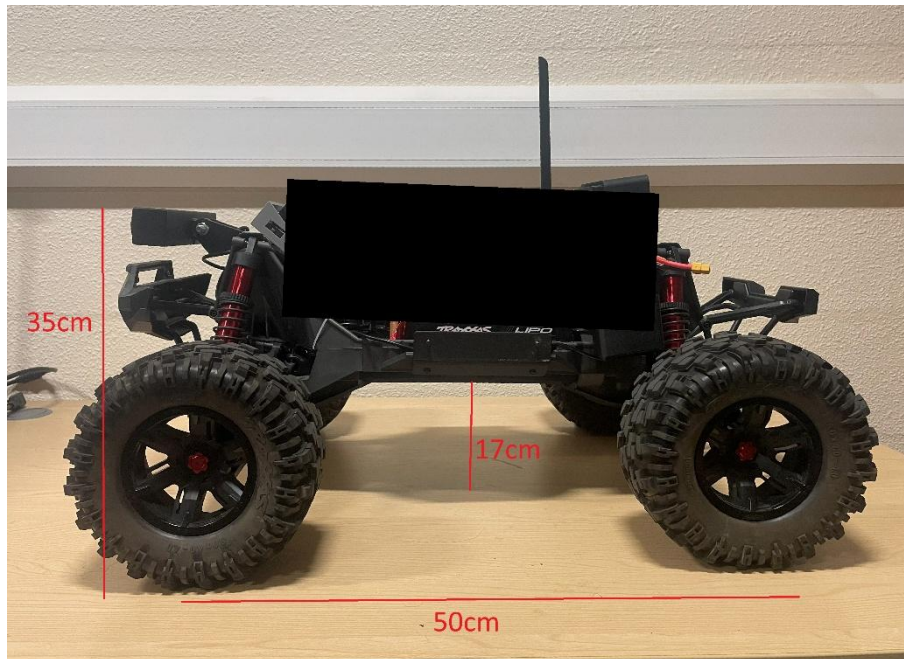


Figure 19 - Robot Dimensions

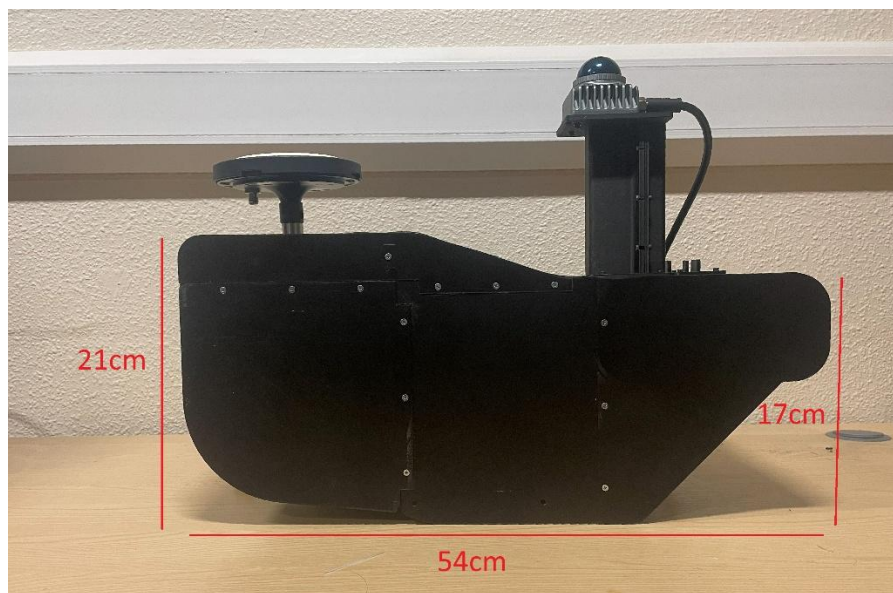


Figure 20 - Outer Enclosure Dimensions, Sagittal Plane

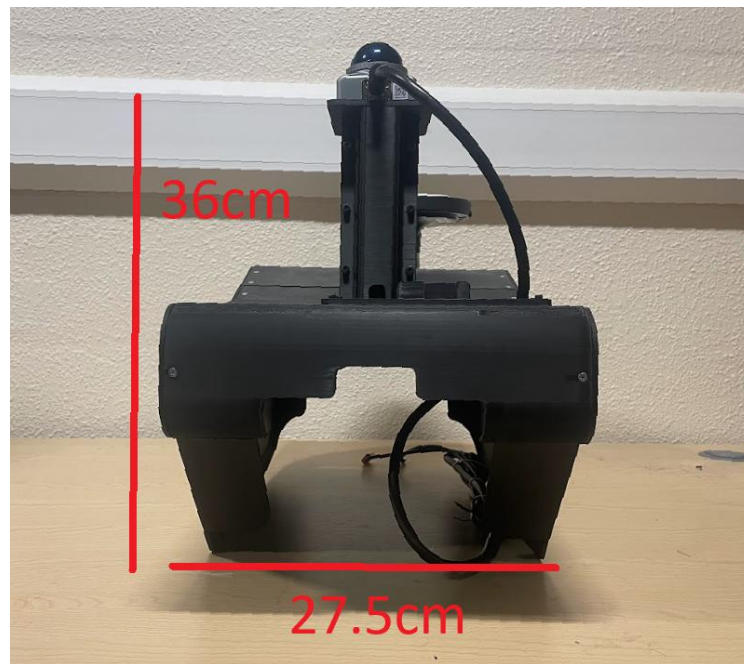


Figure 21 - Outer Enclosure Dimensions, Frontal Plane

5.3.4.2 | Alteration Design

After removal of the portable power station, a suitable mounting region was exposed beneath the existing subsystem structure for integration of the Microstrain GX5-3DM-25 IMU. Figure 22 displays the proposed mounting location, where the blue markers indicate the additional mounting holes required for integration.

The selected placement area proved highly suitable from both a mechanical and sensing perspective. The spacing between the existing mounting nuts measured approximately 4 cm, allowing the IMU body width of 36.6 mm to fit comfortably between the structures whilst still leaving suitable mounting clearance. The device itself measured 36.0 mm × 36.6 mm × 11.1 mm and utilised a four-hole mounting arrangement with an approximate 30 mm square hole spacing. As a result, four additional 2.5 mm mounting holes were proposed to support direct fastening of the IMU assembly using M2.5 hardware.

From an engineering perspective, positioning the IMU beneath the power station and near the approximate centre of the robot chassis was considered highly beneficial. Locating the sensor closer to the centre of mass reduces rotational amplification effects and minimises acceleration disturbances caused by suspension movement during aggressive motion. This positioning therefore helps reduce the likelihood of distorted inertial measurements and improves overall state estimation consistency. A fixed rotational offset would additionally be introduced within the software configuration to correctly align the IMU reference frame with the robot coordinate system.

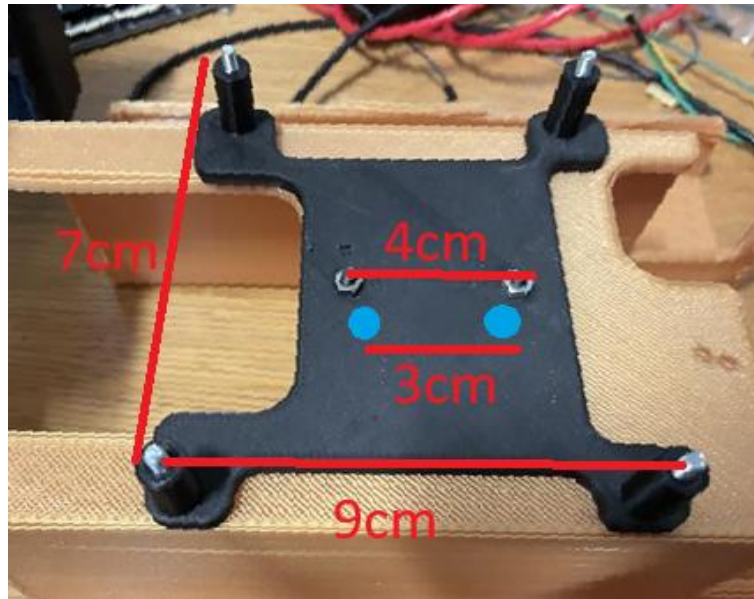


Figure 22 - IMU Mount Design

The original LiDAR enclosure assembly already contained a primary elevated sensor mount designed for integration of the previous 2D LiDAR system. As shown in Figures 23 and 24, the structure provided an elevated mounting platform approximately 15 cm above the enclosure surface, with a mounting plate area of approximately 7 cm × 6 cm. This existing structure provided a highly suitable starting point for integration of the Livox MID-360 LiDAR.

The Livox MID-360 itself measured approximately 65 mm × 65 mm × 60 mm and utilised a four-point M3 mounting arrangement with an approximate 60 mm square hole spacing. During measurement evaluation, it was observed that one axis of the original mounting holes aligned suitably with the required Livox mounting pattern; however, the perpendicular spacing did not match the required dimensions. As a result, only two additional mounting holes were required along one axis of the mounting plate to allow direct fastening of the Livox sensor. The proposed additional drill locations are illustrated in Figure 24 by the blue markers.

Reusing the existing elevated mounting structure simplified the overall integration process considerably, as the LiDAR already remained positioned above the primary enclosure and away from major scan obstructions caused by the robot chassis itself. The original 2D LiDAR assembly had additionally been enclosed within a surrounding casing; however, this approach was intentionally not replicated for the Livox MID-360. Due to the higher thermal output of the sensor and its integrated heatsink-based cooling design, enclosing the device would unnecessarily restrict airflow and reduce heat dissipation during prolonged operation.

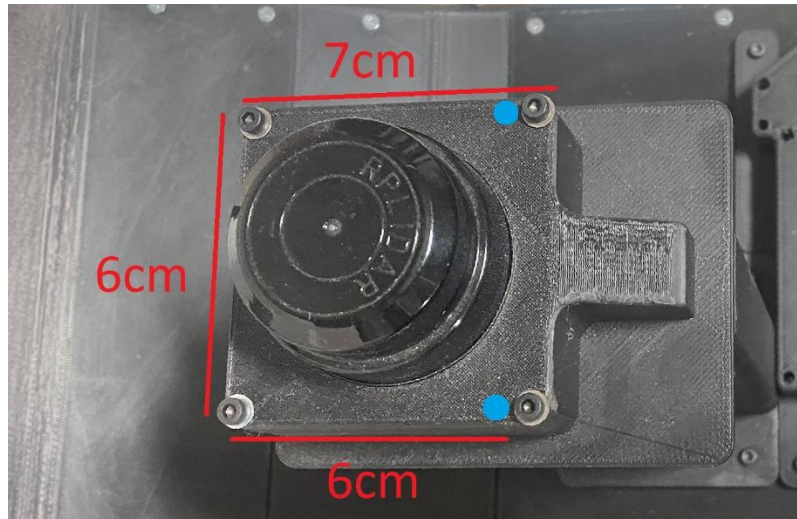


Figure 23 - LiDAR Mount Transversal Plane Measurements

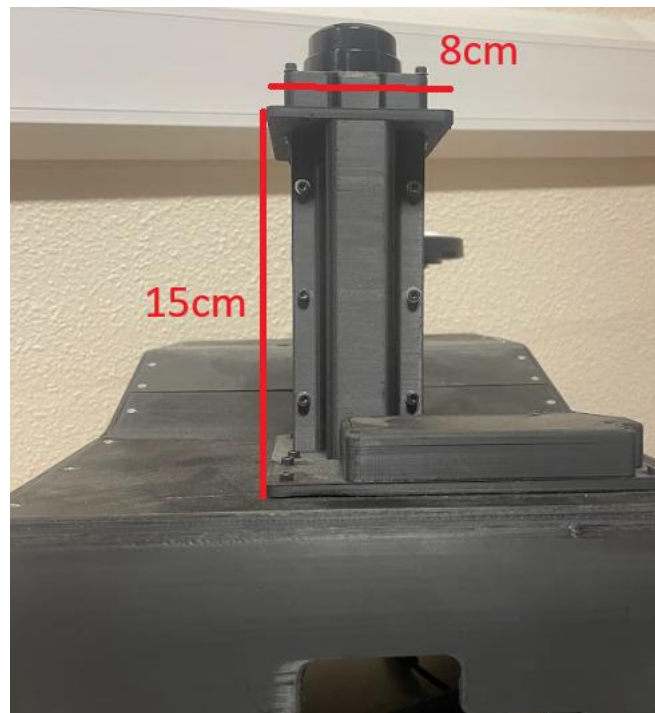


Figure 24 - LiDAR Mount Frontal Plane Measurements

Unlike the existing TX2i configuration, there was no suitable predefined mounting location available for the Jetson Orin Nano within the original subsystem structure. To resolve this, the original frontal battery holder assembly (Item I from Figure 17) was removed, as the additional battery was unnecessary for the final implementation, and a single primary power source was sufficient for the localisation subsystem requirements.

After removal of the holder, the exposed frontal mounting region was measured and determined to provide more than adequate space for integration of a dedicated Jetson Orin Nano mount whilst also maintaining the existing cable routing region within the same section of the robot. This helped minimise unnecessary cable extension, tangling, and wiring complexity throughout the subsystem.

As shown in Figure 24, the existing mounting screw spacing measured approximately 79 mm × 51.5 mm, allowing the original fixture points to be reused directly for securing the new mounting structure. Reusing these existing screw locations simplified integration considerably and avoided unnecessary modification of the surrounding chassis structure.

A small mounting lift of approximately 7 mm was additionally required to raise the Jetson Orin Nano above the surrounding frame geometry and provide suitable clearance for the embedded board and cooling assembly. The suitability of the mounting region was validated using the frontal enclosure dimensions established previously within Figure 25, confirming sufficient available integration volume for the Jetson subsystem.

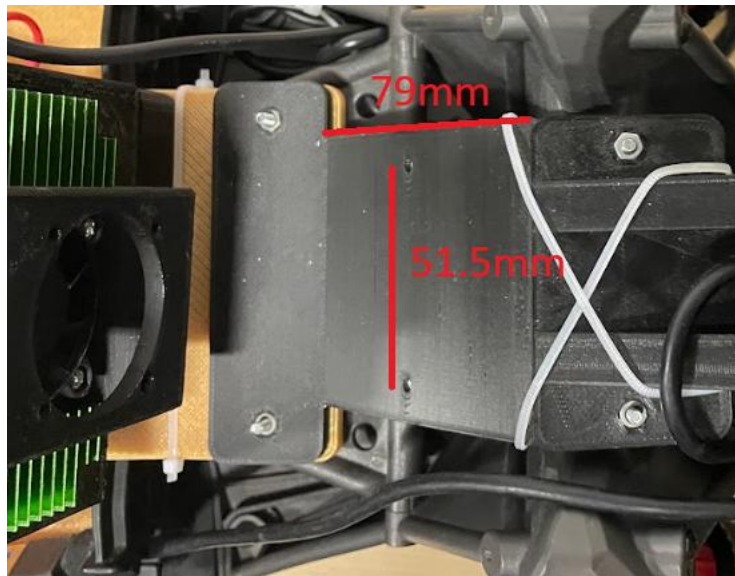


Figure 25 - Jetson Orin Nano Mount Location Dimensions

5.3.5 | 3D Enclosure Design

Unlike the previous mounting alterations, integration of the Jetson Orin Nano required the development of a fully custom 3D-modelled enclosure and mounting structure. The enclosure was designed within Autodesk Fusion using the physical board dimensions, mounting geometry, and connector placement provided within the official device documentation. Figure 26 displays the completed dimensional drawing for the enclosure assembly.

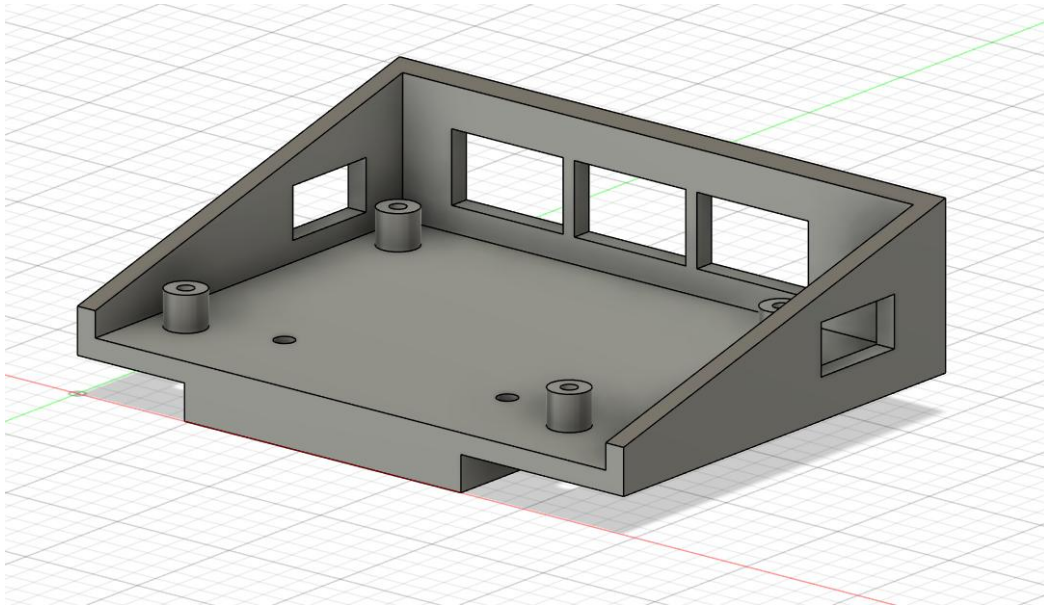


Figure 26 - Jetson Orin Nano Mount

The Jetson Orin Nano developer kit itself measures approximately $100\text{ mm} \times 79\text{ mm}$, which comfortably fit within the available frontal mounting region identified previously during the mechanical measurement stage. The enclosure dimensions and mounting geometry were therefore designed specifically around both the Jetson board footprint and the previously measured mounting-hole spacing already available on the robotic platform. Proper mounting standoffs and screw clearances were incorporated directly into the design to allow secure fastening of the embedded compute hardware whilst maintaining suitable board clearance above the mounting surface.

As shown in Figures 26-27, the enclosure geometry was intentionally designed with tapered side walls and multiple ventilation openings along the enclosure surfaces. This reduced unnecessary airflow obstruction around the embedded compute hardware whilst also allowing easier cable routing and connector access during deployment. The side openings additionally allowed connected cables to remain seated more securely without excessive bending or strain around the enclosure edges.

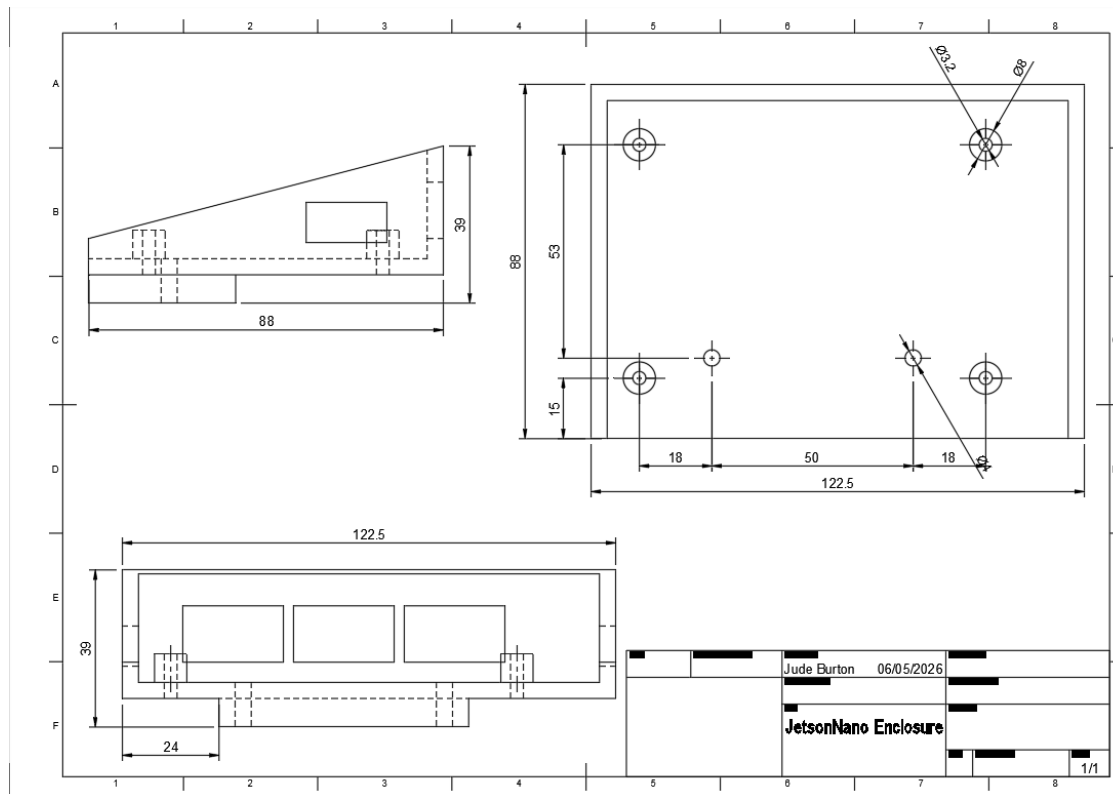


Figure 27 - Jetson Orin Nano Mount, Full Schematic

Following completion of the CAD model, a preliminary static stress analysis was conducted within Autodesk Fusion to evaluate the structural suitability of the enclosure under expected operational loading conditions. The enclosure material was approximated using ABS plastic properties, whilst fixed constraints were applied at the primary mounting interfaces used to secure the assembly to the robotic platform. A downward static force representing the approximate loading from the embedded compute hardware was then applied across the internal mounting surface.

The resulting simulation indicated minimal structural deformation, with a maximum displacement of approximately 0.078 mm observed near unsupported regions of the enclosure, as shown within the displacement analysis in Figure 28. Safety factor analysis additionally indicated that the structure remained well within acceptable operating limits under the expected loading conditions. Although the simulation represented a simplified approximation and did not account for dynamic vibration or manufacturing tolerances, the results suggested that the enclosure provided sufficient rigidity and structural stability for lightweight robotic deployment.

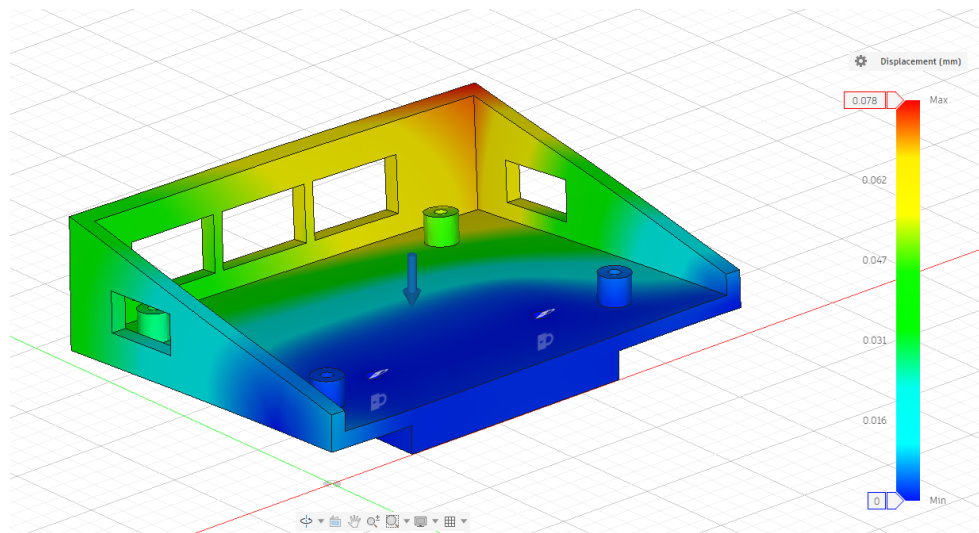


Figure 28 - Jetson Orin Nano Enclosure Displacement Analysis

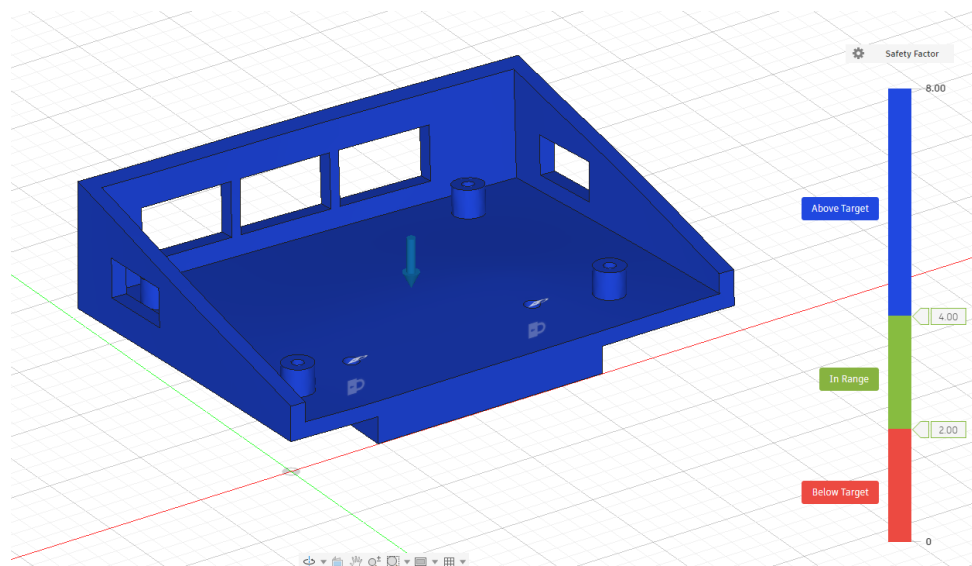


Figure 29 - Jetson Orin Nano Factor Analysis

5.4 | SOFTWARE DESIGN

5.4.1 | Software Architecture Overview

Figure 30 displays the high-level software architecture implemented throughout the localisation and mapping framework. The system was designed using a modular ROS 2 node-based architecture, allowing the sensing, estimation, mapping, and visualisation components to remain logically separated whilst still communicating through the ROS middleware layer. The software pipeline begins with the input layer, where the Livox MID-360 LiDAR and Microstrain IMU publish sensor data through their respective ROS driver packages. The IMU provides high-rate inertial measurements used for continuous motion propagation, whilst the LiDAR provides lower-rate environmental measurements for external correction and mapping.

The front-end estimation stage consists of the NDT LiDAR odometry node, responsible for scan matching and relative motion estimation between consecutive LiDAR frames. These odometry updates are then fused within the EKF-based state estimation node, where inertial

prediction and LiDAR correction are combined to generate a probabilistic estimate of the robot pose, following the Bayesian localisation framework discussed previously throughout the literature review. The corrected pose estimates are subsequently passed into the back-end mapping layer, where keyframe-based scan accumulation is performed to generate a persistent environment representation. Additional pose-graph functionality was also provisioned within the architecture for future extension towards loop closure and global optimisation.

The final layer consists of the visualisation subsystem, providing real-time observation through RViz and ROS TF transforms. This allowed monitoring of sensor frames, estimated robot motion, odometry behaviour, and generated map structures throughout deployment and testing. Maintaining separation between the major ROS 2 nodes also improved overall modularity and deployment flexibility, simplifying debugging and allowing alternative localisation frameworks such as FAST-LIO to be integrated into the same overall system architecture with minimal modification.

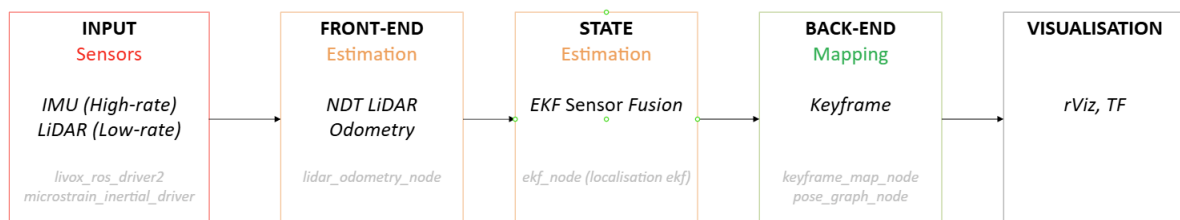


Figure 30 - Software Overall Architecture

5.4.2 | Processing Pipeline

Figure 31 displays the primary runtime processing pipeline implemented throughout the localisation framework. The pipeline begins with the Livox MID-360 LiDAR and Microstrain IMU drivers, which publish the `/livox/lidar` and `/imu/data` sensor streams into the ROS 2 middleware layer. The LiDAR point cloud stream is then passed into the NDT LiDAR odometry node, where scan matching is performed to estimate relative motion between consecutive LiDAR frames. This produces the `/lidar/odom` odometry estimate, which acts as the primary correction source for the localisation framework.

Alongside this, the IMU data stream is continuously fused within the EKF node to provide high-rate state prediction between LiDAR updates. The EKF therefore acts as the primary probabilistic state estimation layer, combining inertial prediction with LiDAR odometry correction to generate the final filtered pose estimates published through `/ekf/odom` and `/ekf/pose`. These corrected estimates are then passed into the keyframe mapping subsystem, where scans are accumulated into a persistent environment representation before being visualised through RViz and the ROS TF framework.

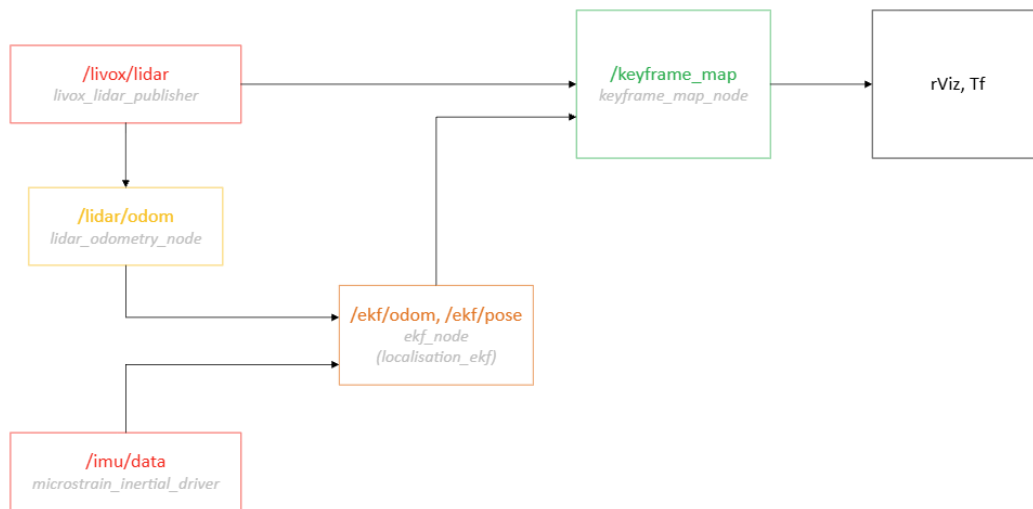


Figure 31 - Primary Processing Pipeline

Figure 32 displays the average runtime frequencies observed throughout testing for the primary ROS topics within both stationary and moving conditions; the frequency of the primary data. The Livox LiDAR consistently operated at approximately 10 Hz, whilst the Livox IMU operated above 200 Hz. The Microstrain IMU similarly approached approximately 200 Hz during motion. These update rates aligned well with the intended localisation architecture, where high-rate inertial prediction is combined with lower-rate environmental correction to maintain stable real-time localisation performance.

Topic file suffix	Stationary	Moving
<code>_livox_lidar</code>	average rate: 9.946	average rate: 9.946
<code>_livox_imu</code>	average rate: 218.122	average rate: 218.122
<code>_imu_data</code>	average rate: 52.549	average rate: 199.549
<code>_lidar_odom_raw</code>	no average rate line	no average rate line
<code>_lidar_odom</code>	average rate: 10.013	average rate: 10.013
<code>_ekf_odom</code>	average rate: 18.663	average rate: 18.663
<code>_tf</code>	average rate: 49.019	average rate: 49.019

Figure 32 - Primary ROS Topics

5.4.3 | Modular Software Approach

The localisation framework was developed using a modular ROS 2 package architecture, allowing the major localisation, odometry, mapping, and deployment components to remain logically separated throughout development. Figure 33 displays the resulting workspace package structure, whilst Figure 34 illustrates the dependency relationship between the major

runtime packages. Table 6 additionally summarises the responsibility of each primary package within the overall system.

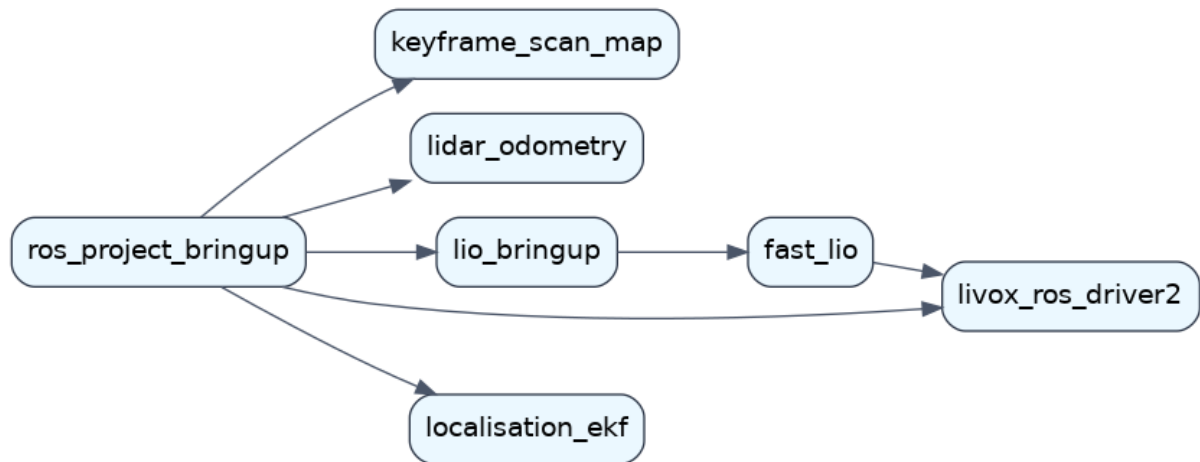


Figure 33 - Package Dependency Tree

Table 11 - Packages Detail

Package	Responsibility
ros_project_bringup	Central launch orchestration
localisation_ekf	State estimation
lidar_odometry	LiDAR odometry
keyframe_scan_map	Mapping
lio_bringup	FAST-LIO integration
livox_ros_driver2	Sensor driver interface

The workspace was separated into dedicated subsystem packages, including 'localisation_ekf' for probabilistic state estimation, 'lidar_odometry' for NDT scan matching, 'keyframe_scan_map' for map accumulation, and 'ros_project_bringup' for overall launch orchestration. Additional integration packages such as 'lio_bringup', 'FAST_LIO_ROS2', and 'livox_ros_driver2' provided support for the alternative FAST-LIO localisation pipeline and the Livox sensing hardware.

This modular structure significantly simplified debugging, tuning, and deployment throughout development, as individual components could be modified or tested independently without heavily affecting the remainder of the system. It also allowed the custom NDT LiDAR odometry pipeline to be swapped with the FAST-LIO framework using only launch parameters and configuration changes, rather than requiring major restructuring of the overall software architecture. Figure 33 additionally demonstrates the modular ROS 2 package structure used throughout implementation. Packages such as localisation_ekf, lidar_odometry, and keyframe_scan_map remain logically separated to improve maintainability, debugging, and deployment flexibility. The ros_project_bringup package acts as the primary orchestration layer responsible for launch configuration and runtime coordination.

The Microstrain IMU driver package was not contained directly within the workspace itself, as it was installed through the ROS 2 Humble environment and accessed as an external ROS dependency during runtime. Maintaining this modular workspace structure simplified debugging, deployment, and future extensibility throughout development.

```
src/
├── localisation_ekf/
├── lidar_odometry/
├── keyframe_scan_map/
├── ros_project_bringup/
├── lio_bringup/
├── livox_ros_driver2/
└── FAST_LIO_ROS2/
```

Figure 34 - Workspace Structure

5.4.4 | ROS 2 Framework and Node Architecture

The localisation framework was implemented entirely within the ROS 2 ecosystem using a distributed node-based architecture. Figure 35 displays the active node graph during runtime operation of the complete SLAM framework.

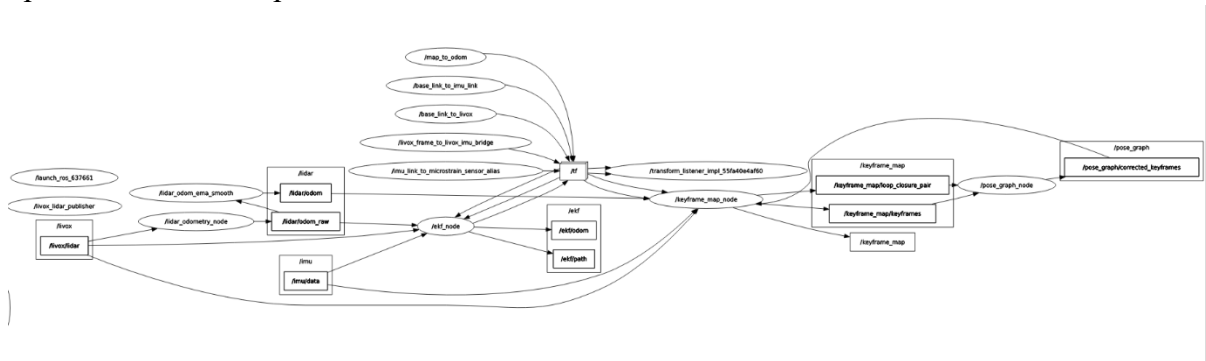


Figure 35 - Full System Pipeline Graph

The runtime architecture was separated into sensing, estimation, mapping, transformation, and visualisation layers. Sensor acquisition nodes published the primary LiDAR and inertial measurement streams into the ROS middleware layer, where the LiDAR odometry subsystem generated relative motion estimates through scan matching before fusion within the EKF state estimation node. Supporting processing nodes were additionally introduced to stabilise LiDAR odometry behaviour prior to downstream fusion and mapping.

The mapping subsystem accumulated corrected scans into a persistent environmental representation, whilst additional pose-graph functionality was provisioned for future loop closure and trajectory optimisation capability. Static transform publishers maintained frame consistency throughout the ROS TF tree and ensured alignment between the differing sensor

coordinate conventions used throughout the localisation stack. Visualisation and offline testing functionality were provided through RViz and ROS bag playback, allowing repeatable validation, debugging, and development testing throughout implementation. Table 7 is the list of active nodes and their primary responsibilities within the localisation stack.

Table 12 - Active Nodes Functionality

Node	Role
livox_lidar_publisher	Publishes raw LiDAR point clouds (/livox/lidar) and embedded IMU data (/livox/imu) from the Livox MID-360 sensor
microstrain_inertial_driver	Publishes high-grade inertial measurements (/imu/data) from the Microstrain GX5-25 IMU
lidar_odometry_node	Performs LiDAR-based odometry estimation using scan matching and publishes motion estimates (/lidar/odom)
ekf_node	Fuses inertial and LiDAR odometry data to generate a consistent robot state estimate (/ekf/odom, /ekf/pose)
keyframe_map_node	Accumulates registered scans into a keyframe-based environmental map (/keyframe_map)
pose_graph_node	Performs loop closure optimisation and trajectory correction for improved global consistency
lidar_odom_ema_smooth	Applies smoothing/filtering to LiDAR odometry outputs before downstream fusion
base_link_to_livox	Static transform publisher defining the LiDAR frame relative to the robot base frame
base_link_to_imu_link	Static transform publisher defining the external IMU frame relative to the robot base frame
livox_frame_to_livox_imu_bridge	Aligns Livox LiDAR and embedded IMU coordinate frame conventions
imu_link_to_microstrain_sensor_alias	Provides frame aliasing and compatibility between Microstrain frame conventions and the ROS TF tree
map_to_odom	Publishes the global map-to-odometry transform used by the localisation stack
rviz2	Provides runtime visualisation of sensor data, odometry, TF frames, and generated maps
rosbag2_player	Replays recorded ROS 2 bag files for offline testing and validation

5.4.4.1 | Sensor Drivers

The stack uses two sensor driver nodes: livox_lidar_publisher (from livox_ros_driver2) and microstrain_inertial_driver. livox_lidar_publisher provides the primary LiDAR stream (/livox/lidar) and optional Livox IMU data (/livox/imu), which upstream localisation and mapping depend on for scan matching and motion estimation. microstrain_inertial_driver provides higher-grade inertial measurements (for example /imu/data or /imu/data_raw) used by the estimator for prediction. Both drivers depend on correct hardware configuration

(device/network settings and frame IDs) and act as the raw data entry points to the ROS graph.

5.4.4.2 | Localisation Node

The localisation front-end is `lidar_odometry_node` (or `lio_odom_relay_node` when FAST-LIO mode is used). `lidar_odometry_node` consumes point clouds (and TF prior information) to estimate frame-to-frame/platform motion and publishes LiDAR odometry outputs such as `/lidar/odom` (and in this project also `/lidar/odom_raw`). This node exists to provide geometry-based motion updates that remain informative even when IMU-only propagation drifts. In simple terms, this node estimates how the robot has moved between consecutive LiDAR scans.

5.4.4.3 | Mapping Node

The mapping component is centered on `keyframe_map_node` (with optional `pose_graph_node` for correction). `keyframe_map_node` subscribes to cloud and odometry/TF inputs, selects/accumulates keyframes, and publishes map/keyframe products (for example `/keyframe_map` and `/keyframe_map/keyframes`). `pose_graph_node` consumes keyframe loop/constraint information and publishes corrected trajectories (for example `/pose_graph/corrected_keyframes`), improving global consistency over long runs. This effectively allows the robot to gradually build a reusable representation of the environment as it moves.

5.4.4.4 | State Estimation Node

The state estimator is `ekf_node` from `localisation_ekf`. It depends on inertial inputs (`/imu/data` or `/livox/imu`) plus LiDAR odometry corrections (`/lidar/odom` or `/lidar/odom_raw`) and fuses them into a smoother, consistent robot state. It publishes fused outputs such as `/ekf/odom`, `/ekf/pose`, and `/ekf/path`, and provides the core `odom -> base_link` transform used by the rest of the stack. This provides a more stable estimate than relying on any individual sensor independently. Without this layer, many ROS components would interpret the robot orientation and sensor positions differently, leading to inconsistent localisation behaviour.

5.4.4.5 | Processing Nodes

Supporting processing nodes in this pipeline include static transform publishers (e.g., `map_to_odom`, `base_link_to_livox`, `base_link_to_imu_link`, `livox_frame_to_livox_imu_bridge`, `imu_link_to_microstrain_sensor_alias`) and optional relay/smoothing utilities depending on launch mode. These nodes do not create new sensing modalities; instead, they standardize coordinate frames, align sensor conventions, and condition message flow so localisation, estimation, and mapping nodes can interoperate reliably.

5.4.5 | ROS Communication Architecture

Communication throughout the localisation framework was handled entirely through the ROS 2 publish-subscribe middleware architecture. Sensor data, odometry estimates, transforms, maps, and visualisation outputs were exchanged between nodes using ROS topics and TF

transforms, allowing each subsystem to operate independently whilst still remaining synchronised within the overall localisation pipeline.

The primary sensing topics consisted of the LiDAR point cloud stream (/livox/lidar) and inertial measurement streams (/livox/imu and /imu/data). These measurements formed the primary inputs to the localisation stack, where LiDAR odometry estimates generated by lidar_odometry_node were fused within ekf_node to generate corrected robot pose estimates. The resulting fused outputs were then consumed by the mapping and visualisation subsystems to generate the final SLAM representation.

Alongside the topic-based communication architecture, the ROS TF framework was heavily utilised to maintain coordinate frame consistency throughout the system. Static transform publishers defined the physical relationship between the robot base frame and the sensing hardware, whilst dynamic transforms published by the estimator maintained the continuously updated robot pose throughout runtime operation. This TF structure was critical for ensuring that LiDAR scans, inertial measurements, odometry outputs, and mapping data all remained spatially aligned throughout the localisation process.

Table 8 summarises the primary runtime ROS topics used throughout the localisation framework, whilst Table 9 summarises the major TF transforms and their associated publisher nodes. The complete topic, TF, and QoS configuration listings are additionally provided within the Appendix.

Table 13 - Primary ROS Topics

Topic	Message Type	Publisher Node	Purpose
/livox/lidar	sensor_msgs/msg/PointCloud2	livox_lidar_publisher	Publishes raw LiDAR point cloud data used for odometry estimation and mapping
/livox/imu	sensor_msgs/msg/Imu	livox_lidar_publisher	Publishes embedded Livox IMU measurements for motion compensation and estimation
/imu/data	sensor_msgs/msg/Imu	microstrain_inertial_driver	Publishes inertial measurements from the Microstrain GX5-25 IMU
/lidar/odom	nav_msgs/msg/Odometry	lidar_odometry_node	Publishes LiDAR-based odometry estimates used for EKF correction
/lidar/odom_raw	nav_msgs/msg/Odometry	lidar_odometry_node	Publishes unsmoothed LiDAR odometry data prior to filtering
/ekf/odom	nav_msgs/msg/Odometry	ekf_node	Publishes fused robot odometry estimates generated by EKF sensor fusion

/ekf/pose	geometry_msgs/msg/PoseStamped	ekf_node	Publishes fused robot pose estimates for visualisation
/ekf/path	nav_msgs/msg/Path	ekf_node	Publishes the estimated robot trajectory over time
/keyframe_map	sensor_msgs/msg/PointCloud2	keyframe_map_node	Publishes the accumulated keyframe-based map representation
/keyframe_map/keyframes	nav_msgs/msg/Path	keyframe_map_node	Publishes stored keyframe trajectory information
/pose_graph/corrected_keyframes	nav_msgs/msg/Path	pose_graph_node	Publishes globally corrected trajectories after loop closure optimisation
/tf	tf2_msgs/msg/TFMessage	Multiple TF broadcasters	Publishes dynamic coordinate frame transforms throughout the ROS graph
/tf_static	tf2_msgs/msg/TFMessage	Static transform publishers	Publishes persistent static coordinate transforms between fixed frames
/rosout	rcl_interfaces/msg/Log	Multiple ROS nodes	Publishes runtime logging and debugging information
/parameter_events	rcl_interfaces/msg/ParameterEvent	ROS parameter system	Publishes runtime parameter update events

Table 14 - Primary TF Transforms

TF Publisher Node	Transform Published	Purpose
map_to_odom	map $\rightarrow$ odom	Defines the global localisation reference frame
ekf_node	odom $\rightarrow$ base_link	Publishes fused robot pose transform
base_link_to_livox	base_link $\rightarrow$ livox_frame	Defines LiDAR mounting position relative to robot base
base_link_to_imu_link	base_link $\rightarrow$ imu_link	Defines IMU mounting position relative to robot base
livox_frame_to_livox_imu_bridge	livox_frame $\rightarrow$ livox_imu	Aligns Livox IMU frame conventions
imu_link_to_microstrain_sensor_alias	imu_link $\rightarrow$ sensor	Provides compatibility alias for Microstrain IMU frame
odom_to_camera_init_for_fastlio_record	odom $\rightarrow$ camera_init	Supports FAST-LIO frame initialisation and recording alignment

5.4.6 | Coordinate Frame and TF Architecture

Figure 36 displays the coordinate frame and TF architecture used throughout the localisation framework. The TF tree defines the spatial relationship between the robot body, sensing

hardware, odometry frames, and global mapping reference frames. Maintaining a correct and consistent TF structure was critical throughout development, as incorrect frame alignment or transform direction immediately resulted in unstable localisation behaviour, distorted mapping, incorrect odometry fusion, or inconsistent visualisation within RViz.

The global reference frame for the localisation stack is defined by the map frame, which represents the persistent world coordinate system used for long-term localisation and mapping. Beneath this sits the odom frame, which acts as the locally continuous odometry reference frame generated throughout runtime operation. The base_link frame represents the physical robot body reference point and acts as the primary parent frame for the sensing hardware mounted to the platform.

Sensor-specific frames are then attached relative to base_link. The external Microstrain IMU is connected through the imu_link frame, whilst the Livox MID-360 LiDAR is attached through livox_frame. The embedded Livox IMU measurements are additionally aligned through the livox_imu frame. FAST-LIO-related operation also introduces the camera_init and body frames, which are used internally by the FAST-LIO localisation pipeline for initialisation and motion estimation.

A major consideration throughout implementation was ensuring that all transforms remained physically meaningful and correctly oriented relative to the actual robot geometry. Incorrect frame rotations, reversed transform directions, inconsistent axis conventions, or duplicated parent-child relationships frequently resulted in localisation divergence, unstable EKF behaviour, incorrect robot motion direction, or severe mapping distortion. As a result, maintaining a clean and interpretable TF hierarchy became one of the most important debugging and validation stages throughout development.

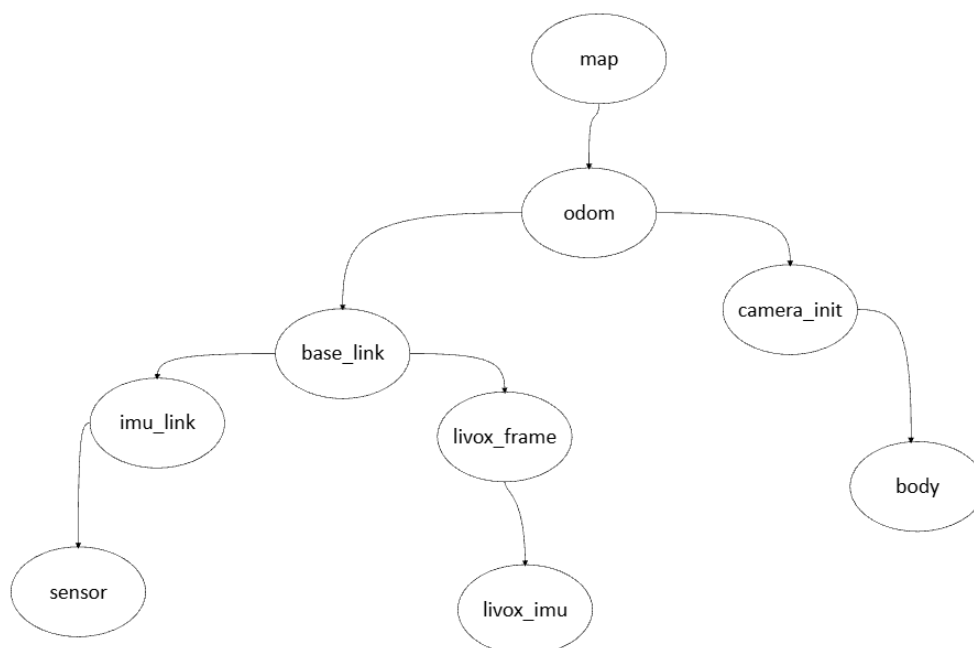


Figure 36 - TF Architecture

5.4.8 | Launch System Design and Configuration

Figure 37 displays the launch and configuration hierarchy used throughout the localisation framework. The launch system was designed around a layered configuration approach, allowing default parameters, deployment-specific overlays, and runtime launch arguments to be combined dynamically during system startup.

The primary entry point for the localisation stack was `launch_slam.launch.py`, located within the 'ros_project_bringup' package. During runtime initialisation, the launch system first loaded the baseline configuration from `slam_bringup.yaml`, before applying deployment-specific overlay configuration files and additional runtime launch arguments. These configurations were then merged into a unified parameter dictionary and distributed across the major subsystem nodes.

This architecture allowed parameters relating to the EKF estimator, LiDAR odometry, mapping subsystem, static transforms, and sensor drivers to remain centrally managed whilst still supporting rapid runtime reconfiguration. Maintaining this layered launch structure simplified testing considerably throughout development, as alternative localisation pipelines, tuning profiles, and deployment modes could be enabled through configuration changes alone without requiring modification of the underlying source code.

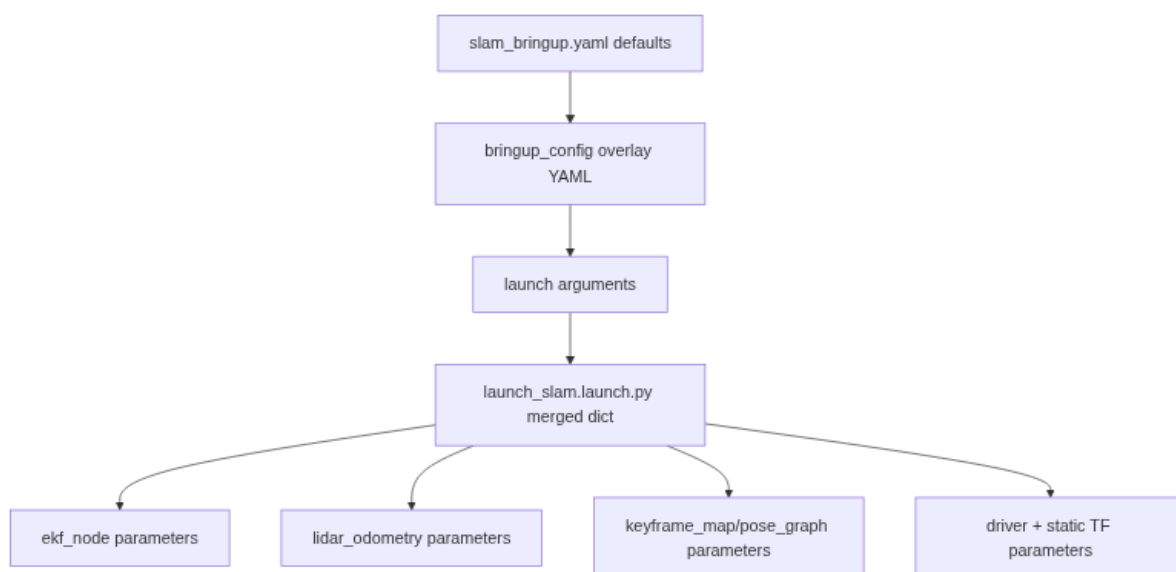


Figure 37 - Launch and Configuration Hierarchy

5.5 | PROCUREMENT AND COST ANALYSIS

The total estimated procurement cost for the complete robotic localisation and SLAM platform was approximately £6,525.10, as shown in Table 10. However, most of this hardware originated from a previous autonomous RC vehicle project, including the Traxxas X-Maxx platform, TX2i infrastructure, Microstrain IMU, power systems, and supporting integration hardware. This significantly reduced the effective development cost of the project.

The primary purchases made specifically for this project were the Livox MID-360 (£599.00), two Traxxas 6700mAh 14.8V LiPo batteries (£280.00 total), and two NVIDIA Jetson Orin Nano Developer Kits (£449.00 each), resulting in an approximate direct expenditure of £1,777.00, excluding smaller fabrication and integration consumables. The Jetson Orin Nano systems were introduced to provide improved computational performance for ROS 2 Humble, LiDAR processing, and SLAM deployment.

If the platform were developed entirely from scratch without pre-existing infrastructure, the total system cost would likely exceed £7,000 once sensing hardware, embedded compute systems, batteries, fabrication materials, and the robotic chassis are included.

Table 15 - Procurement Table

ID	Item	Purpose	Qty	Estimated Cost (£)
1	Livox MID-360	Primary LiDAR sensing and SLAM	1	£599.00 [30]
2	Microstrain 3DM-GX5-25	High-rate inertial sensing	1	£2,000.50 [29]
3	NVIDIA Jetson Orin Nano Developer Kit	Main embedded compute platform	2	£898.00 [31]
4	NVIDIA Jetson TX2i	Legacy compute platform	1	£614.04 [28]
5	Quasar Carrier for Jetson TX2i	TX2i expansion and connectivity	1	£203.92 [32]
6	Jetson TX2i Active Heat Sink	Thermal management	1	£106.62 [33]
7	Tri-M HPS3512	Main subsystem power distribution	1	£299.99 [34]
8	Tri-M 203W Smart PC104 PSU	Voltage regulation	1	£336.00 [34]
9	RSHTECH 7-Port USB 3.0 Hub	Peripheral expansion	1	£28.99 [35]
10	Traxxas X-Maxx 8S	Robotic chassis platform	1	£1,401.97 [36]
11	Traxxas 6700mAh 14.8V LiPo Battery	Robot drive power	2	£280.00 [37]
12	TP-Link 300Mbps Mini Wireless N USB Adapter	Wireless SSH/debugging access	1	£7.00 [38]
13	D-Link AC600 USB Wi-Fi Adapter	Improved wireless connectivity	1	£19.99 [39]
14	SunFounder PCA9685 PWM Driver	PWM control interface	1	£9.99 [40]
15	Ethernet Cables (CAT6)	LiDAR communication	2	£9.98
16	USB Cables / Adapters	Sensor communication	Multiple	£19.99
17	XT60 Connectors	Power distribution	Pack	£3.41 [43]
18	14AWG Silicone Wire	Power cable fabrication	1	£1.43 [44]
19	Turnigy 5000mAh 3S LiPo Battery	Portable subsystem power	2	£37.48 [41]
20	EV-Peak LiPo Charger	Battery charging	1	£31.34 [42]
21	3D Printing Filament (ABS/PLA)	Custom enclosure fabrication	Multiple prints	£24.99

22	M2.5 / M3 / M4 Hardware	Mechanical mounting	Assorted	£14.50
23	Cable Ties / Adhesive Mounts	Cable management	Pack	£6.99
24	5V Cooling Fan	Additional enclosure cooling	1	£14.32 [45]
25	Autodesk Fusion	CAD and enclosure analysis	Student License	£0.00 [46]
26	Ubuntu 22.04	Operating system	1	£0.00
27	ROS 2 Humble / Foxy	Robotics middleware	1	£0.00 [47]
28	MATLAB	Analysis and graph generation	Student License	£0.00 [48]
29	RViz2	Runtime visualisation	1	£0.00 [49]

5.6 | SUMMARY

Overall, this section established the final system architecture used throughout the localisation framework, including both the hardware and software integration required for deployment. The system was designed around a modular ROS 2 SLAM architecture consisting of LiDAR sensing, IMU-based state estimation, LiDAR odometry, mapping, and visualisation.

The hardware design focused on creating a modular and easily deployable subsystem which could be integrated onto the robot platform whilst still remaining easy to modify, debug, and maintain throughout development. Alongside this, the software architecture defined the ROS 2 node structure, processing pipeline, TF hierarchy, communication framework, and launch configuration system used throughout runtime operation. Together, these designs formed the foundation for the implementation, testing, optimisation, and deployment stages discussed throughout the remainder of the project.

6 | Methodology

6.1 | IMPLEMENTATION STRATEGY

Development of the localisation and SLAM framework followed a staged and iterative engineering methodology, allowing each subsystem to be progressively validated before full system integration. Figure 38 illustrates the overall development workflow implemented throughout the project, beginning from ROS 2 environment setup and sensor integration, through to localisation, mapping, playback testing, hardware deployment, and final system evaluation.

The initial stages of development focused on establishing a stable ROS 2 environment and validating the primary sensing hardware. This involved integration of the IMU and LiDAR drivers, verification of ROS topic communication, TF validation, and confirming that reliable sensor data could be obtained before localisation development began. Once stable sensing and communication had been achieved, development progressed towards implementation of the EKF-based state estimator and LiDAR odometry pipeline, allowing the robot pose to be estimated from fused inertial and LiDAR measurements.

Following successful localisation integration, additional stages focused on mapping integration, refinement of sensor fusion behaviour, TF consistency validation, rosbag playback testing, and iterative system optimisation. Throughout development, the workflow remained highly iterative, where testing, debugging, parameter tuning, and subsystem refinement continuously fed back into earlier stages of the framework when instability or inconsistency was observed. This proved particularly important during SLAM development, where incorrect TF alignment, timing offsets, or odometry drift could rapidly propagate instability throughout the remainder of the localisation pipeline. The final stages of the methodology focused on deployment onto the embedded robotic platform and experimental evaluation under real operating conditions.

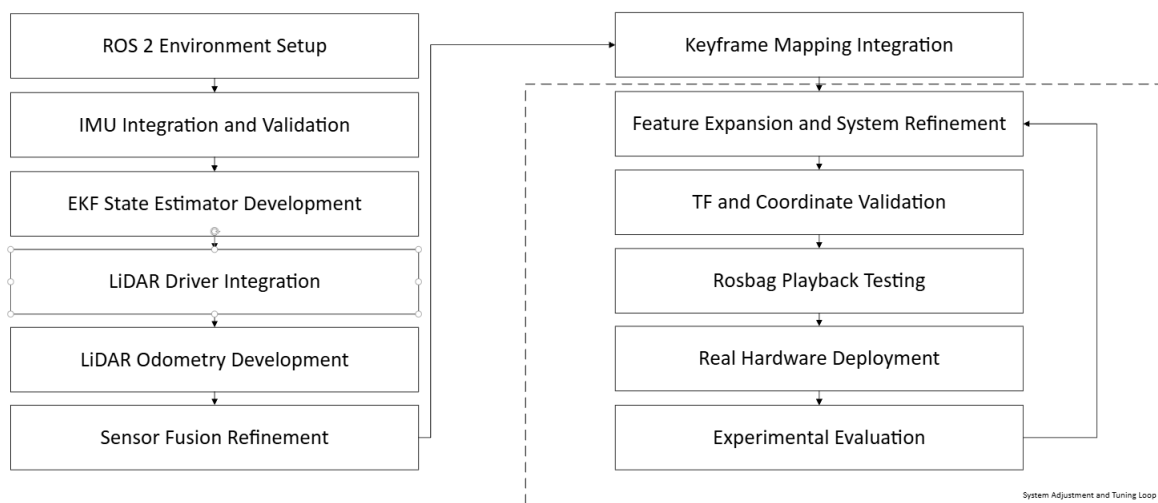


Figure 38 - Development Strategy

6.2 | STAGED DEVELOPMENT PROCESS

Briefly explain the primary subsystems developed during each stage of the project, why they were developed in that order, and how functionality was validated at a high level. Keep this section conceptual rather than deeply technical.

6.2.1 | ROS 2 Environment and Workspace Setup

The first stage of development focused on establishing a stable ROS 2 environment and modular workspace structure to support later localisation and SLAM development. This stage was prioritised first, as all later sensing, estimation, mapping, and deployment functionality depended on a reliable middleware and package framework. Development was carried out primarily using ROS 2 Humble on Ubuntu 22.04 on both the development laptop and Jetson Orin Nano platform. A separate lightweight deployment workspace was additionally configured using a custom ROS 2 Foxy installation on Ubuntu 18.04 for compatibility with the Jetson TX2i platform.

A modular package structure was adopted to separate localisation, LiDAR odometry, mapping, launch orchestration, and sensor integration into independent subsystems. This simplified staged development considerably, as individual components could be tested, modified, or replaced without affecting the remainder of the framework.

Validation during this stage focused primarily on successful package compilation, dependency resolution, ROS topic communication, launch sequencing, and TF publication before progressing.

6.2.2 | Sensor Integration and Validation

6.2.2.1 | Microstrain GX5-3DM-25 IMU

Following establishment of the ROS 2 environment, development progressed towards integration and validation of the Microstrain GX5-3DM-25 IMU within the ROS middleware framework. Initial validation focused on confirming successful communication, stable topic publication, and consistency of the inertial measurements prior to integration with the localisation stack. Figure 39 displays example runtime IMU topic output observed during validation testing.

Integration on the development laptop and Jetson Orin Nano using ROS 2 Humble was relatively straightforward using the standard Microstrain ROS packages. However, deployment onto the Jetson TX2i platform required modification of the driver due to unsupported dependencies and library compatibility limitations associated with Ubuntu 18.04 and ROS 2 Foxy.

Once communication had been established, the IMU outputs published on `/imu/data` and `/imu/data_raw` were validated through ROS topic inspection and runtime monitoring of orientation, angular velocity, and linear acceleration measurements before progressing towards EKF integration.



```

macia@MACLA-UBUNTU-DESKTOP: ~/ROS_Deploy
macia@MACLA-UBUNTU-DESKTOP: ~/ROS_Project/src$ ros2 topic list | grep "/imu/data"
/imu/data
macia@MACLA-UBUNTU-DESKTOP: ~/ROS_Project/src$ ros2 topic echo /imu/data
header:
  stamp:
    sec: 1777852659
    nanosec: 745967616
  frame_id: sensor
orientation:
  x: -0.02089962645725628
  y: -0.003686653327876729
  z: 0.1851864245674097
  w: -0.9824742274508379
orientation_covariance:
  9.0
  0.0
  0.0
  0.0
  0.0
  0.0
  0.0
  0.0
  0.0
angular_velocity:
  x: -0.004053059034049511
  y: -0.0023055942729115408
  z: 0.0026233067665723562
angular_velocity_covariance:
  9.0
  0.0
  0.0
  0.0
  0.0
  0.0
  0.0
  0.0
  0.0
linear_acceleration:
  x: -0.18630513682384642
  y: 0.47905170975960787
  z: 9.84486136497259
linear_acceleration_covariance:
  9.0
  0.0
  0.0
  0.0
  0.0
  0.0
  0.0
  0.0
  0.0

```

Figure 39 - Microstrain IMU Message

6.2.2.2 | Livox MID-360 LiDAR

Following IMU integration, the Livox MID-360 LiDAR was integrated and validated within the ROS 2 framework. Initial validation focused on confirming successful point cloud publication on `/livox/lidar` alongside communication from the embedded Livox IMU on

/livox/imu. Figure 40 displays example LiDAR point cloud output observed during runtime testing.

Integration primarily required configuration of the LiDAR network interface, including disabling Wi-Fi whilst using the Ethernet connection and assigning the LiDAR IP address within the Livox driver configuration files. Integration on the development laptop and Jetson Orin Nano platforms was relatively straightforward, whilst deployment onto the Jetson TX2i required removal of several unsupported dependencies to achieve stable operation under Ubuntu 18.04 and ROS 2 Foxy.

Validation at this stage focused primarily on confirming stable point cloud publication, correct frame assignment, embedded IMU communication, and successful ROS topic transmission prior to localisation and mapping development; Figure 40.

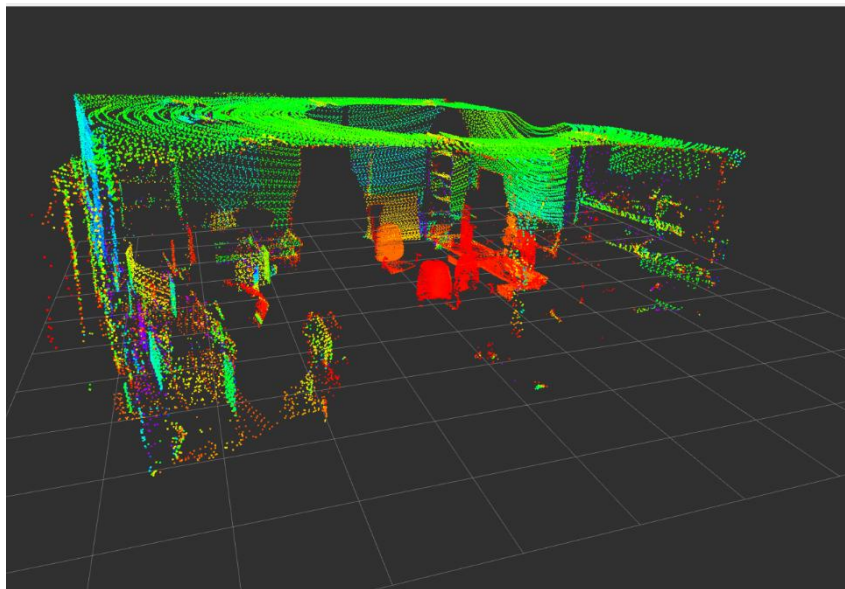


Figure 40 - Raw LiDAR Point-Cloud

6.2.3 | EKF State Estimator Development

Following successful sensor integration, development progressed towards implementation of the EKF-based localisation system. Initial development focused on validating IMU-based state propagation behaviour using simulated sensor data prior to the introduction of external correction sources. Figure 41 displays an example EKF trajectory (green line) behaviour during early untuned testing using simulated inertial inputs without correction, where gradual trajectory drift can be observed over time.

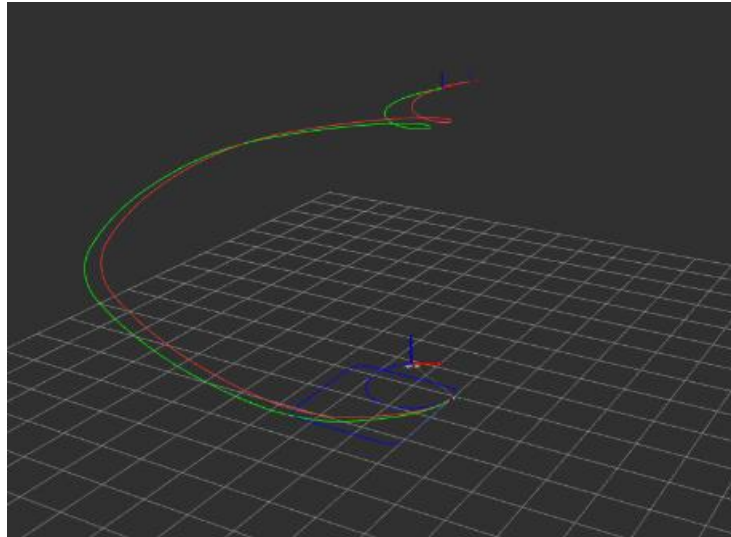


Figure 41 - Pure IMU Estimation

As development progressed, corrective odometry inputs were introduced into the EKF framework to stabilise the trajectory estimates and reduce accumulated drift. FAST-LIO localisation outputs were additionally used as a baseline reference throughout refinement and validation of the EKF behaviour. Figure 42 displays a comparison between the developed EKF framework and FAST-LIO localisation outputs when both systems utilised the same corrective data source.

Validation during this stage focused primarily on observing trajectory consistency, reduction in drift accumulation, and overall agreement between localisation outputs during playback testing. The EKF framework was found to already provide relatively stable behaviour with minimal refinement required beyond general parameter tuning and frame validation.

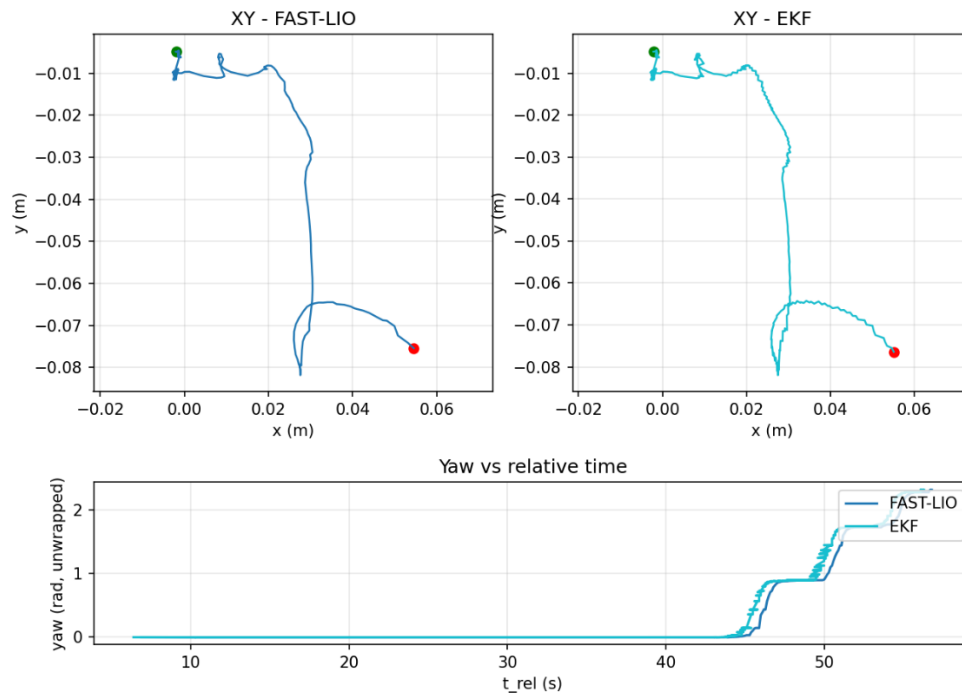


Figure 42 - EKF Performance Comparison

6.2.4 | LiDAR Odometry Development

Following development of the EKF localisation framework, implementation progressed towards NDT-based LiDAR odometry estimation. The LiDAR odometry subsystem was developed separately from the remainder of the localisation stack initially, allowing scan matching behaviour and trajectory estimation to be validated in isolation prior to full EKF fusion and mapping integration.

Development focused on generating relative robot motion estimates from sequential LiDAR scans using NDT registration techniques. FAST-LIO localisation outputs were again used as a baseline reference during validation, allowing trajectory behaviour and yaw estimation from the NDT pipeline to be compared against a stable reference localisation system. Figure 43 displays an example comparison between FAST-LIO and the initial unsmoothed NDT odometry output, where noticeable trajectory instability and noise can be observed throughout the estimated path.

Additional refinement stages then focused on stabilising the LiDAR odometry behaviour prior to downstream EKF fusion. Figure 44 displays the resulting smoothed NDT odometry output following refinement, where significantly improved agreement with the FAST-LIO reference trajectory can be observed. Validation throughout this stage primarily focused on trajectory consistency, reduction in odometry noise, and overall alignment between the NDT and FAST-LIO localisation behaviour during repeated playback testing.

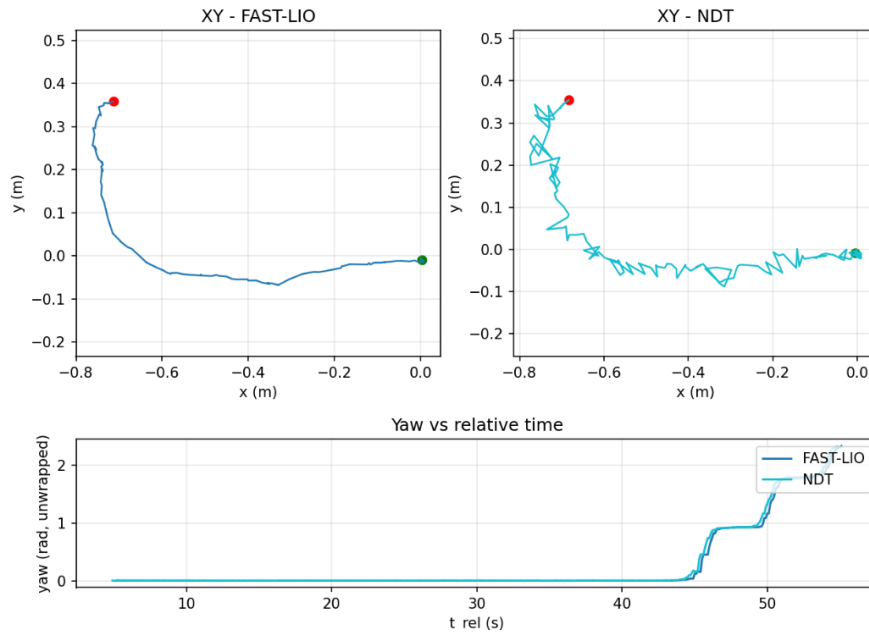


Figure 43 - NDT Before Tuning

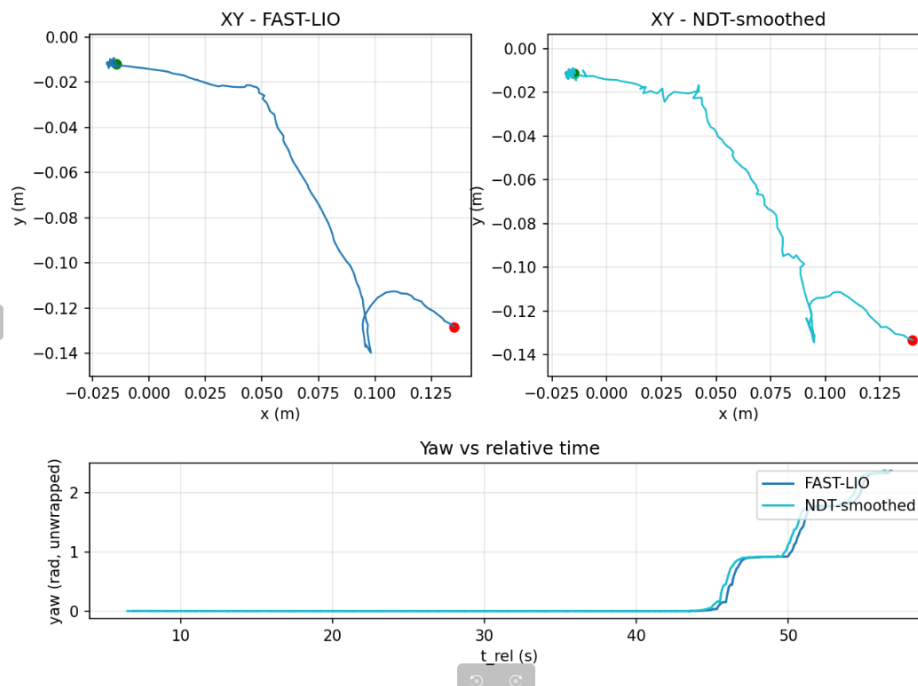


Figure 44 - NDT After Tuning

6.2.5 | Sensor Fusion Refinement

Following isolated validation of both the EKF and LiDAR odometry subsystems, development progressed towards integration of the complete sensor fusion pipeline. This stage focused on combining inertial prediction from the EKF with corrective LiDAR odometry updates to generate a unified localisation estimate suitable for downstream mapping and SLAM operation.

Initial refinement primarily focused on validating TF consistency, odometry alignment, timing behaviour, and coordinate frame agreement between the localisation components. Playback-based testing using recorded rosbag data was heavily utilised throughout this stage,

allowing localisation behaviour to be repeatedly evaluated under identical operating conditions whilst avoiding the inconsistencies associated with live hardware testing.

Figure 45 displays example localisation behaviour observed during refinement, where the fused EKF trajectory (green) can be compared against the LiDAR odometry (orange) estimate throughout runtime operation. Validation during this stage focused primarily on trajectory stability, reduction in localisation noise, consistency between prediction and correction behaviour, and overall agreement between the combined localisation outputs prior to mapping integration.

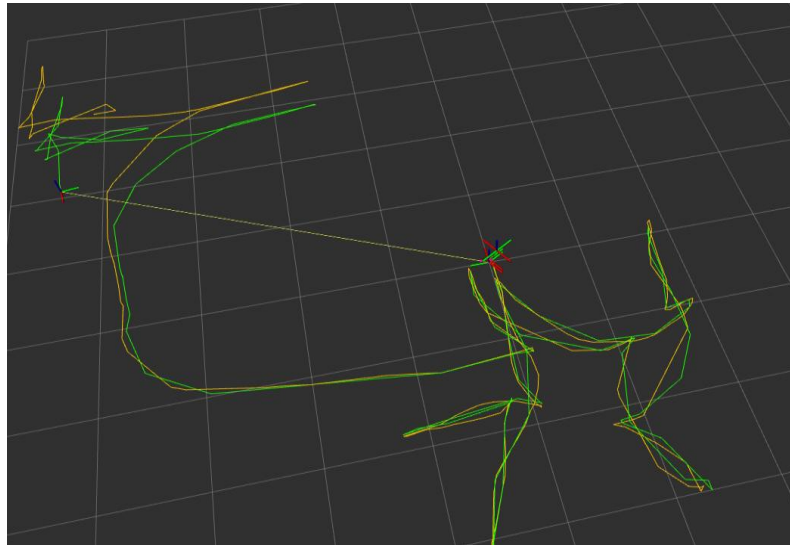


Figure 45 - EKF vs LiDAR Odom Path

6.2.6 | Keyframe Mapping Integration

Following successful refinement of the localisation pipeline, development progressed towards integration of the keyframe-based mapping subsystem. This stage focused on using the fused localisation outputs generated by the EKF framework to position and accumulate LiDAR scans into a persistent environmental map representation.

The mapping subsystem utilised corrected robot pose estimates alongside incoming LiDAR point cloud data to progressively insert scans into the keyframe map structure during robot motion. Figure 46 displays example keyframe mapping output generated during runtime operation, where sequential scans can be observed accumulating throughout the environment.

Validation during this stage focused primarily on confirming consistency between localisation and mapping behaviour, stability of scan placement, and overall continuity of the generated map structure during playback testing and live runtime operation.

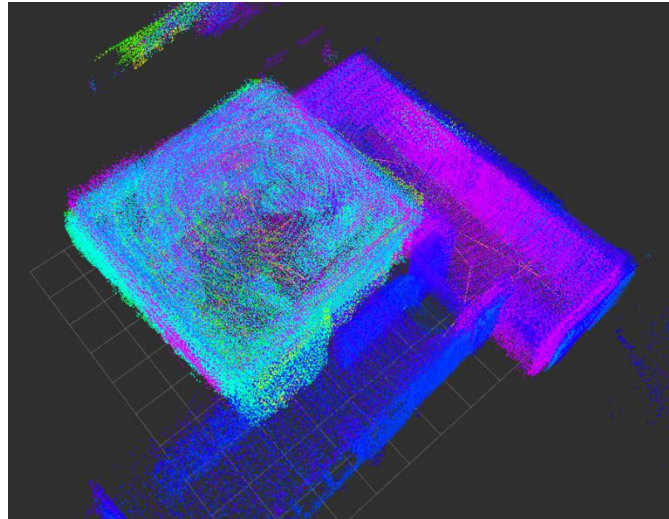


Figure 46 - Keyframe Map

6.2.7 | Feature Expansion and System Refinement

Throughout development, the system gradually evolved beyond the initial localisation objectives through iterative debugging, optimisation, and feature additions. Many of these refinements were introduced to improve runtime stability, mapping quality, deployment reliability, and overall system robustness during real-world operation.

- EKF tuning and optimisation
- NDT LiDAR odometry refinement
- FAST-LIO integration and evaluation
- LiDAR deskewing using integrated IMU data
- Auto-levelling for suspension and chassis tilt compensation
- Filtering robot-mounted structures from LiDAR scans
- Sensor extrinsic calibration and axis correction
- ROS 2 launch and deployment automation
- Network and DDS communication refinement
- Improved cable management and hardware protection
- TX2i and Jetson Orin Nano deployment support
- Simulation-first workflow and real-world deployment testing
- Future loop-closure integration consideration
- Support for integration with external/student project modules

6.2.8 | Playback-Based Validation and Iterative Testing

Rosbag playback testing was utilised extensively throughout development to provide repeatable and controlled evaluation of the localisation and SLAM framework. Figure 47 illustrates the general playback-based refinement workflow implemented throughout the project, beginning from dataset recording through to playback, behaviour analysis, parameter refinement, and repeated subsystem evaluation.

Rather than relying solely on live hardware testing, sensor datasets were first recorded during runtime operation and then replayed repeatedly through the localisation stack using `roslaunch rosbag2_player`. This allowed localisation, mapping, TF behaviour, and sensor fusion

performance to be analysed under identical operating conditions across multiple refinement iterations.

Playback testing proved particularly useful throughout development of the EKF, LiDAR odometry, mapping, and TF subsystems, where small configuration changes could be rapidly validated without requiring repeated live data collection. Validation during this stage primarily focused on observing trajectory behaviour, map consistency, TF stability, topic timing behaviour, and overall localisation performance throughout repeated playback operation.

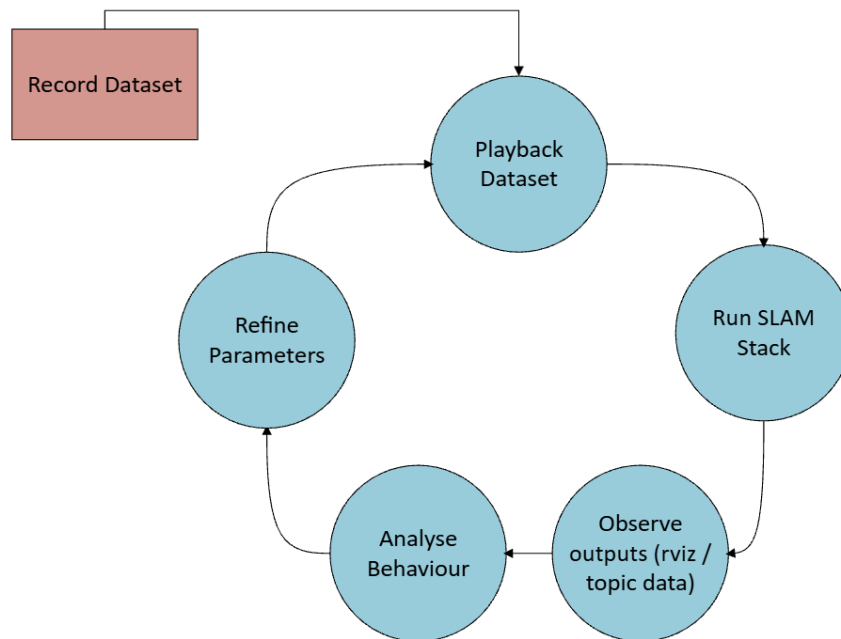


Figure 47 - Refinement Workflow

6.3 | HARDWARE SETUP

6.3.1 | Assembly and Integration

The first stage of assembly involved preparing the power and communication wiring. Several existing wires on the platform were either loose or damaged and therefore required repair. Due to limited connector availability and time constraints, the original connectors were removed and replaced using soldered or crimped terminations. Figure 47 displays the preparation process for the first cable set. These cables provide the switching functionality shown in Figure 49, allowing the system to easily swap between the TX2i and Jetson Nano platforms. In this configuration, the switching connector is implemented using a bullet connector. The loose power lead connects to the positive terminal of the TX2i, while an additional black wire provides the ground connection to the power station. XT60 connectors were also prepared to supply power to the Livox LiDAR, as shown in Figure 50.

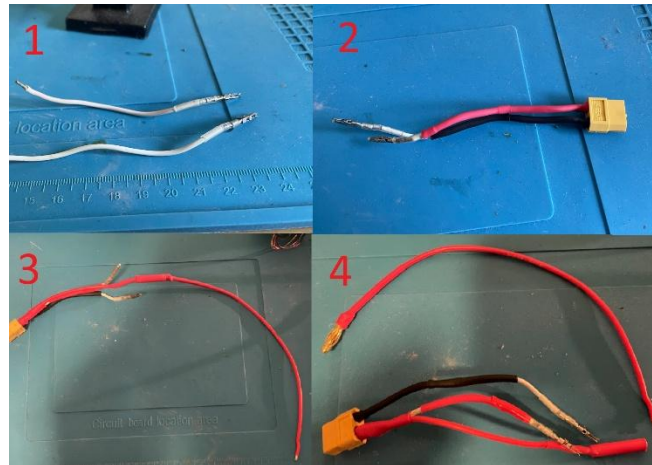


Figure 48 - TX2i + LiDAR Power Cable Procurement

The next step is to connect the prepared TX2i power and ground wires to their corresponding terminals.



Figure 49 - TX2i Power Cable Installation

The final part of the pre-requisite is to prepare the Jetson Nano power cable using a similar process. A barrel connector was removed from an external power supply previously used to power the power station from mains power. Wires were soldered to this connector, terminated appropriately, and insulated. One side should terminate with the male bullet connector, while the opposite side should terminate with the crimped connector used for the power station.

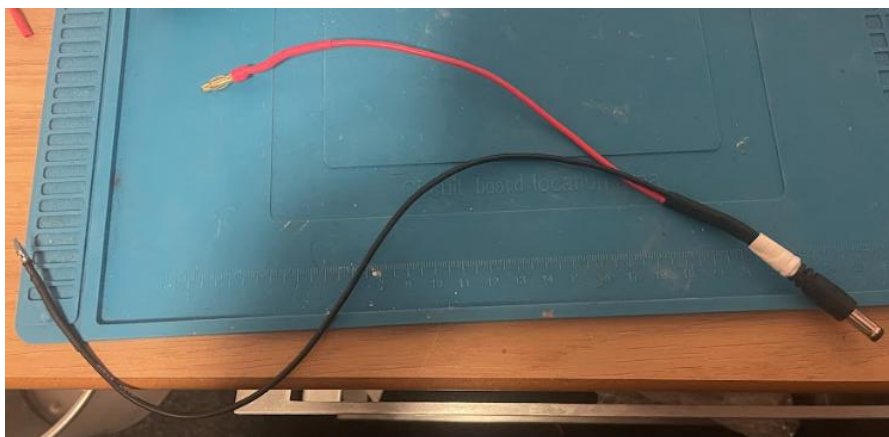


Figure 50 - Jetson Orin Nano Power Cable Procurement

Assembly is as instructed below:

1. Drill the two labelled 2.5 mm holes through the power-station mount.

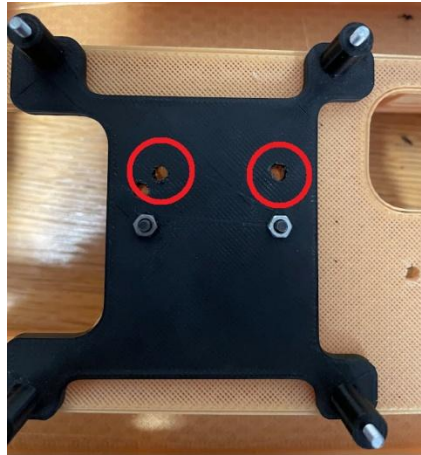


Figure 51 - IMU Mount

2. Secure the IMU to the mount using nuts and bolts through the drilled holes.



Figure 52 - IMU Mounted

3. The resulting assembly state after IMU installation:



Figure 53 - Expected Result 1

4. Secure the power station onto the same mount as the IMU by fastening nuts through the red-circled mounting points. Connect the cables shown in the figure below to the power-station terminals indicated by the blue circle.

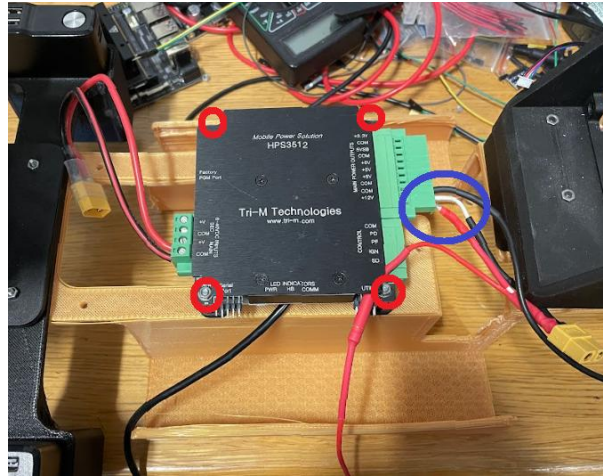


Figure 54 - Power Station Integration

5. Remove the cooling fan indicated by the red circle in Figure 55.

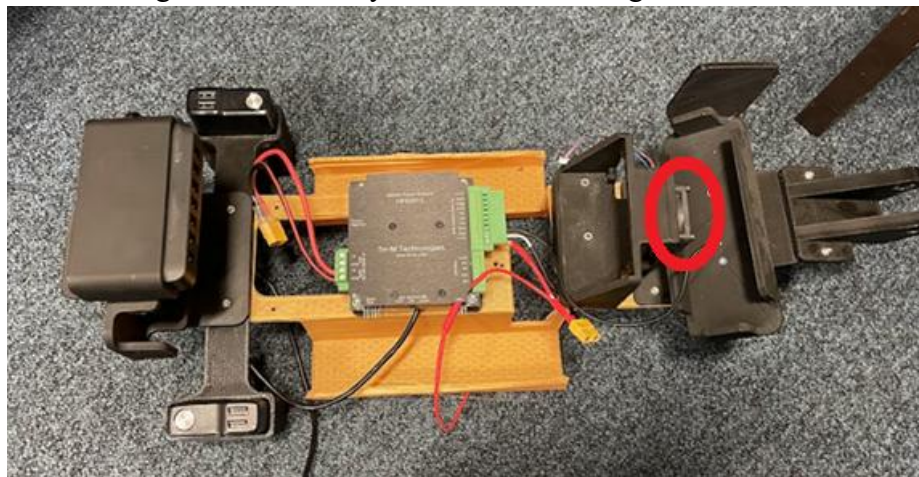


Figure 55 - Extra Fan Removal

6. Place the chassis assembly into the robot frame. The IMU cable should then be wrapped into a circular bundle and secured using two zip ties, red cable. A third zip tie should secure the cable bundle to the power station casing. Care should be taken not to overtighten this final zip tie, as excessive pressure may damage the power station casing. Secure the TX2i to its mounting bracket by passing the heatsink screws through the mount and into the TX2i mounting holes, orange circle. Secure the front section of the chassis using two zip ties in the zigzag arrangement shown in figure below, blue circle. The IMU Cable can then be connected either to the USB hub or TX2i directly

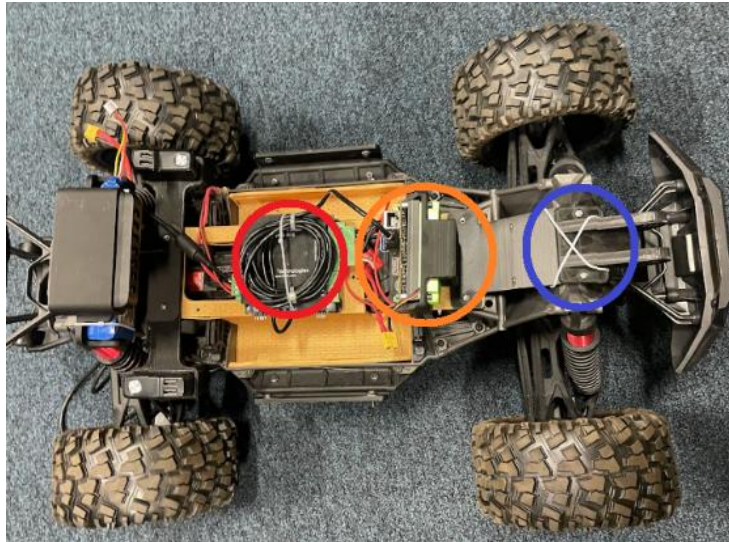


Figure 56 - Inner Enclosure Integration

7. Secure one side of the 70 cm Velcro strip to the battery mount using superglue or another strong adhesive. The Velcro strip is then used to secure the battery in the location indicated by the blue circle. The rear section of the chassis should then be secured by passing a bolt through both the chassis and frame before fastening with a nut, as shown by the red circle.



Figure 57 - Inner Enclosure Fixture + Battery Strap

8. Connect the wireless antenna to the USB hub at the location indicated by the red circle.

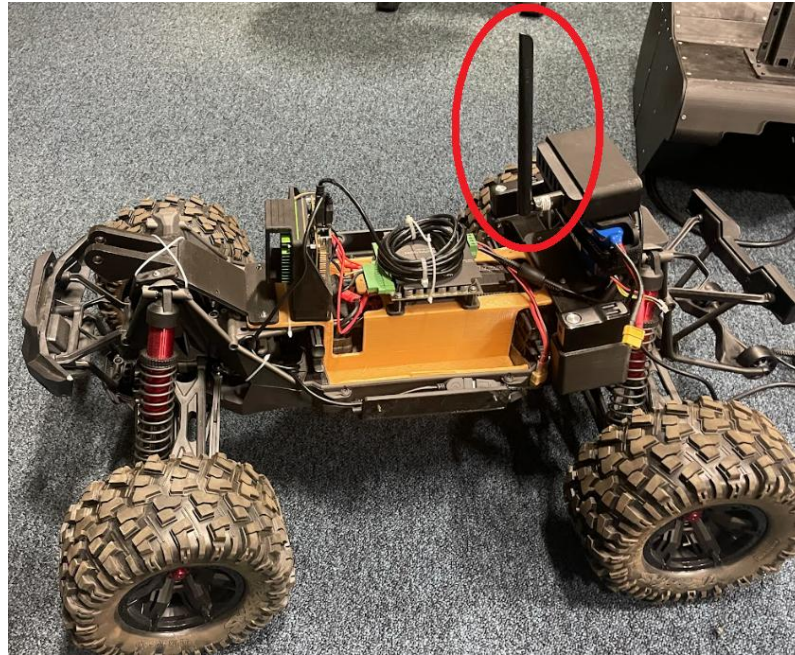


Figure 58 - Wi-Fi Adapter

9. Route a USB extension cable through the side rails of the robot and connect it to the USB hub cable. If excessive cable length remains, coil the cable and secure it beneath the TX2i mount using zip ties.

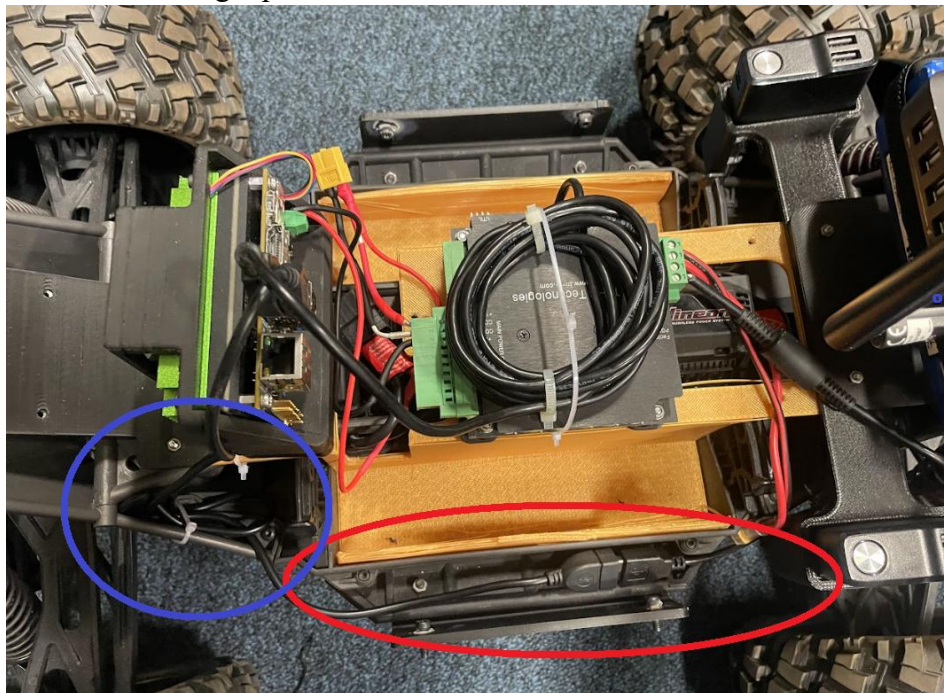


Figure 59 - USB Hub Connection

10. Secure the camera to its mount using a nut, bolt, and suitable washer. Route the camera cable through the centre of the mount to allow connection to the Jetson Nano.



Figure 60 - Camera Integration

11. Secure the printed Jetson Nano mount to the chassis using the mounting locations indicated by the red circles. Connect the Jetson power-cable ground to the power station.



Figure 61 - Jetson Power Cable Connection

12. Secure the Jetson Nano to the mount using the mounting holes shown in the illustration below.

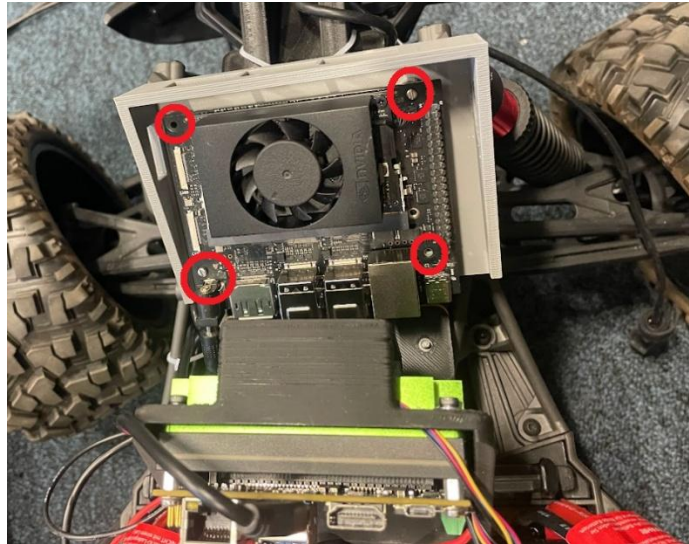


Figure 62 - Jetson Nano Mounted

13. Attach the prepared power connector from Figure 50 to the Jetson Nano.



Figure 63 - Jetson Nano Power Cable Insertion

14. Below is the expected internal assembly configuration. The final appearance may differ depending on whether the TX2i or Jetson Nano configuration is used.

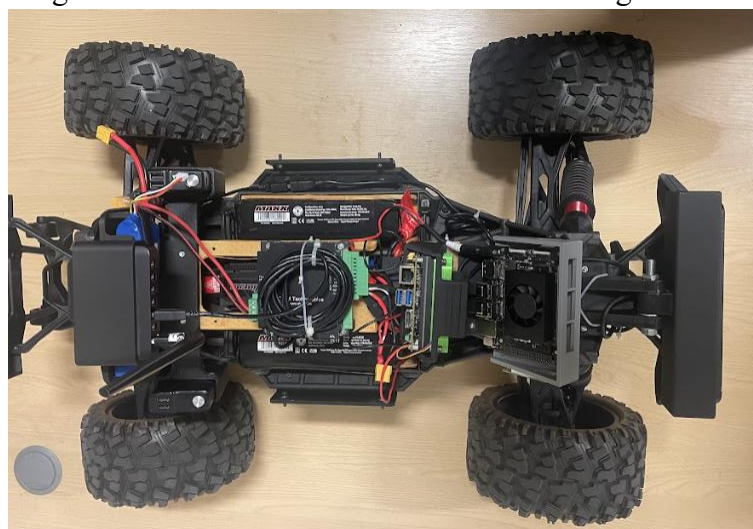


Figure 64 - Expected Result 2

15. Prepare the outer enclosure by drilling the holes specified in the design files. Secure the LiDAR to the tower mount using the appropriate mounting hardware.



Figure 65 - LiDAR Mount Integration

16. Connect the Livox communication and power cable to the LiDAR.



Figure 66 - LiDAR Cable Connection

17. Route the cable through the outer enclosure using the opening indicated by the red circle.

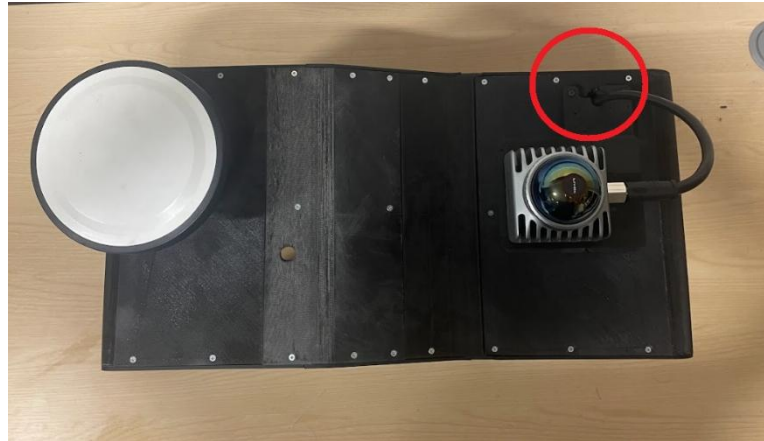


Figure 67 - LiDAR Cable Wiring

18. Secure the LiDAR cable bundle using zip ties to reduce cable movement and tangling during operation.



Figure 68 - LiDAR Cable Organisation

19. Connect the battery at the location indicated by the red circle.

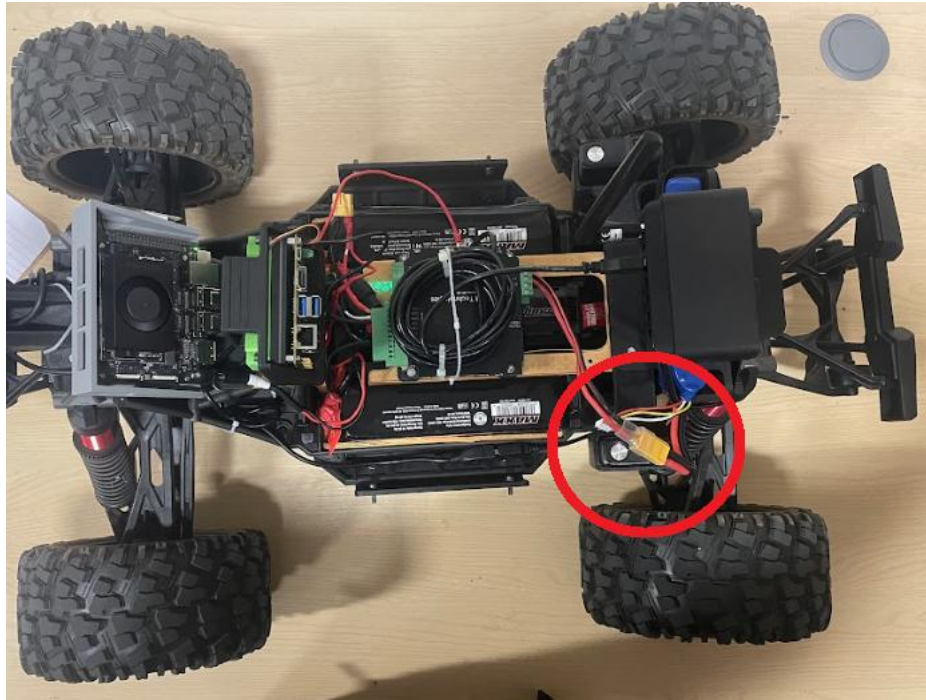


Figure 69 - Battery Connection

20. Connect the LiDAR Ethernet cable to the Jetson platform and connect the LiDAR power cable to the prepared XT60 connector.

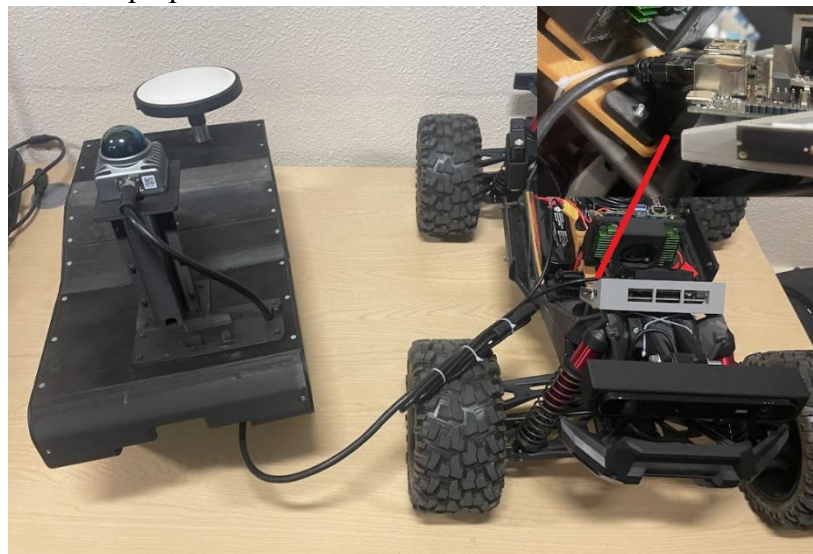


Figure 70 - LiDAR Ethernet Connection

21. Secure the outer chassis assembly to the robot platform using the two bolts located on either side of the robot frame.

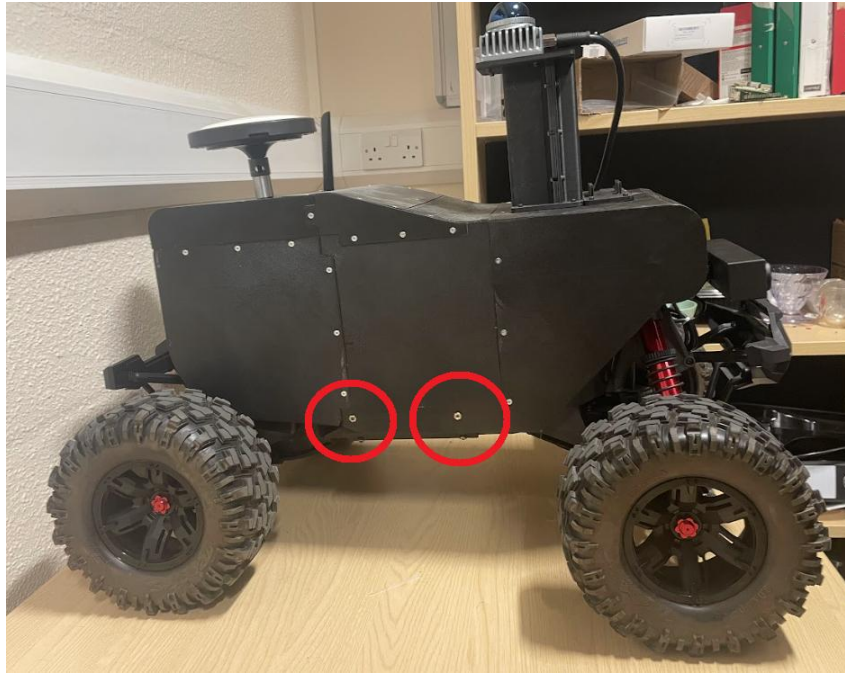


Figure 71 - Final Integration

22. Below is the final assembled robotic localisation and SLAM platform.



Figure 72 - Expected Result 3

6.3.2 | Hardware Layout Grid

The following list conveys the identification for the various components shown in *Figure 73* – 74:

- A - RTK GPS Antenna (Unused)
- B - LiDAR
- C - Wifi Antenna
- D - Outer Enclosure

- E - Camera
- F - Wifi Antenna
- G - Power Station
- H - Robot Batteries
- I - IMU
- J - Localisation System Battery
- K - USB Hub
- L - TX2i
- M - Jetson Orin Nano

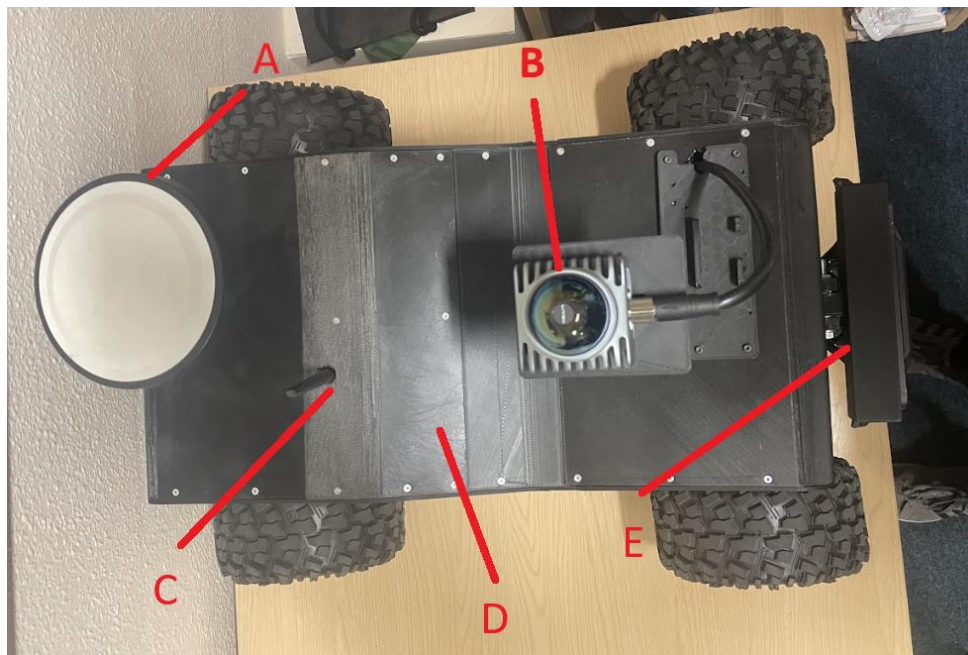


Figure 73 - Final Result, Top-Down View

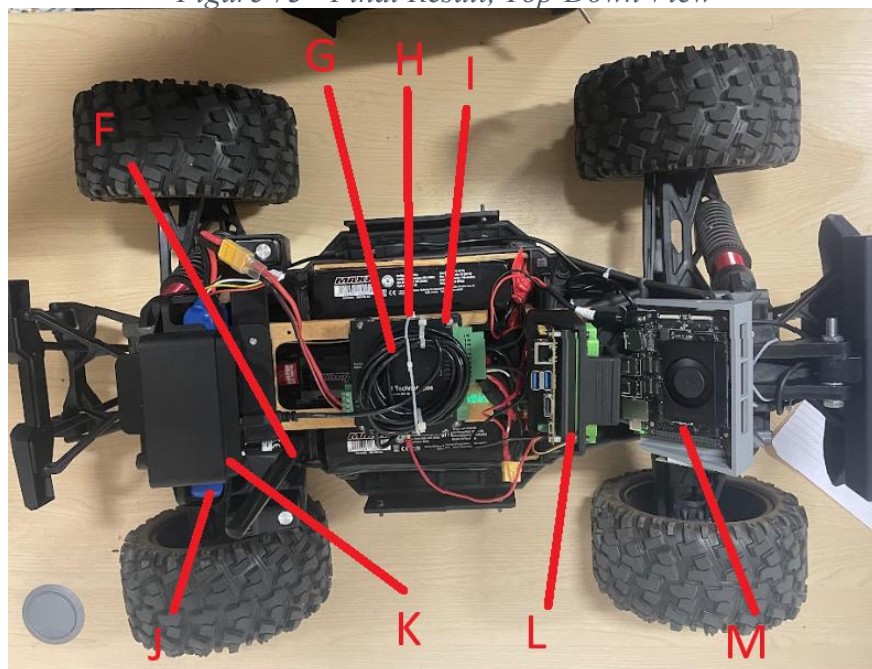


Figure 74 - Exposed Final Result, Top-Down View

6.3.3 | Coordinate Offsets From Origin

For consistency throughout the localisation and SLAM framework, a fixed robot origin was defined relative to the physical chassis. In this project, the robot origin was defined as the centre point between the four wheels, positioned at approximately half the wheel height relative to the ground plane and centred laterally within the robot chassis. This provided a stable and intuitive reference frame for sensor integration, TF generation, and motion estimation. Figures 75 and 76 display the physical location of the defined robot origin relative to the platform.

The coordinate-axis orientation used throughout the project is illustrated in Figure 75. All sensor offsets and transforms were defined relative to this coordinate convention.

Within the prototype implementation, both the IMU and LiDAR were mounted near the centre region of the platform to minimise motion distortion caused by aggressive chassis movement and suspension effects. The IMU position was recorded relative to the robot origin as approximately:

- $x = 12$
- $y = -0.7$
- $z = 38$

with an applied yaw rotation of -90° .

Similarly, the LiDAR position was recorded as:

- $x = 12$
- $y = 0.7$
- $z = 38$

with an applied yaw rotation of 180° .

These offsets and rotational transforms were integrated directly into the sensor driver and SLAM configuration files to ensure that sensor data aligned correctly within the global coordinate frame. Incorrect configuration of these offsets initially resulted in severely distorted motion estimation, including forward motion appearing as backwards movement within the generated map. After applying the corrected transforms, sensor alignment and localisation behaviour improved substantially.

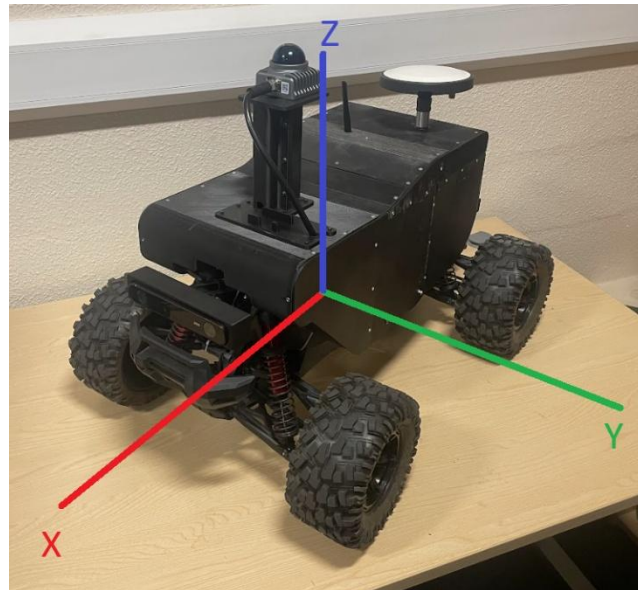


Figure 75 - Axis Relative to Robot

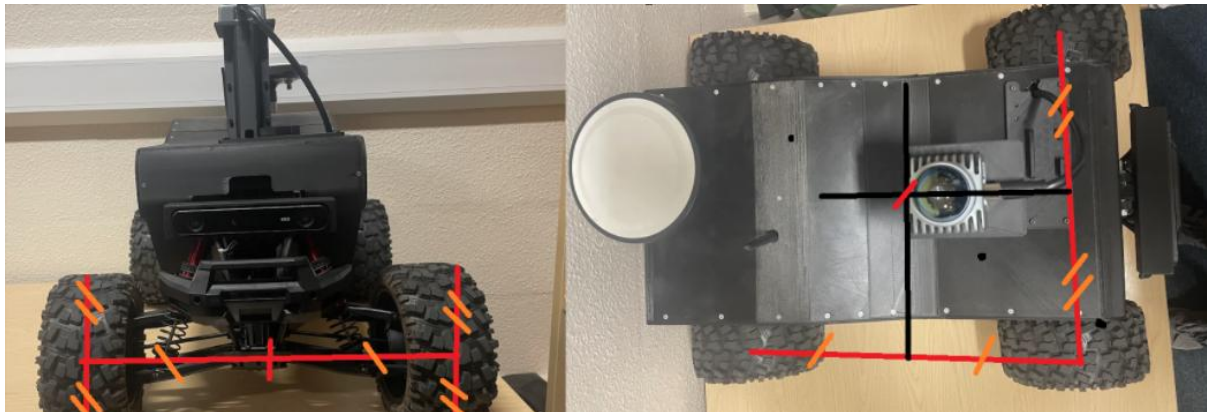


Figure 76 - Robot Origin Visual Identification

6.4 | SOFTWARE SETUP AND DEPLOYMENT

The localisation framework was deployed across multiple hardware platforms running different Ubuntu and ROS 2 distributions depending on computational capability and hardware compatibility requirements. The primary development laptop and Jetson Orin Nano platforms utilised Ubuntu 22.04 with ROS 2 Humble, whilst the legacy Jetson TX2i platform required Ubuntu 18.04 with a custom ROS 2 Foxy installation due to compatibility limitations associated with the older NVIDIA JetPack environment.

The software architecture was additionally separated into three deployment workspaces depending on device responsibility. 'ROS_Deploy' was used on platforms responsible for full SLAM execution, localisation, mapping, and sensor fusion. 'ROS_Robot' was used on legacy sensing-only platforms running ROS 2 Foxy, whilst 'ROS_Robot_Humble' provided equivalent sensing-only functionality for Humble-based systems. This separation simplified deployment considerably, reduced unnecessary runtime dependencies on embedded hardware, and allowed sensing and SLAM functionality to remain modular throughout development.

6.4.1 | Environment Setup

6.4.1.1 | Ubuntu 22.04, ROS 2 Humble

The primary development laptop and Jetson Orin Nano platform utilised Ubuntu 22.04 with ROS 2 Humble. ROS 2 Humble was installed using the standard binary installation procedure provided by the ROS 2 documentation, followed by installation of the required development tools, ROS dependencies, and workspace build utilities.

A dedicated deployment workspace was then created for the SLAM framework, where the required localisation, mapping, and sensor packages were compiled using the colcon build system. Automatic environment sourcing was additionally configured through the .bashrc file to simplify deployment and terminal initialisation during development and testing.

Example environment setup procedure:

```
source /opt/ros/humble/setup.bash
cd ~/ROS_Deploy
colcon build
source install/setup.bash
```

This environment was primarily used for full SLAM operation, rosbag playback testing, localisation development, and runtime deployment on modern embedded hardware. See *Appendix B* for a full installation guide.

6.4.1.2 | Ubuntu 18.04, ROS 2 Foxy

The Jetson TX2i platform utilised Ubuntu 18.04 due to NVIDIA JetPack compatibility limitations. Since ROS 2 Humble was unsupported on this platform, a custom ROS 2 Foxy installation was configured instead to provide compatibility with the legacy embedded hardware environment.

Following installation of the required ROS dependencies and development utilities, a lightweight sensing workspace was created and compiled using the colcon build system. Automatic workspace sourcing was again configured through the .bashrc file to simplify deployment and runtime setup.

Example environment setup procedure:

```
source ~/ros2_foxy/install/setup.bash
cd ~/ROS_Robot
colcon build
source install/setup.bash
```

This environment was used primarily for sensing-only deployment, sensor validation, and distributed ROS communication testing on the Jetson TX2i platform. See *Appendix B* for resources on this installation.

6.4.2 | Package Installation and Deployment

Once the ROS 2 environment had been configured, deployment of the localisation framework primarily consisted of cloning the required workspace repository and building the included ROS packages. All required localisation, mapping, launch, and sensor integration packages were contained directly within the repository workspaces, removing the need for additional manual driver installation during deployment.

For Ubuntu 22.04 and ROS 2 Humble systems, the primary deployment workspace was 'ROS_Deploy' for full SLAM operation, whilst 'ROS_Robot_Humble' provided sensing-only functionality. For Ubuntu 18.04 and the custom ROS 2 Foxy environment, the lightweight 'ROS_Robot' workspace was used primarily for sensing-only deployment on the Jetson TX2i platform.

Workspace deployment followed the standard ROS 2 clone and colcon build workflow:

```
git clone https://github.com/maclajurekifl/ROS_Deploy.git
cd ROS_Deploy
colcon build
source install/setup.bash
```

Equivalent deployment steps were used for both the 'ROS_Robot' and 'ROS_Robot_Humble' workspaces.

An optional custom .bashrc configuration file was additionally provided within the repository to simplify deployment and runtime operation. This included automatic workspace sourcing, environment setup shortcuts, ROS domain configuration, and launch aliases to reduce repetitive terminal setup during testing and deployment.

Prior to runtime operation, the Livox MID-360 LiDAR additionally required network configuration and IP validation. The LiDAR IP address first had to be identified through terminal inspection of connected network devices:

```
arp -a
```

Once identified, the detected LiDAR IP address was required to replace the "ip" parameter located under "lidar_configs" within:

```
src/livox_ros_driver2/config/MID360_config.json
```

Following modification of the configuration file, the workspace then required rebuilding:

```
colcon build
source install/setup.bash
```

In most deployment cases, stable LiDAR communication additionally required disabling conflicting Wi-Fi routing behaviour whilst using the Ethernet interface connected to the LiDAR. Ethernet IPv4 settings were configured manually to ensure both the host device and Livox sensor remained on the same subnet, allowing successful point cloud and embedded IMU communication through the ROS driver.

Once the workspace had been built and network communication established, the localisation framework could then be launched directly through the provided ROS 2 launch files and deployment configurations.

6.4.3 | Runtime Procedure and Launch Sequence

Describe the runtime startup workflow used to initialise sensors, localisation, mapping, and visualisation systems.

Following successful package deployment and workspace configuration, the localisation framework could be launched through the provided ROS 2 launch system. Separate runtime procedures were provided depending on whether the target platform was responsible for full SLAM execution or sensing-only operation. This separation simplified deployment considerably and reduced unnecessary computational load on embedded sensing platforms. The primary full-stack workspace used throughout development was 'ROS_Deploy', which handled LiDAR integration, EKF localisation, LiDAR odometry, optional FAST-LIO operation, mapping, and visualisation. Sensor-only deployment workspaces were additionally provided for both ROS 2 Humble and ROS 2 Foxy systems, allowing sensing hardware to operate independently whilst publishing sensor streams across DDS to a separate host running the main localisation stack.

6.4.3.1 | Full SLAM Deployment

The complete localisation and SLAM framework was launched through the primary bringup launch file:

```
source /opt/ros/humble/setup.bash
source ~/ROS_Deploy/install/setup.bash
ros2 launch ros_project_bringup launch_slam.launch.py
```

The launch system supported multiple runtime arguments allowing individual subsystems to be enabled, disabled, or replaced dynamically during deployment. Common runtime launch arguments are summarised in Table 11.

Table 16 - SLAM Basic Launch Arguments

Launch Argument	Typical Values	Purpose
launch_sensors	true / false	Enable or disable onboard sensor drivers
start_livox_driver	true / false	Start or skip Livox driver initialisation
start_rviz	true / false	Enable or disable RViz visualisation
use_lidar_fusion	true / false	Enable LiDAR odometry correction within the EKF

use_lio	true / false	Replace NDT odometry with FAST-LIO odometry
start_keyframe_map	true / false	Enable or disable keyframe mapping
keyframe_loop_closure_enable	true / false	Enable loop closure logic
start_pose_graph	true / false	Enable pose graph optimisation
ekf_lidar_fuse_z_from_odom	auto, true, false	Control EKF z-axis fusion behaviour
bringup_config	File path	Override default YAML configuration

6.4.3.2 | Sensor Deployment

For sensing-only operation on ROS 2 Humble systems, the lightweight robot bringup launch file was used:

```
source /opt/ros/humble/setup.bash
source ~/ROS_Robot_Humble/install/setup.bash
ros2 launch ros_robot_bringup robot_sensors.launch.py
```

This workspace primarily handled Livox sensing, Microstrain IMU publication, static transforms, and DDS communication to an external localisation host.

The primary runtime configuration file was:

```
ROS_Robot_Humble/src/ros_robot_bringup/config/robot_sensors_bringup.yaml
```

This configuration controlled sensor enabling, Livox extrinsic calibration, Microstrain configuration, IMU mounting offsets, and runtime sensing behaviour.

6.4.3.3 | Sensor Deployment

For legacy Jetson TX2i deployment using ROS 2 Foxy, the equivalent sensing-only runtime procedure was used:

```
source /opt/ros/foxy/setup.bash
source ~/livox_ws/install/setup.bash
source ~/ROS_Robot/install/setup.bash
ros2 launch ros_robot_bringup robot_sensors.launch.py
```

The primary runtime configuration file for this deployment was:

```
ROS_Robot/src/ros_robot_bringup/config/robot_sensors_bringup.yaml
```

This deployment configuration allowed older embedded hardware platforms to continue publishing sensing data into the ROS 2 middleware layer whilst offloading computationally intensive SLAM processing onto more capable hardware platforms.

6.5 | SUMMARY

Overall, the methodology implemented throughout this project followed a staged and highly iterative engineering workflow, progressing from initial ROS 2 environment setup and sensor

validation through to localisation development, LiDAR odometry refinement, sensor fusion integration, mapping implementation, and final embedded deployment.

A major focus throughout development was maintaining subsystem modularity and deployment flexibility, allowing sensing, localisation, mapping, and visualisation components to be validated both independently and as part of the complete SLAM framework. Playback-based testing using recorded rosbag datasets was utilised extensively throughout refinement, allowing repeatable validation of localisation behaviour, TF consistency, mapping performance, and runtime communication prior to live deployment.

The resulting framework successfully combined IMU-based prediction through the EKF estimator with LiDAR-based correction and keyframe mapping, forming a complete localisation and SLAM pipeline suitable for deployment across both modern and legacy embedded robotic platforms.

7 | System Development and Integration

This section provides a look into the most significant development milestones as well as a general description to ‘fill in the gaps’; as with any development, there was a myriad of issues throughout the process, but it is simply not realistic to include them all. These including will allow for a majority of issues to be solved for those who encounter them as well as demonstrate the skill-set scope of the project.

7.1 | DEVELOPMENT OVERVIEW

Generally, our approach remained consistent; major development alterations were more a case of “we need to add this feature to compensate for this expected behaviour or system limitation” rather than complete redesigns of the framework itself. The overall localisation architecture, prediction-correction structure, and modular ROS 2 deployment approach remained largely unchanged throughout development, with most refinement stages focusing on improving robustness, deployment flexibility, stability, and interoperability between the sensing, localisation, and mapping subsystems.

7.2 | Hardware

7.2.1 | Challenges and Solutions

7.2.1.1 | Jetson Orin Nano Mount

The initial iteration of the Jetson Orin Nano mount assumed that the length of the board extending beyond the mounting-hole region was symmetrical on both sides of the enclosure. However, after physical test fitting, it was identified that this assumption was incorrect, resulting in the Jetson enclosure being positioned off-centre relative to the mount. As a result, the Jetson Orin Nano could not be seated correctly within the intended placement area. Figure 77 displays the resulting alignment issue.

The following iteration resolved this issue by extending the right-hand side of the mount, increasing the effective base length of the enclosure support area. This allowed the Jetson enclosure to sit centrally within the mount while maintaining proper clearance and alignment with the mounting holes.



Figure 77 - Incorrect Jetson Mount

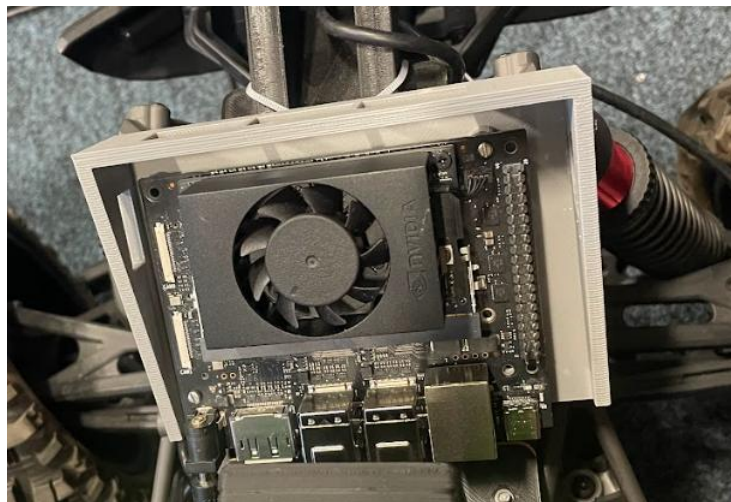


Figure 78 - Final Jetson Mount

7.2.1.2 | Batteries and correct storage

During development, a limitation of the mobile power station was identified regarding its minimum operating voltage. The power station would continue drawing power from the connected battery until the input voltage dropped below approximately 6V. Initially, this behaviour was not known, which resulted in one of the 11.1V LiPo batteries becoming excessively discharged to approximately 6.2V.

Although partial recovery attempts were made using a NiMH charging recovery method, repeated over-discharge events caused permanent degradation to multiple LiPo batteries over time. Additionally, it proved difficult to accurately determine the remaining runtime of the

battery during extended development sessions, particularly when continuously powering the embedded compute system and sensors.

To mitigate this issue during development, the robot was instead powered using a modified laptop power supply, where the original connector was removed and adapted to the existing power input. This provided a significantly more stable and predictable power source for prolonged testing and debugging sessions. The LiPo battery system was then reserved primarily for deployment and untethered runtime testing. Future development would benefit from additional power-management considerations, such as voltage monitoring, low-voltage cutoff protection, or dedicated battery management circuitry.

7.2.1.3 | *IMU USB Port*

As the IMU operates at a relatively high update rate, consideration was initially given to connecting the sensor directly to one of the motherboard USB ports instead of routing it through the external USB hub. This was done to determine whether the hub introduced measurable latency, instability, or packet loss that could negatively impact localisation performance.

However, after evaluating the received IMU topic data within ROS 2, no significant degradation in performance was observed when using the USB hub. The `/imu/data` topic maintained a stable update rate of approximately 199 Hz, with only minor timing variation observed between samples. The measured standard deviation remained relatively low, indicating that the USB hub did not introduce any substantial communication instability during operation.

As a result, it was decided to retain the USB hub configuration, as it simplified cable management and hardware integration without producing any meaningful reduction in IMU performance for the intended localisation pipeline.

7.2.1.4 | *Wi-Fi Adapter*

Nearing the final stages of deployment and testing on the Jetson platform, an abrupt network failure was encountered where the system could no longer establish a wireless connection regardless of software troubleshooting attempts. During inspection, it was observed that the blue activity LED associated with the wireless adapter was no longer illuminated. Reconnecting the adapter and testing alternative devices produced the same result, suggesting a hardware-related issue rather than a software or driver problem.

As an interim workaround, the original developer-kit WiFi antenna was reattached directly to the Jetson network card and temporarily secured externally to the enclosure. Figure 79 displays this makeshift antenna arrangement during testing. This restored wireless functionality; however, the effective connection range remained extremely poor, with reliable operation only occurring within approximately 4 metres during development testing.

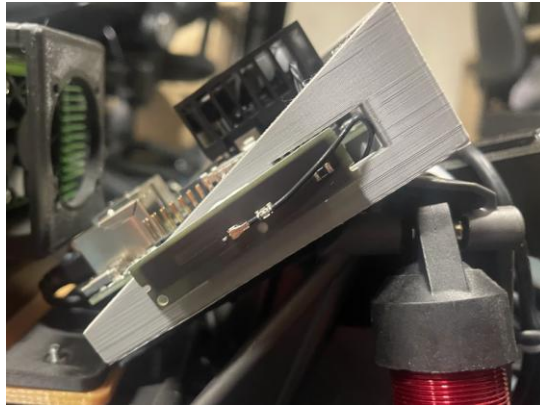


Figure 79 – Wi-Fi Antenna Implementation

Following this, further inspection of the USB wireless adapter itself identified physical damage near the USB connector region. Figure 80 displays the damaged adapter, with the failed component circled in red. A surface-mount capacitor positioned extremely close to the connector interface had detached from the PCB.

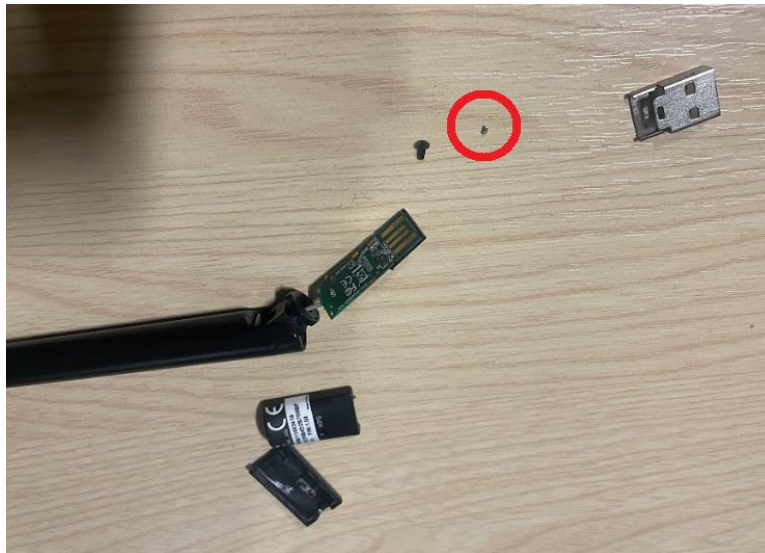


Figure 80 - Broken Wi-Fi Adapter

It is believed that the location of this capacitor made it highly susceptible to mechanical stress during robot operation. Due to aggressive robot movement and vibration, slight instability within the USB hub connection likely caused repeated connector wobble. Over time, this transferred force directly into the surrounding PCB region, eventually causing the capacitor and associated pads to fail.

To repair the issue, the damaged PCB pads were first cleaned and restored before the capacitor was micro-soldered back into position. Figure 81 shows the repair process. Following successful electrical restoration, epoxy adhesive was applied around the repaired connector region as a reinforcement measure to better distribute mechanical stress during future operation. Figure 81 displays the final reinforced repair.

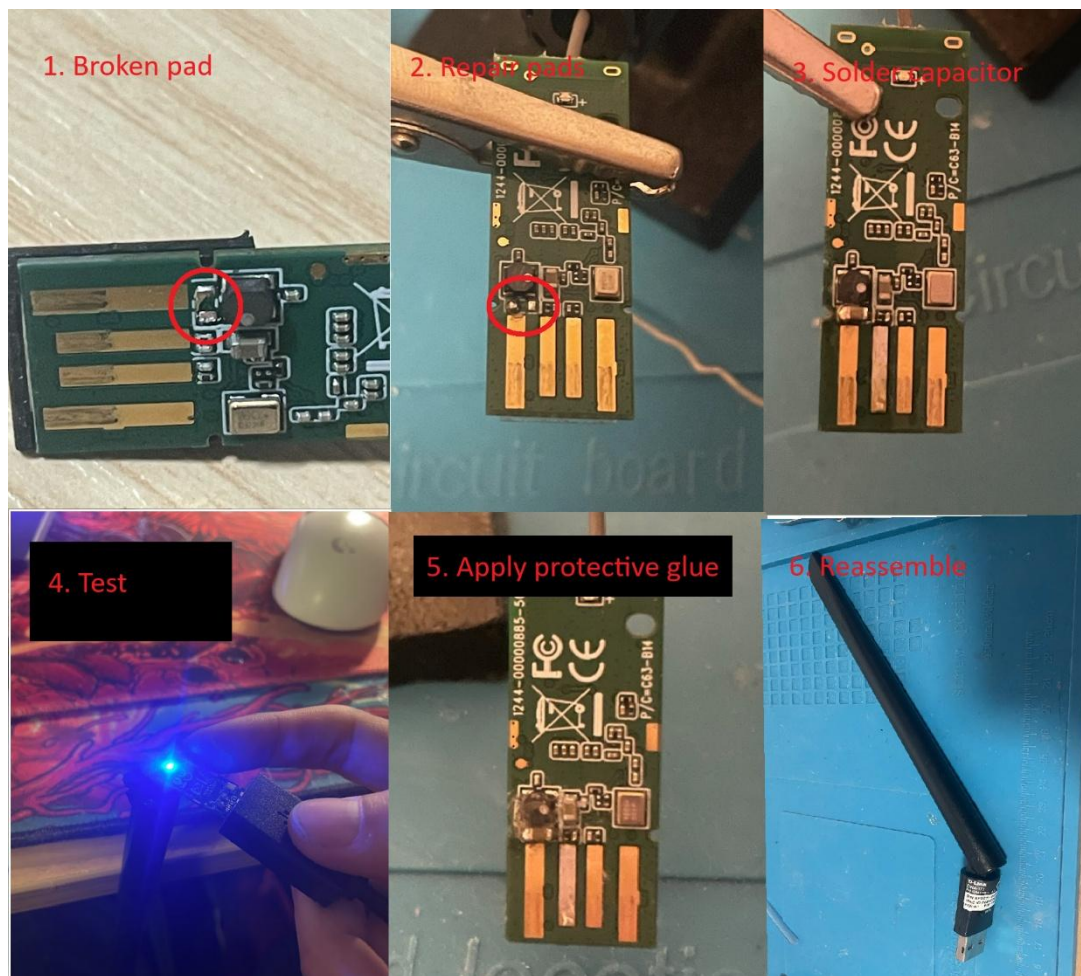


Figure 81 - Wi-Fi Adapter Repair Process

7.2.1.5 | Jetson Nano Failure

During development, an unfortunate hardware failure occurred due to an incorrect power-input assumption regarding the Jetson Orin Nano. Previously, a laptop power supply had been modified by removing its original connector to provide a stable external power source for development purposes. As this supply had already been functioning correctly elsewhere within the system, it was later attempted as a direct power source for the Jetson Orin Nano.

However, insufficient verification was carried out regarding the suitability of the USB-C interface for direct voltage injection from the barrel-style power supply. Unlike regulated USB-C power negotiation methods, the modified supply immediately outputs voltage once connected. The Jetson Orin Nano USB-C interface was not designed for this direct-input configuration, resulting in catastrophic failure of the onboard power-management IC shortly after connection.

The failure rendered the Jetson unusable, requiring complete replacement of the unit. Fortunately, a secondary Jetson Orin Nano was available, allowing development and deployment work to continue with minimal delay. Nevertheless, the mistake resulted in a significant financial cost and highlighted the importance of verifying embedded power-input

requirements before applying external supplies, particularly on modern development boards containing dedicated power-negotiation circuitry.

7.2.1.6 | *Robot Speed*

During initial hardware testing, the robot platform was found to be excessively sensitive and aggressive for safe indoor operation. Small throttle inputs resulted in rapid acceleration and abrupt movement, which introduced a safety concern for both nearby individuals and surrounding equipment during development and testing sessions.

To mitigate this issue, research was conducted into the electronic speed controller configuration options provided by the robot platform. It was identified that the controller supported a reduced-performance “Training Mode”, designed to limit throttle sensitivity and overall output power. This mode significantly improved controllability for indoor testing and reduced the likelihood of accidental collisions or unstable motion during localisation experiments.

The training mode was enabled by holding the EZ-Set button on the controller until the LED indicator sequence was observed. The controller cycles through multiple operating profiles, indicated through progressive LED blink counts. The required mode was selected once the LED sequence reached the appropriate training-mode indication during the one-blink, two-blink, and three-blink transition stages. After enabling this mode, the robot became substantially more manageable for indoor SLAM and localisation testing.

7.3 | Software

7.3.1 | Challenges and Solutions

7.3.1.1 | *TX2i Driver Compatibility*

During deployment onto the TX2i platform, compatibility issues were encountered between the Livox ROS driver and the ROS 2 Foxy environment running on the system. Certain libraries and dependencies utilised within the driver were either unsupported or incompatible with the older ROS 2 distribution, preventing successful compilation and deployment.

To resolve this, modifications were made directly to the Livox driver implementation and associated package dependencies. Specifically, several components related to extended custom-message functionality were removed from the build process. These features primarily existed to support additional custom topic-message implementations that were not required for the intended localisation and SLAM pipeline.

As the core LiDAR and IMU data streams remained unaffected by removing these dependencies, there was no negative impact on the required system functionality. Following these modifications, the driver compiled successfully under ROS 2 Foxy and allowed stable communication with the Livox sensor on the TX2i platform.

7.3.1.3 | IMU Setup

During setup of the IMU on the TX2i platform, communication issues were encountered due to incorrect device IP configuration within the driver files. Unlike the LiDAR configuration, the required IP settings were not located within the primary configuration directory and instead existed within nested internal configuration files, making the issue significantly more difficult to identify.

To locate the relevant configuration entries, iterative grep searches were used throughout the workspace to identify all files containing the previous IP configuration. Each discovered configuration file was then manually inspected and corrected before rebuilding the workspace.

```
mac1a@MACLA-UBUNTU-DESKTOP: /opt/ros/humble/share/microstrain_inertial_driver/microstrain_inertial_driver_common/config$ ls
params.yml
```

Figure 85 - Microstrain Configuration File Location

Following recompilation and deployment, stable IMU communication was successfully established.

```
^Cmac1a@MACLA-UBUNTU-DESKTOP:~/ROS_Deployment$ ros2 topic hz /imu/data
average rate: 27.053
    min: 0.000s max: 0.307s std dev: 0.08025s window: 31
average rate: 29.764
    min: 0.000s max: 0.307s std dev: 0.07548s window: 64
average rate: 32.964
    min: 0.000s max: 0.307s std dev: 0.06483s window: 107
average rate: 34.355
    min: 0.000s max: 0.307s std dev: 0.06153s window: 146
average rate: 36.865
    min: 0.000s max: 0.307s std dev: 0.05648s window: 199
```

Figure 86 - IMU Data-Stream

7.3.1.4 | Auto-Starting Program

As development progressed, repeatedly sourcing workspaces, configuring environments, and manually launching drivers became increasingly time consuming, particularly when debugging across multiple devices. To reduce setup overhead, a series of runtime shortcuts and autorun scripts were developed for both the desktop and robot platforms.

These scripts automated common tasks such as:

- sourcing ROS 2 environments,
- configuring network settings,
- launching sensor drivers,
- starting SLAM components,
- and restoring communication settings between devices.

An example deployment workflow used on the robot platform is shown below:

```
#!/bin/bash
```

```
source /opt/ros/humble/setup.bash
```



```

source ~/livox_ws/install/setup.bash
source ~/ROS_Deployment/install/setup.bash

export ROS_DOMAIN_ID=10
export ROS_LOCALHOST_ONLY=0
export RMW_IMPLEMENTATION=rmw_fastrtps_cpp

sudo ip addr flush dev enp58s0
sudo ip addr add 192.168.1.5/24 dev enp58s0
sudo ip link set enp58s0 up

ros2 launch ros_project_bringup launch_slam.launch.py \
  use_lio:=true \
  launch_sensors:=true

```

This significantly reduced deployment time and improved consistency when repeatedly restarting the system during testing and debugging.

7.3.1.5 | Data Disruption

During development, unstable network behaviour was observed when relying on the built-in Jetson Orin Nano antenna configuration for wireless communication. Although a connection was established, data transfer was highly inconsistent, resulting in unstable motion estimation and significant delay within the keyframe-mapping pipeline.

This caused distorted SLAM behaviour during robot movement, particularly during faster motion. Following investigation, the issue was linked to the damaged external WiFi adapter discussed previously. After repair of the adapter, network stability improved significantly and normal localisation and mapping behaviour was restored.

Figure 87 displays an example of a runtime with bad data; imu/data_raw only published 2551, whereas the livox/imu published 8175: significantly more than the standalone IMU.

```

mac1a@MACLA-UBUNTU-DESKTOP: ~/bags/new$ ros2 bag info run13
Files:          run13_0.db3
Bag size:       12.7 MiB
Storage id:     sqlite3
Duration:       123.523649571s
Start:          May  5 2026 22:37:45.568597357 (1778017065.568597357)
End:            May  5 2026 22:39:49.092246928 (1778017189.092246928)
Messages:       34840
Topic information:
Topic: /livox/imu | Type: sensor_msgs/msg/Imu | Count: 8175 | Serialization Format: cdr
Topic: /tf | Type: tf2_msgs/msg/TFMessage | Count: 19591 | Serialization Format: cdr
Topic: /imu/data_raw | Type: sensor_msgs/msg/Imu | Count: 2551 | Serialization Format: cdr
Topic: /imu/data | Type: sensor_msgs/msg/Imu | Count: 4336 | Serialization Format: cdr
Topic: /livox/lidar | Type: sensor_msgs/msg/PointCloud2 | Count: 9 | Serialization Format: cdr
Topic: /ekf/status | Type: microstrain_inertial_msgs/msg/HumanReadableStatus | Count: 110 | Serialization Format: cdr
Topic: /parameter_events | Type: rcl_interfaces/msg/ParameterEvent | Count: 1 | Serialization Format: cdr
Topic: /rosout | Type: rcl_interfaces/msg/Log | Count: 53 | Serialization Format: cdr
Topic: /tf_static | Type: tf2_msgs/msg/TFMessage | Count: 14 | Serialization Format: cdr

```

Figure 87 - Bad Data Info

7.4 | Localisation and SLAM

7.4.1 | Challenges and Solutions

7.4.1.1 | *Keyframe Smear*

Significant keyframe smear was initially observed during robot motion, particularly during more aggressive acceleration and turning behaviour. Investigation suggested that the issue was related to scan deskew and motion compensation timing.

To improve this, the built-in Livox MID-360 IMU was utilised directly for LiDAR deskew and scan handling operations, as the IMU is physically integrated within the LiDAR itself. This provided a more accurate representation of the LiDAR sensor motion during scan acquisition compared to using externally mounted IMU data.

Following this adjustment, scan consistency and keyframe stability improved noticeably during motion.

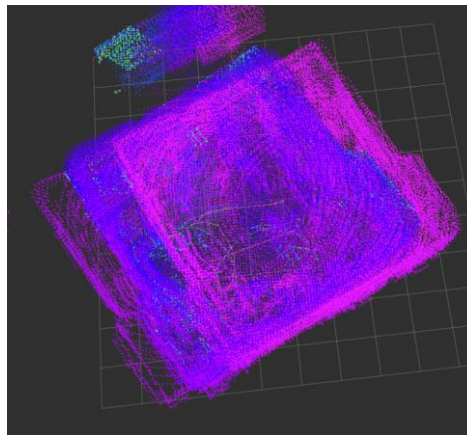


Figure 88 - Keyframe Smear

7.4.1.2 | *Sensor Relative Pose Tuning*

Initially, the LiDAR and IMU extrinsic parameters were not correctly configured, resulting in severely incorrect motion estimation and mapping behaviour. The most obvious indicator of this issue was that forward robot motion would appear as backwards motion within the SLAM output.

To resolve this, measured positional and rotational offsets for both sensors were manually integrated into the driver configuration files. For the prototype setup, the IMU was configured with a yaw rotation of -90° and positional offsets of approximately (12, -0.7, 38), while the LiDAR was configured with positional offsets of (12, 0.7, 38) and a yaw rotation of 180° .

After applying the corrected extrinsic parameters and axis orientations, motion estimation aligned correctly with real robot movement and mapping behaviour improved substantially.

7.4.1.3 | *Fixing NDT*

During initial deployment, persistent skew and tilt were observed within the generated map, particularly during acceleration, braking, and uneven floor transitions. Investigation

suggested that this behaviour was influenced by the physical placement of the LiDAR relative to the robot chassis and suspension system.

As the LiDAR was mounted above the suspension-supported body of the robot, small variations in chassis pitch and roll introduced unintended angular offsets into the scan data during motion. These effects became more noticeable during agile movement, where suspension compression and rebound altered the effective LiDAR orientation relative to the ground plane.

To mitigate this issue, an auto-levelling approach was introduced within the mapping pipeline to compensate for these orientation changes and stabilise the generated map. Following this adjustment, map consistency and overall alignment improved significantly during runtime.

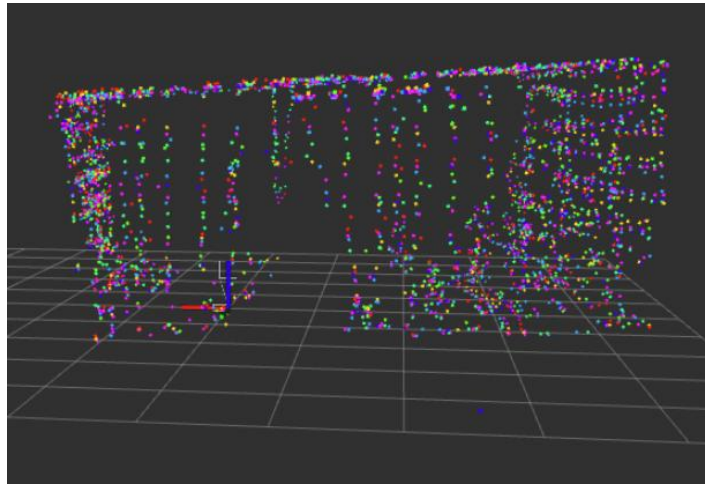


Figure 89 - Tilted Keyframe Map

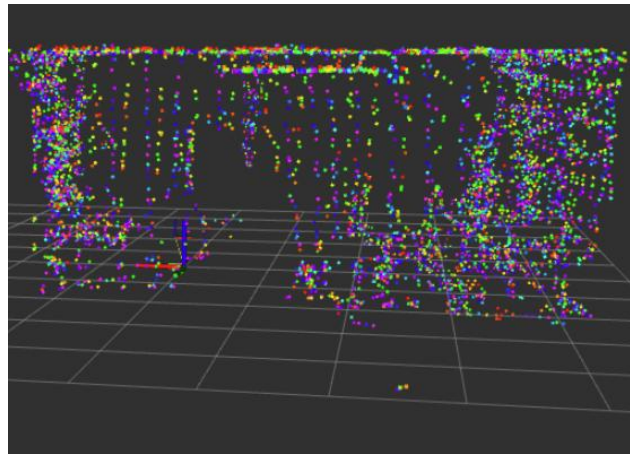


Figure 90 - Correct Keyframe map

7.4.1.4 | Object Obstruction

During testing, portions of the robot itself, including the RTX sensor housing and other nearby mounted components, were repeatedly detected within the LiDAR scans. This introduced unwanted noise and artefacts into the generated map, particularly around the immediate vicinity of the robot platform.

To mitigate this, a local exclusion region was introduced around the robot where LiDAR points would be ignored during processing. This prevented nearby robot-mounted structures from continuously polluting the scan data and keyframe map.

As the robot moved through the environment, valid environmental data within these excluded regions could still be recovered naturally through later scans from different viewpoints, allowing map consistency to remain unaffected while significantly reducing self-detection artefacts.



Figure 91 - RTK GPS Obstruction

7.5 | System Optimisation and Engineering Judgement

Throughout development, the system was continuously refined to improve runtime stability, localisation consistency, and real-time performance on embedded hardware. Optimisation primarily focused on balancing scan quality, computational load, timing consistency, and mapping reliability during live robot operation.

The primary optimisation area was the LiDAR odometry front-end, particularly the NDT scan-matching pipeline. Multiple parameter sweeps were performed across voxel size, NDT resolution, and fitness thresholds to balance registration stability against continuous update behaviour. Deskew handling and extrinsic calibration were also heavily refined, as incorrect sensor alignment initially caused significant mapping instability and rotational smear during motion. Using the integrated Livox IMU directly for LiDAR deskewing improved scan consistency substantially.

The EKF implementation was intentionally maintained as a simplified planar model to improve runtime efficiency. LiDAR correction weighting was also balanced against IMU influence to prioritise stable and repeatable trajectory behaviour rather than aggressive responsiveness.

Back-end mapping processes introduced additional computational trade-offs. Keyframe mapping and optional pose-graph refinement improved global consistency but increased CPU

and memory usage. To maintain reliable odometry performance during constrained runs, optional back-end modules could be disabled independently when required. Deployment optimisation also extended beyond the SLAM algorithms themselves. Runtime automation, ROS 2 launch overlays, parameter externalisation, and reusable utility scripts significantly reduced debugging overhead and improved repeatability across deployment scenarios.

Table 17 - SLAM Component Optimisation

Optimisation Area	Benefit	Trade-Off
NDT parameter tuning	Improved scan stability and odometry consistency	Increased tuning complexity
LiDAR deskewing using integrated IMU	Reduced rotational smear and scan distortion	Greater dependency on TF accuracy
Planar EKF motion model	Reduced runtime cost and improved robustness	Reduced dynamic-motion representation
Strong LiDAR correction weighting	Reduced long-term drift	Reduced IMU influence
Keyframe mapping	Improved global map quality	Increased CPU and memory usage
Optional back-end processes	Graceful degradation under limited compute	Reduced global refinement capability
ROS 2 launch overlays and automation	Faster debugging and deployment	Increased configuration surface area

8 | Results and Evaluation

8.1 | Results

The final system demonstrated successful real-time localisation and mapping operation using the developed LiDAR–IMU SLAM framework on the robotic platform. Following iterative refinement of the NDT odometry pipeline, EKF fusion, LiDAR deskewing, and sensor calibration, the system was capable of generating stable keyframe maps while maintaining continuous pose estimation during live robot operation.

Figure 92 displays a generated keyframe map with the estimated robot path visible throughout operation. The resulting map demonstrates generally consistent environmental reconstruction and trajectory behaviour during indoor testing. Following optimisation, keyframe smear and rotational distortion were significantly reduced compared to earlier development stages.

Figure 93 displays runtime loop-closure logs associated with the mapping session shown in Figure 92. Successful loop-edge insertion and pose-graph updates were observed during runtime, indicating that the system was capable of identifying revisited regions and applying global refinement corrections to the generated map.

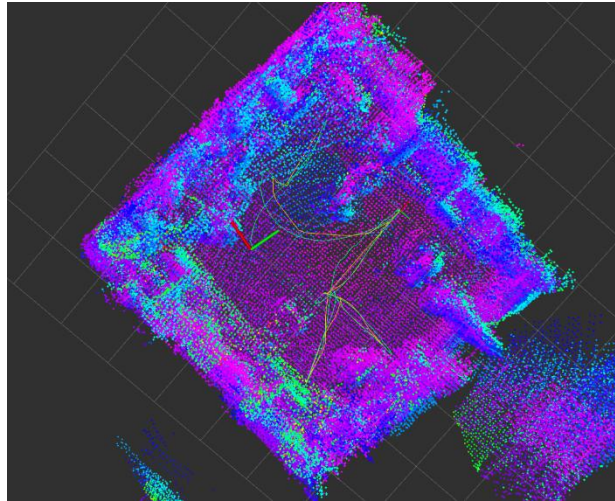


Figure 92 - Keyframe Map with Path Visible

```
keyframe_map_node-10 [INFO] [1778159120.656788338] [keyframe_map_node] keyframes: 20, map points: 1182
keyframe_map_node-10 [INFO] [1778159120.781484001] [keyframe_map_node] Pose graph applied to map: 20 keyframes, 6082 map pts
keyframe_map_node-10 [INFO] [1778159121.318052936] [keyframe_map_node] Pose graph applied to map: 21 keyframes, 6824 map pts
keyframe_map_node-10 [INFO] [1778159122.087672234] [keyframe_map_node] Pose graph applied to map: 22 keyframes, 6844 map pts
keyframe_map_node-10 [INFO] [1778159122.568815615] [keyframe_map_node] Pose graph applied to map: 23 keyframes, 7808 map pts
keyframe_map_node-10 [INFO] [1778159123.131490681] [keyframe_map_node] Pose graph applied to map: 24 keyframes, 7170 map pts
keyframe_map_node-10 [INFO] [1778159123.984477872] [keyframe_map_node] Pose graph applied to map: 25 keyframes, 7236 map pts
keyframe_map_node-10 [INFO] [1778159124.405312865] [keyframe_map_node] Pose graph applied to map: 26 keyframes, 7316 map pts
keyframe_map_node-10 [INFO] [1778159125.245254410] [keyframe_map_node] Pose graph applied to map: 27 keyframes, 7484 map pts
keyframe_map_node-10 [INFO] [1778159125.768463594] [keyframe_map_node] Pose graph applied to map: 28 keyframes, 7496 map pts
keyframe_map_node-10 [INFO] [1778159126.444878859] [keyframe_map_node] Loop closure: new_kf=28 matches past_kf=0 overlap=1.000
pose_graph_node-11 [INFO] [1778159126.445093551] [pose_graph_node] Pose graph: added loop edge (0, 28)
keyframe_map_node-10 [WARN] [1778159126.522152718] [keyframe_map_node] pose_graph path len 28 vs keyframes 29 batches 29 - skip map rebuild
keyframe_map_node-10 [INFO] [1778159126.855549516] [keyframe_map_node] Pose graph applied to map: 29 keyframes, 7636 map pts
keyframe_map_node-10 [INFO] [1778159127.398477946] [keyframe_map_node] Pose graph applied to map: 30 keyframes, 7636 map pts
keyframe_map_node-10 [INFO] [1778159128.195143545] [keyframe_map_node] Pose graph applied to map: 31 keyframes, 7721 map pts
keyframe_map_node-10 [INFO] [1778159128.828947761] [keyframe_map_node] Pose graph applied to map: 32 keyframes, 7793 map pts
keyframe_map_node-10 [INFO] [1778159129.421621378] [keyframe_map_node] Pose graph applied to map: 33 keyframes, 7869 map pts
keyframe_map_node-10 [INFO] [1778159130.0013346839] [keyframe_map_node] Pose graph applied to map: 34 keyframes, 7932 map pts
keyframe_map_node-10 [INFO] [1778159130.617839486] [keyframe_map_node] Pose graph applied to map: 35 keyframes, 8018 map pts
keyframe_map_node-10 [INFO] [1778159131.266365182] [keyframe_map_node] Pose graph applied to map: 36 keyframes, 8066 map pts
keyframe_map_node-10 [INFO] [1778159131.915867456] [keyframe_map_node] Pose graph applied to map: 37 keyframes, 8141 map pts
keyframe_map_node-10 [WARN] [1778159132.158577397] [keyframe_map_node] pose_graph path len 37 vs keyframes 38 batches 38 - skip map rebuild
keyframe_map_node-10 [INFO] [1778159132.705738776] [keyframe_map_node] Loop closure: new_kf=38 matches past_kf=0 overlap=1.000
pose_graph_node-11 [INFO] [1778159132.706275033] [pose_graph_node] Pose graph: added loop edge (0, 38)
keyframe_map_node-10 [INFO] [1778159133.364047722] [keyframe_map_node] keyframes: 40, map points: 15345
keyframe_map_node-10 [WARN] [1778159133.868982454] [keyframe_map_node] pose_graph path len 44 vs keyframes 47 batches 47 - skip map rebuild
keyframe_map_node-10 [INFO] [1778159139.342466648] [keyframe_map_node] Loop closure: new_kf=48 matches past_kf=0 overlap=1.000
pose_graph_node-11 [INFO] [1778159140.863921617] [pose_graph_node] Pose graph: added loop edge (0, 48)
keyframe_map_node-10 [WARN] [1778159143.076743754] [keyframe_map_node] pose_graph path len 48 vs keyframes 55 batches 55 - skip map rebuild
keyframe_map_node-10 [INFO] [1778159146.123472546] [keyframe_map_node] Loop closure: new_kf=58 matches past_kf=0 overlap=1.000
keyframe_map_node-10 [INFO] [1778159146.543331895] [keyframe_map_node] keyframes: 60, map points: 19998
pose_graph_node-11 [INFO] [1778159147.966539448] [pose_graph_node] Pose graph: added loop edge (0, 58)
keyframe_map_node-10 [WARN] [1778159149.531668811] [keyframe_map_node] pose_graph path len 52 vs keyframes 64 batches 64 - skip map rebuild
keyframe_map_node-10 [INFO] [1778159152.769388058] [keyframe_map_node] Loop closure: new_kf=68 matches past_kf=32 overlap=0.989
keyframe_map_node-10 [WARN] [1778159154.676342078] [keyframe_map_node] pose_graph path len 59 vs keyframes 75 batches 75 - skip map rebuild
keyframe_map_node-10 [INFO] [1778159156.389688396] [keyframe_map_node] keyframes: 80, map points: 46468
pose_graph_node-11 [INFO] [1778159156.898051994] [pose_graph_node] Pose graph: added loop edge (32, 68)
keyframe_map_node-10 [WARN] [1778159163.177843683] [keyframe_map_node] pose_graph path len 69 vs keyframes 97 batches 97 - skip map rebuild
keyframe_map_node-10 [INFO] [1778159164.6321188349] [keyframe_map_node] keyframes: 100, map points: 75438
keyframe_map_node-10 [WARN] [1778159174.172895211] [keyframe_map_node] pose_graph path len 85 vs keyframes 117 batches 117 - skip map rebuild
keyframe_map_node-10 [INFO] [1778159175.907809819] [keyframe_map_node] keyframes: 120, map points: 96585
keyframe_map_node-10 [WARN] [1778159181.838724531] [keyframe_map_node] pose_graph path len 106 vs keyframes 130 batches 130 - skip map rebuild
keyframe_map_node-10 [INFO] [1778159188.243528845] [keyframe_map_node] Loop closure: new_kf=137 matches past_kf=71 overlap=0.836
keyframe_map_node-10 [INFO] [1778159189.498313658] [keyframe_map_node] keyframes: 140, map points: 118851
keyframe_map_node-10 [WARN] [1778159191.356836525] [keyframe_map_node] pose_graph path len 120 vs keyframes 146 batches 146 - skip map rebuild
```

Figure 93 - Loop Closure Logs

Figure 94 compares the generated keyframe map against the corresponding real-world environment image. Visual comparison demonstrates that the resulting map maintained reasonable structural consistency with the physical environment following optimisation and calibration refinement.

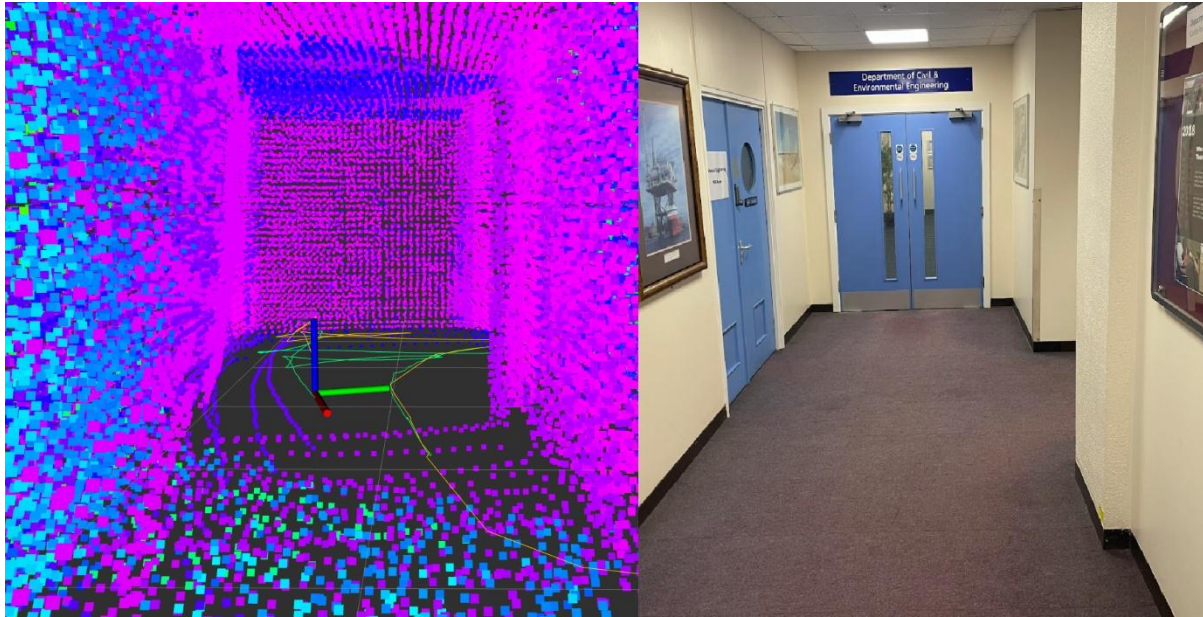


Figure 94 - Keyframe vs Image

Figure 95 displays a comparison between FAST-LIO and the developed NDT-smoothed localisation pipeline. Both systems produced broadly similar trajectory behaviour, particularly during major directional changes and rotational motion. However, small trajectory deviations and smoothing differences were still present between the two approaches.

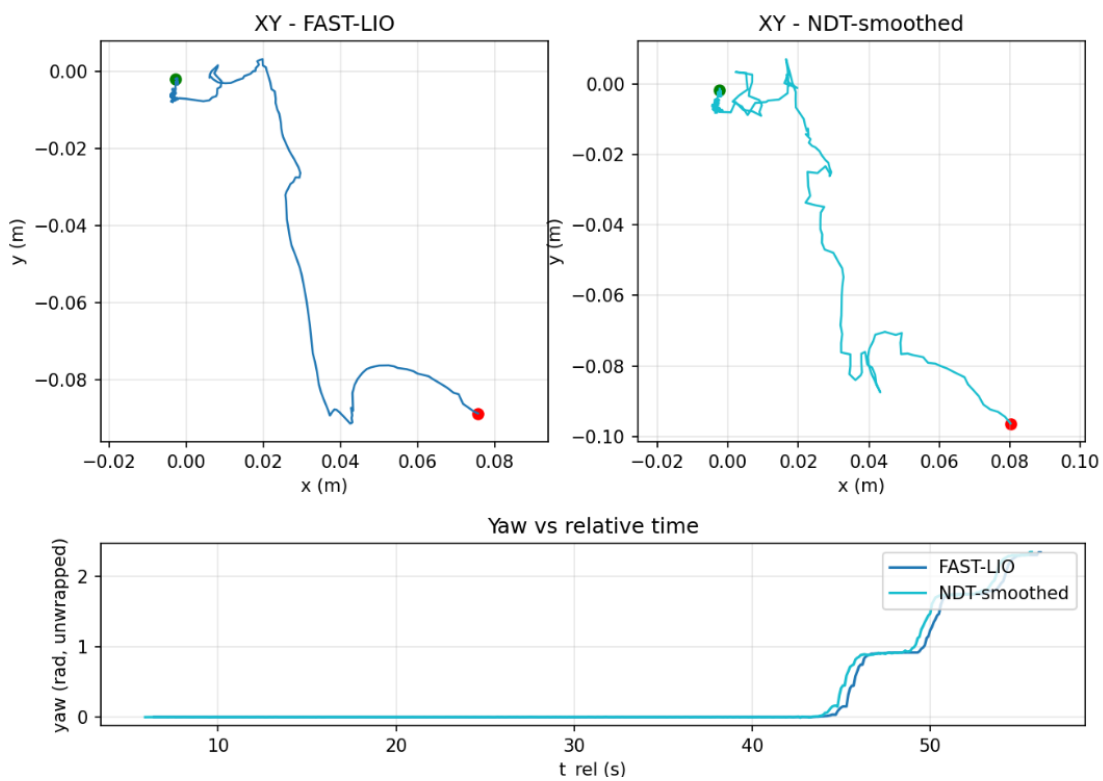


Figure 95 - FAST-LIO Comparison

Figure 96 displays additional views of the generated keyframe maps from alternative perspectives. These views further illustrate overall map consistency, environmental structure retention, and the effectiveness of the deskew and LiDAR alignment refinements introduced throughout development.

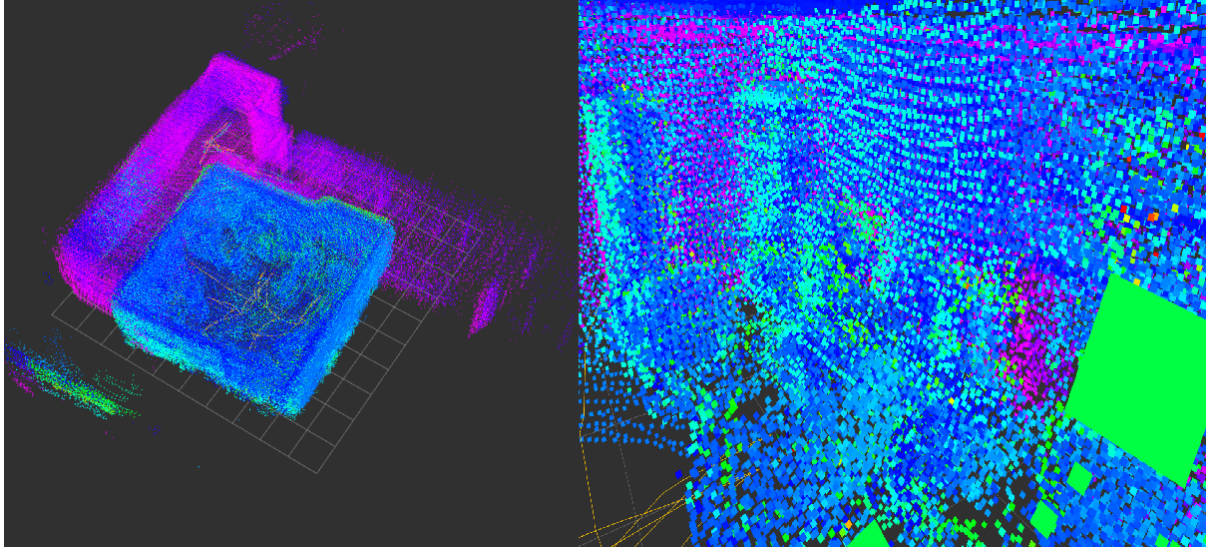


Figure 96 - Keyframe Map Views

Finally, Figure 97 displays the system operating alongside the integrated perception model during runtime. This demonstrates the modularity of the developed ROS 2 framework and its ability to support simultaneous localisation, mapping, and additional robotic-processing pipelines within the same deployment environment.

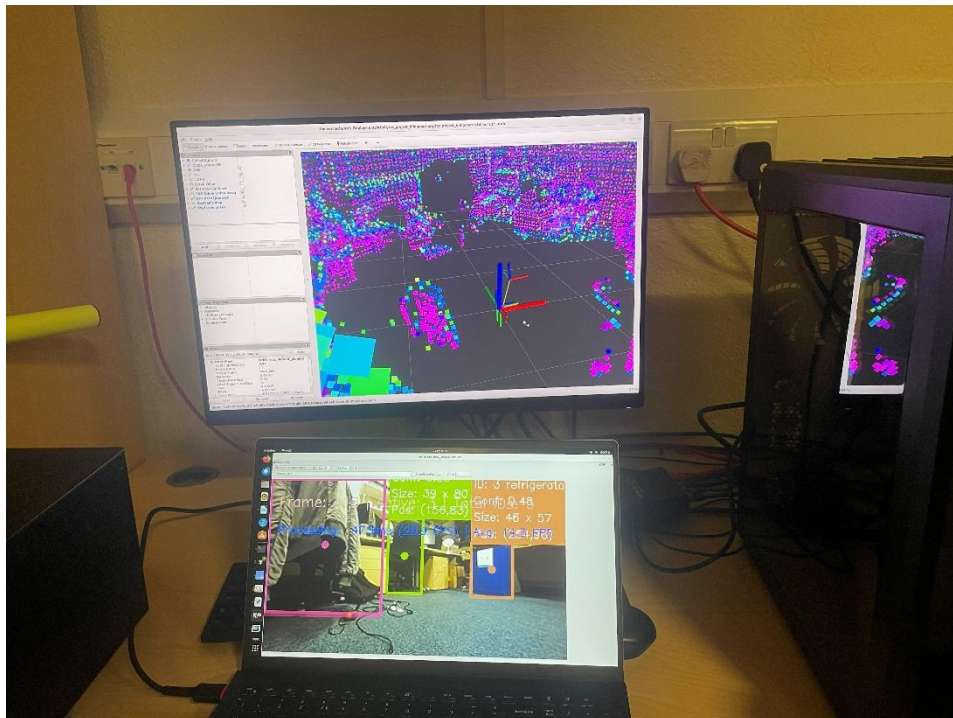


Figure 97 - Runtime with Perception Model

Figure 98 displays the generated keyframe map during operation at approximately 60% throttle. The system maintained continuous localisation and mapping behaviour during higher-speed robot movement.

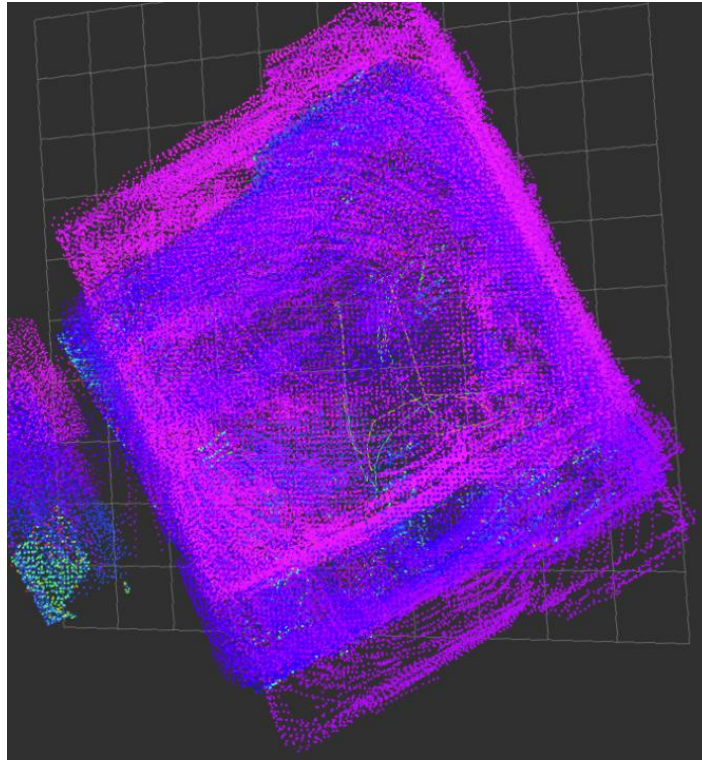


Figure 98 - 60% Throttle Keyframe Map

8.2 | Assessment

Overall, the developed system successfully met the primary objectives and specifications established at the start of the project. The final platform was capable of performing real-time LiDAR–IMU localisation and mapping on embedded robotic hardware while maintaining stable operation during live deployment. Key objectives such as modular ROS 2 deployment, LiDAR odometry, EKF fusion, keyframe mapping, deskewing, and embedded runtime operation were all achieved successfully.

The system also demonstrated successful deployment across both the TX2i and Jetson Orin Nano platforms, with runtime optimisation and configuration overlays allowing flexible deployment under multiple operating conditions. Following refinement of the sensor calibration, deskew pipeline, and NDT parameters, the resulting maps and trajectories demonstrated generally stable and repeatable behaviour during indoor testing.

The only specification not fully completed within the scope of the project was full standalone integration with the external perception system. However, a proof-of-concept implementation was successfully demonstrated, where both the SLAM framework and perception model were operating simultaneously on the same embedded device during runtime, as shown previously in Figure 97. Although further integration work would still be required to combine both systems into a fully unified autonomous framework, the project successfully demonstrated that simultaneous deployment was computationally achievable on the target hardware.

8.3 | ADVANTAGES AND LIMITATIONS

A simple evaluation is suitable for the assessing the current strength of the SLAM system.

Advantages

- Modular ROS 2 architecture allowing independent testing of localisation, mapping, and sensor components
- Support for both custom NDT odometry and FAST-LIO workflows
- Config-driven deployment using YAML overlays and launch configurations
- Robust LiDAR–IMU fusion following optimisation and calibration refinement
- Reduced scan smear through integrated Livox IMU deskewing
- Ability to disable optional back-end processes during constrained runs
- Reusable runtime automation and deployment scripts
- Flexible deployment across TX2i and Jetson Orin Nano platforms
- Improved debugging capability through isolated runtime modes and diagnostics
- Keyframe mapping produced visually consistent indoor maps following refinement

Limitations

- High dependency on correct sensor calibration and TF alignment
- Runtime behaviour highly sensitive to LiDAR–IMU timing consistency
- NDT scan matching could become unstable within weak-geometry environments
- Increased configuration complexity due to large parameter surface
- Back-end mapping processes introduced additional CPU and memory overhead
- Planar EKF model limited representation of full 3D dynamics
- Aggressive robot movement could still introduce mapping instability and potentially motion explosion

Overall, the final system demonstrated stable real-time localisation and mapping behaviour following iterative optimisation and refinement. The modular architecture and configurable deployment workflow proved especially beneficial during debugging and testing, allowing rapid isolation of issues and experimentation across multiple runtime configurations.

However, the project also demonstrated how heavily LiDAR–IMU localisation systems depend on accurate calibration, timing consistency, and stable runtime conditions, particularly when operating on embedded robotic hardware.

8.4 | LESSONS LEARNED

One of the biggest things learned throughout this project was that most major SLAM problems were not actually caused by the algorithms themselves, but by everything around them. A large amount of debugging time ended up being spent on TF alignment, timestamp consistency, network communication, sensor orientation, extrinsic calibration, and deployment configuration rather than the EKF or LiDAR odometry directly. Small mistakes within these areas could completely destroy localisation behaviour, with issues such as backwards motion, rotational smear, unstable mapping, and corrupted keyframes all being observed at different stages of development.

Another major lesson was how important proper sensor calibration and timing consistency are within LiDAR–IMU systems. Correct deskew behaviour depended heavily on accurate frame alignment and timestamp consistency between the LiDAR and IMU. Initially, this was underestimated, especially once aggressive robot movement and suspension effects were introduced. Once the sensor transforms and deskew pipeline were corrected, mapping quality improved significantly.

The project also highlighted how different real-world deployment is compared to simulation. Although simulation-first development massively helped during early testing, deploying onto actual hardware introduced many additional engineering problems that were not originally expected. These included vibration, unstable networking, USB instability, cable management, power delivery issues, mounting problems, and thermal considerations. Several hardware failures also occurred throughout development, including over-discharged LiPo batteries, failure of the wireless adapter due to vibration damage, and destruction of a Jetson Orin Nano after incorrect power application. These mistakes were costly, but also ended up being some of the most valuable learning experiences within the project.

Another thing learned quite quickly was how important physical integration is within robotics projects. The actual placement of the LiDAR and IMU affected mapping quality far more than initially expected, especially once suspension movement and chassis tilt were introduced. Even smaller things such as loose USB connections, unstable wireless communication, or robot-mounted objects appearing in LiDAR scans, ended up having noticeable effects on localisation performance.

Finally, the project reinforced how important modularity and debugging infrastructure are within larger robotic systems. Utility scripts, launch overlays, runtime shortcuts, and isolated operating modes ended up saving huge amounts of development time later in the project. Overall, the final system became less about finding one “perfect” SLAM configuration, and more about building something that was stable, repeatable, diagnosable, and practical to deploy on real hardware.

8.6 | SUMMARY

Overall, the project was successful in achieving its primary objective of developing and deploying a real-time LiDAR–IMU localisation and SLAM framework onto embedded robotic hardware. The final system was capable of generating stable localisation estimates and keyframe maps during live operation while supporting multiple deployment modes and configurable runtime behaviour.

Although a large number of software and hardware challenges were encountered throughout development, these issues ultimately contributed heavily towards the refinement of the final system and overall engineering understanding. The project evolved significantly beyond the initial scope through iterative optimisation, debugging, and deployment refinement, resulting in a modular and practical SLAM framework suitable for future robotic integration and expansion.

9 | Conclusion and Further Work

The primary aim of this project was to develop and deploy a modular LiDAR–IMU localisation and SLAM framework capable of operating on embedded robotic hardware in real-world environments. Overall, this objective was successfully achieved through the development of a ROS 2-based system integrating LiDAR odometry, IMU fusion, keyframe mapping, and embedded deployment across both TX2i and Jetson Orin Nano platforms.

Throughout development, the project evolved significantly beyond the original scope. Initially, the focus was mainly on localisation itself; however, a large amount of development time was ultimately spent on deployment refinement, sensor calibration, runtime optimisation, networking, debugging infrastructure, hardware integration, and overall system reliability. Multiple runtime configurations were successfully implemented, including both the custom NDT-based pipeline and FAST-LIO integration, allowing flexible deployment depending on runtime requirements and available compute resources.

One of the major achievements of the project was successful real-world deployment rather than remaining simulation-only. The final system was capable of generating stable localisation estimates and keyframe maps during live operation while supporting configurable deployment modes and reusable ROS 2 launch infrastructure. Significant improvements were also achieved through refinement of the deskew pipeline, sensor extrinsic calibration, runtime automation, and mapping stability during more aggressive robot movement.

The project also successfully demonstrated simultaneous runtime operation of both the SLAM framework and the perception model on the same embedded device. Although this was not fully integrated into a standalone autonomous framework within the scope of the project, it still provided a successful proof-of-concept for future multi-system robotic deployment.

However, several limitations still remain within the final implementation. Runtime behaviour remained highly sensitive to timing consistency, TF alignment, network stability, and sensor calibration quality. Incorrect extrinsic parameters or unstable communication could quickly result in distorted mapping and unstable odometry behaviour. The simplified planar EKF model also limits representation of full 3D robot dynamics, while NDT scan matching could still become unstable within weak-geometry environments or during highly aggressive movement.

Additionally, optional back-end processes such as keyframe mapping and pose-graph refinement introduced noticeable CPU and memory overhead on embedded hardware. Although the modular architecture allowed these components to be selectively disabled, this still remains a practical limitation during constrained runs.

Several areas for future work were identified throughout development. One of the largest future improvements would be full integration between the localisation framework and the perception system into a fully unified autonomous robotic platform. Additional future work could also include improved loop closure, enhanced global optimisation, dynamic obstacle

handling, GPU acceleration, improved calibration tooling, and further embedded runtime optimisation.

Overall, the project successfully produced a modular, configurable, and deployment-ready localisation and SLAM framework while also providing substantial practical experience in real-world robotic deployment, ROS 2 integration, embedded systems, and LiDAR–IMU sensor fusion.

References

- [1] S. Thrun, W. Burgard, and D. Fox, *Probabilistic Robotics*. Cambridge, MA, USA: MIT Press, 2005. Accessed: Jan. 14, 2026.
- [2] R. Siegwart, I. R. Nourbakhsh, and D. Scaramuzza, *Introduction to Autonomous Mobile Robots*, 2nd ed. Cambridge, MA, USA: MIT Press, 2011. Accessed: Jan. 13, 2026.
- [3] J. Borenstein, H. R. Everett, and L. Feng, “Where am I? Sensors and methods for mobile robot positioning,” Univ. Michigan, Ann Arbor, MI, USA, Tech. Rep., 1996. Accessed: Jan. 14, 2026.
- [4] G. Welch and G. Bishop, “An introduction to the Kalman filter,” Univ. North Carolina at Chapel Hill, Chapel Hill, NC, USA, Tech. Rep. TR 95-041, 2006. Accessed: Jan. 5, 2026.
- [5] S. J. Julier and J. K. Uhlmann, “A new extension of the Kalman filter to nonlinear systems,” in *Proc. AeroSense*, Orlando, FL, USA, 1997, pp. 182–193. Accessed: Dec. 15, 2025.
- [6] R. Mahony, T. Hamel, and J.-M. Pflimlin, “Nonlinear complementary filters on the special orthogonal group,” *IEEE Trans. Autom. Control*, vol. 53, no. 5, pp. 1203–1218, Jun. 2008. Accessed: Nov. 27, 2025.
- [7] T. D. Barfoot, *State Estimation for Robotics*. Cambridge, U.K.: Cambridge Univ. Press, 2017. Accessed: Jan. 10, 2026.
- [8] A. I. Mourikis and S. I. Roumeliotis, “A multi-state constraint Kalman filter for vision-aided inertial navigation,” in *Proc. IEEE Int. Conf. Robot. Autom. (ICRA)*, Rome, Italy, 2007, pp. 3565–3572. Accessed: Dec. 09, 2025.
- [9] P. J. Besl and N. D. McKay, “A method for registration of 3-D shapes,” *IEEE Trans. Pattern Anal. Mach. Intell.*, vol. 14, no. 2, pp. 239–256, Feb. 1992. Accessed: December. 28, 2025.
- [10] E. Olson, “Real-time correlative scan matching,” in *Proc. IEEE Int. Conf. Robot. Autom. (ICRA)*, Kobe, Japan, 2009, pp. 4387–4393. Accessed: Nov. 06, 2025.
- [11] M. Magnusson, A. J. Lilienthal, and T. Duckett, “Scan registration for autonomous mining vehicles using 3D-NDT,” *J. Field Robot.*, vol. 24, no. 10, pp. 803–827, Oct. 2007. Accessed: Nov. 22, 2025.
- [12] J. Zhang and S. Singh, “LOAM: Lidar odometry and mapping in real-time,” in *Proc. Robot.: Sci. Syst. (RSS)*, Berkeley, CA, USA, 2014. Accessed: Dec. 19, 2025.
- [13] S. Kohlbrecher, J. Meyer, O. von Stryk, and U. Klingauf, “A flexible and scalable SLAM system with full 3D motion estimation,” in *Proc. IEEE Int. Symp. Safety, Security, Rescue Robot. (SSRR)*, Kyoto, Japan, 2011, pp. 155–160. Accessed: December. 14, 2025.

- [14] W. Hess, D. Kohler, H. Rapp, and D. Andor, “Real-time loop closure in 2D LiDAR SLAM,” in *Proc. IEEE Int. Conf. Robot. Autom. (ICRA)*, Stockholm, Sweden, 2016, pp. 1271–1278. Accessed: Dec. 07, 2025.
- [15] G. Grisetti, R. Kümmerle, C. Stachniss, and W. Burgard, “A tutorial on graph-based SLAM,” *IEEE Intell. Transp. Syst. Mag.*, vol. 2, no. 4, pp. 31–43, Winter 2010. Accessed: Dec. 11, 2025.
- [16] Open Robotics, “ROS 2 documentation.” [Online]. Available: <https://docs.ros.org>. Accessed: Jan. 15, 2026.
- [17] Open Robotics, “ROS 2 design: Discovery and middleware.” [Online]. Available: <https://design.ros2.org>. Accessed: Jan. 12, 2026.
- [18] Object Management Group (OMG), *Data Distribution Service (DDS) Specification*, Version 1.4, 2015. Accessed: Dec. 04, 2025.
- [19] Docker Inc., “Docker documentation.” [Online]. Available: <https://docs.docker.com>. Accessed: Jan. 11, 2026.
- [20] NVIDIA Corp., “Jetson Nano developer kit—Technical specifications.” [Online]. Available: <https://developer.nvidia.com>. Accessed: Jan. 09, 2026.
- [21] NVIDIA Corp., “NVIDIA container toolkit documentation.” [Online]. Available: <https://docs.nvidia.com/datacenter/cloud-native>. Accessed: Jan. 08, 2026.
- [22] D. Titterton and J. Weston, *Strapdown Inertial Navigation Technology*, 2nd ed. London, U.K.: IET, 2004. Accessed: Jan. 10, 2025.
- [23] P. Burggräf *et al.*, “Model-based systems engineering for autonomous systems,” *SN Appl. Sci.*, vol. 1, no. 3, pp. 1–15, 2019. Accessed: Nov. 24, 2025.
- [24] M. Šmelík, “Error analysis of the inertial measurement unit,” *Acta Avionica*, Faculty of Aeronautics, Technical Univ. of Košice, Košice, Slovakia, 2018. Accessed: Jan. 16, 2025.
- [25] G. Grisetti, C. Stachniss, and W. Burgard, “A tutorial on graph-based SLAM,” *IEEE Intell. Transp. Syst. Mag.*, vol. 2, no. 4, pp. 31–43, Winter 2010. Accessed: Jan. 16, 2025.
- [26] S. Beaus Gratacos, “Modelling and sensor integration for an autonomous RC car using CAD and ROS/Gazebo,” M.Eng. thesis, Dept. Mech. Eng. Sci., Univ. of Surrey, Guildford, U.K., May 2022. Accessed: Jan. 16, 2025.
- [28] NVIDIA Corporation, *Jetson TX2i Module*, NVIDIA Corporation, 2022. Available at: [NVIDIA Jetson TX2i](#) [Accessed 7 May 2026].
- [29] Parker Hannifin Corp, *3DM-GX5-25 Datasheet and Specifications*, Parker Hannifin Corporation, 2022. Available at: [Microstrain GX5-25](#) [Accessed 7 May 2026].

- [30] Livox Technology, *Livox MID-360 LiDAR Sensor*, Livox Technology, 2026. Available at: [Livox MID-360](#) [Accessed 7 May 2026].
- [31] NVIDIA Corporation, *Jetson Orin Nano Developer Kit*, NVIDIA Corporation, 2026. Available at: [Jetson Orin Nano Developer Kit](#) [Accessed 7 May 2026].
- [32] Connect Tech, *Quasar Carrier for NVIDIA Jetson TX2/TX2i*, Connect Tech Inc., 2022. Available at: [Quasar Carrier Board](#) [Accessed 7 May 2026].
- [33] Connect Tech, *Active Heat Sink for Jetson TX2i*, Connect Tech Inc., 2022. Available at: [Jetson TX2i Active Heat Sink](#) [Accessed 7 May 2026].
- [34] TRI-M Technologies, *203W Smart PC104 Power Supply Unit*, TRI-M Technologies, 2022. Available at: [Tri-M PC104 PSU](#) [Accessed 7 May 2026].
- [35] RSHTECH, *7-Port Powered USB 3.0 Hub*, RSHTECH, 2022. Available at: [RSHTECH USB Hub](#) [Accessed 7 May 2026].
- [36] TRAXXAS, *X-Maxx 8S Monster Truck Specifications*, TRAXXAS, 2022. Available at: [Traxxas X-Maxx 8S](#) [Accessed 7 May 2026].
- [37] TRAXXAS, *6700mAh 14.8V LiPo Battery Pack*, TRAXXAS, 2026. Available at: [Traxxas 6700mAh LiPo Battery](#) [Accessed 7 May 2026].
- [38] TP-Link, *300Mbps Mini Wireless N USB Adapter*, TP-Link Technologies, 2022. Available at: [TP-Link Wireless USB Adapter](#) [Accessed 7 May 2026].
- [39] D-Link Corporation, *Wireless AC600 Dual Band USB Adapter*, D-Link Corporation, 2022. Available at: [D-Link AC600 USB Adapter](#) [Accessed 7 May 2026].
- [40] SunFounder, *PCA9685 16 Channel PWM Driver Board*, SunFounder, 2022. Available at: [SunFounder PCA9685 PWM Driver](#) [Accessed 7 May 2026].
- [41] HobbyKing, *Turnigy 5000mAh 3S 20C LiPo Battery*, HobbyKing, 2022. Available at: [Turnigy 5000mAh 3S LiPo](#) [Accessed 7 May 2026].
- [42] EV-PEAK, *E4 50W Balance Charger*, EV-PEAK, 2018. Available at: [EV-PEAK E4 Charger](#) [Accessed 7 May 2026].
- [43] AMASS, *XT60 Nylon Connectors*, AMASS Electronics, 2022. Available at: [XT60 Connectors](#) [Accessed 7 May 2026].
- [44] Turnigy, *14AWG Silicone Wire Specifications*, Turnigy, 2022. Available at: [Turnigy 14AWG Silicone Wire](#) [Accessed 7 May 2026].
- [45] RS PRO, *5V DC Axial Fan Specifications*, RS Components Ltd., 2022. Available at: [RS PRO 5V Axial Fan](#) [Accessed 7 May 2026].
- [46] Autodesk Inc., *Autodesk Fusion for Education*, Autodesk Inc., 2026. Available at: [Autodesk Fusion](#) [Accessed 7 May 2026].

[47] Open Robotics, *ROS 2 Documentation*, Open Robotics, 2026. Available at: [ROS 2 Documentation](#) [Accessed 7 May 2026].

[48] MathWorks, *MATLAB and Simulink Student Version*, MathWorks Inc., 2026. Available at: [MATLAB Student Version](#) [Accessed 7 May 2026].

[49] Open Robotics, *RViz2 Visualisation Tool*, Open Robotics, 2026. Available at: [RViz2 Documentation](#) [Accessed 7 May 2026].

[50] MicroStrain, “3DM-GX5-25 Datasheet,” MicroStrain Sensing Systems, 2020. [Online]. Available: [3DM-GX5-25 Datasheet](#). [Accessed: 7-May-2026].

[51] Livox Technology, “MID-360 Specifications,” Livox Technology Co., Ltd. [Online]. Available: [Livox MID-360 Specifications](#). [Accessed: 7-May-2026].

Appendices

APPENDIX A | PLANNING

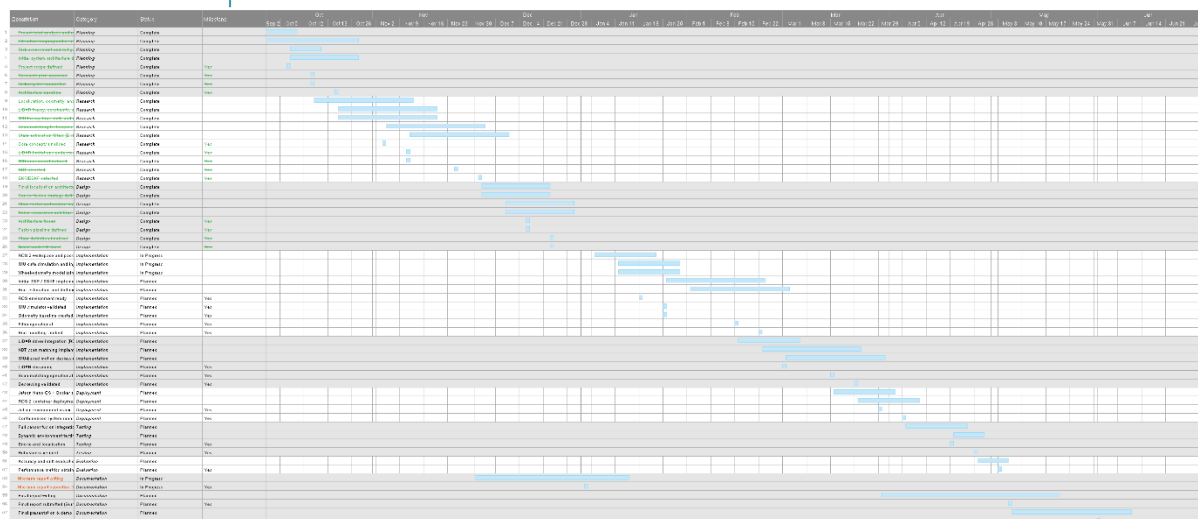


Figure 99 - Complete Gantt Chart

	Phase	Description	Category	Status	Start	Finish	Milestone	Duration	Predecessor	At Risk
1	Phase 1 – Project Definition & Planning				01/10/25	01/10/25		1d		☐
2		Project brief analysis and requirements review	Planning	Complete	01/10/25	09/10/25		7d		☒
3		Literature scoping and research plan	Planning	Complete	01/10/25	27/10/25		19d		☐
4		Risk assessment and mitigation strategy	Planning	Complete	08/10/25	16/10/25		7d		☐
5		Initial system architecture definition	Planning	Complete	08/10/25	27/10/25		14d		☐
6		Project scope defined	Planning	Complete	07/10/25	07/10/25	Yes	1d		☐
7		Research plan approved	Planning	Complete	14/10/25	14/10/25	Yes	1d		☐
8		Risk register completed	Planning	Complete	14/10/25	14/10/25	Yes	1d		☐
9		Architecture baseline	Planning	Complete	21/10/25	24/10/25	Yes	1d		☐
10	Phase 2 – Literature Review & Technical Foundations				15/10/25	15/10/25		1d		☐
11		Localisation, odometry, and sensor fusion review	Research	Complete	15/10/25	16/11/25		21d		☐
12		LIDAR theory, constraints, and error analysis	Research	Complete	22/10/25	19/11/25		21d		☐
13		IMU theory, bias, drift, and noise modelling	Research	Complete	22/10/25	19/11/25		21d		☐
14		Scan matching techniques (ICP, GSM, NDT)	Research	Complete	05/11/25	09/12/25		21d		☐
15		State estimation filters (EKF, ESKF, PF, GF)	Research	Complete	12/11/25	10/12/25		21d		☐
16		Gaze concepts finalised	Research	Complete	04/11/25	04/11/25	Yes	1d		☐
17		LIDAR limitations understood	Research	Complete	11/11/25	11/11/25	Yes	1d		☐
18		IMU error model defined	Research	Complete	11/11/25	11/11/25	Yes	1d		☐
19		NDT selected	Research	Complete	25/11/25	26/11/25	Yes	1d		☐
20		EKF/ESKF selected	Research	Complete	02/12/25	02/12/25	Yes	1d		☐
21	Phase 3 – System Design & Algorithm Selection				03/12/25	03/12/25		1d		☐
22		Final localisation architecture definition	Design	Complete	03/12/25	22/12/25		14d		☐
23		Sensor fusion strategy definition	Design	Complete	03/12/25	20/12/25		14d		☐
24		State vector and motion model definition	Design	Complete	10/12/25	29/12/25		14d		☐
25		Noise covariance and bias strategy	Design	Complete	10/12/25	29/12/25		14d		☐
26		Architecture frozen	Design	Complete	16/12/25	16/12/25	Yes	1d		☐
27		Fusion pipeline defined	Design	Complete	16/12/25	16/12/25	Yes	1d		☐
28		State definition finalised	Design	Complete	23/12/25	23/12/25	Yes	1d		☐
29		Noise model defined	Design	Complete	23/12/25	23/12/25	Yes	1d		☐
30	Phase 4 – Simulation & Offline Development				05/01/26	05/01/26		1d		☐
31		ROS 2 workspace and package setup	Implementation	In Progress	05/01/26	22/01/26		14d		☐
32		IMU data simulation and injection	Implementation	In Progress	12/01/26	29/01/26		14d		☐
33		Wheel-odometry model (simulation only)	Implementation	In Progress	12/01/26	29/01/26		14d		☐
34		Initial EKF / ESKF implementation	Implementation	Planned	26/01/26	23/02/26		21d		☐
35		Bias estimation and drift evaluation	Implementation	Planned	02/02/26	02/03/26		21d		☐
36		ROS environment ready	Implementation	Planned	18/01/26	18/01/26	Yes	1d		☐
37		IMU simulator validated	Implementation	Planned	25/01/26	25/01/26	Yes	1d		☐
38		Odometry baseline created	Implementation	Planned	25/01/26	25/01/26	Yes	1d		☐
39		Filter operational	Implementation	Planned	15/02/26	15/02/26	Yes	1d		☐
40		Bias handling verified	Implementation	Planned	22/02/26	22/02/26	Yes	1d		☐
41	Phase 5 – LiDAR Integration & Scan Matching				16/02/26	16/02/26		1d		☐
42		LIDAR driver integration (ROS 2)	Implementation	Planned	16/02/26	05/03/26		14d		☐
43		NDT scan matching implementation	Implementation	Planned	23/02/26	23/03/26		21d		☐
44		IMU-based motion deskewing	Implementation	Planned	02/03/26	30/03/26		21d		☐
45		LIDAR streaming	Implementation	Planned	01/03/26	01/03/26	Yes	1d		☐
46		Scan matching operational	Implementation	Planned	15/03/26	15/03/26	Yes	1d		☐
47		Deskewing validated	Implementation	Planned	22/03/26	22/03/26	Yes	1d		☐
48		Phase 6 – Embedded Deployment				16/03/26	16/03/26	1d		☐
49		Jetson Nano OS + Docker setup	Deployment	Planned	16/03/26	02/04/26		14d		☐
50		ROS 2 container deployment	Deployment	Planned	23/03/26	09/04/26		14d		☐
51		Jetson environment ready	Deployment	Planned	29/03/26	29/03/26	Yes	1d		☐
52		Containerised system running	Deployment	Planned	05/04/26	05/04/26	Yes	1d		☐
53		Phase 7 – System Integration & Testing				06/04/26	06/04/26	1d		☐
54		Full sensor fusion integration	Testing	Planned	06/04/26	23/04/26		14d		☐
55		Dynamic environment testing	Testing	Planned	20/04/26	28/04/26		7d		☐
56		End-to-end localisation	Testing	Planned	19/04/26	19/04/26	Yes	1d		☐
57		Robustness verified	Testing	Planned	26/04/26	26/04/26	Yes	1d		☐
58		Phase 8 – Evaluation & Analysis				27/04/26	27/04/26	1d		☐
59		Accuracy and drift evaluation	Evaluation	Planned	27/04/26	05/05/26		7d		☐
60		Performance metrics obtained	Evaluation	Planned	03/05/26	03/05/26	Yes	1d		☐
61		Phase 9 – Documentation & Final Submission				01/12/25	01/12/25	1d		☐
62		Mid-term report writing	Documentation	In Progress	01/12/25	14/01/26		33d		☒
63		Mid-term report submitted (SurreyLearn, 4pm)	Documentation	In Progress	02/01/26	02/01/26	Yes	1d		☐
64		Final report writing	Documentation	Planned	30/03/26	20/05/26		38d		☐
65		Final report submitted (SurreyLearn, 4pm)	Documentation	Planned	06/05/26	06/05/26	Yes	1d		☐
66		Final presentation & demo prep	Documentation	Planned	07/05/26	10/06/26		25d		☐
67		Final presentation / viva day	Documentation	Planned	01/06/26	01/06/26	Yes	1d		☐

Table 18 - Risk Assessment Matrix

ID	Risk Description	Likelihood	Impact	Risk Level	Mitigation Strategy
1	Delay in obtaining a suitable development computer for ROS programming	Medium	High	High	Use a temporary personal machine or university lab PC; begin development in simulation while awaiting hardware availability
2	IMU drift dominates pose estimation during aggressive motion	High	Medium	High	Use LiDAR-based corrections via scan-to-map matching
3	LiDAR scan distortion during fast robot motion	Medium	High	High	Deskew scans using high-rate IMU angular velocity before scan matching.

4	Computational overload on Jetson Nano	Medium	High	High	Avoid full SLAM; use map-based localisation; reduce algorithm complexity.
5	ROS 2 incompatibility with Jetson Nano OS	High	Medium	High	Deploy ROS 2 using Docker containers with NVIDIA Container Toolkit
6	Sensor synchronisation errors between IMU and LiDAR	Medium	Medium	Medium	Timestamp all sensor messages and ensure ROS time configuration
7	Poor scan matching performance in feature-poor environments	Medium	Medium	Medium	Rely more heavily on inertial prediction
8	Limited real-time performance during integrated testing	Low	High	Medium	Incremental testing: validate IMU-only, then LiDAR-only, before full fusion
9	Insufficient time for experimental validation	Medium	Medium	Medium	Prioritise core localisation performance measures; reduce optional features if required
10	Documentation/report writing falls behind technical development	Low	Medium	Low	Maintain parallel report drafting alongside development; update sections incrementally

APPENDIX B: ADDITIONAL PROJECT INFORMATION

/Odometry	nav_msgs/msg/Odometry
/clicked_point	geometry_msgs/msg/PointStamped
/clock	roscpp_msgs/msg/Clock
/cloud_effected	sensor_msgs/msg/PointCloud2
/cloud_registered	sensor_msgs/msg/PointCloud2
/cloud_registered_body	sensor_msgs/msg/PointCloud2
/ekf/imu_source	std_msgs/msg/String
/ekf/odom	nav_msgs/msg/Odometry
/ekf/path	nav_msgs/msg/Path
/ekf/pose	geometry_msgs/msg/PoseStamped
/events/read_split	roscpp_interfaces/msg/ReadSplitEvent
/imu/data	sensor_msgs/msg/Imu
/initialpose	geometry_msgs/msg/PoseWithCovarianceStamped
/keyframe_map	sensor_msgs/msg/PointCloud2
/keyframe_map/keyframes	nav_msgs/msg/Path
/keyframe_map/loop_closure_anchor_pose	geometry_msgs/msg/PoseStamped
/keyframe_map/loop_closure_match_index	std_msgs/msg/UInt32
/keyframe_map/loop_closure_overlap	std_msgs/msg/Float32
/keyframe_map/loop_closure_pair	std_msgs/msg/Int32MultiArray
/lidar/odom	nav_msgs/msg/Odometry
/lidar/odom_raw	nav_msgs/msg/Odometry
/livox/imu	sensor_msgs/msg/Imu
/livox/lidar	sensor_msgs/msg/PointCloud2
/mag	sensor_msgs/msg/MagneticField
/move_base_simple/goal	geometry_msgs/msg/PoseStamped
/parameter_events	rcl_interfaces/msg/ParameterEvent
/path	nav_msgs/msg/Path
/pose_graph/corrected_keyframes	nav_msgs/msg/Path
/rosout	rcl_interfaces/msg/Log
/tf	tf2_msgs/msg/TFMessage
/tf_static	tf2_msgs/msg/TFMessage


```

1  ROS_Deploy/
2  |  ├── .cursor/
3  |  ├── .vscode/
4  |  |  ├── browse.vc.db
5  |  |  ├── c_cpp_properties.json
6  |  |  └── settings.json
7  |  ├── docs/
8  |  |  ├── dissertation_assets/
9  |  |  |  ├── runtime/
10 |  |  |  └── static/
11 |  |  ├── dissertation_figure_automation.md
12 |  |  ├── imu_mount_note.md
13 |  |  ├── livox_mount_note.md
14 |  |  ├── SLAM_CONFIGURATION.md
15 |  |  └── tuning.MD
16 |  ├── documentation/
17 |  |  └── issues-and-solutions-summary.md
18 |  ├── readme/
19 |  |  └── TUNING.md
20 |  ├── scripts/
21 |  |  └── dissertation/
22 |  |  |  ├── build_report_assets.py
23 |  |  |  ├── capture_runtime_assets.sh
24 |  |  |  ├── export_static_assets.py
25 |  |  |  └── run_all.sh
26 |  ├── src/
27 |  |  ├── FAST_LIO_ROS2/
28 |  |  |  ├── .github/
29 |  |  |  ├── config/
30 |  |  |  ├── doc/
31 |  |  |  ├── include/
32 |  |  |  ├── launch/
33 |  |  |  ├── Log/
34 |  |  |  ├── msg/
35 |  |  |  ├── PCD/
36 |  |  |  ├── rviz/
37 |  |  |  ├── rviz_cfg/
38 |  |  |  ├── src/
39 |  |  |  ├── .gitignore
40 |  |  |  ├── .gitmodules
41 |  |  |  ├── CMakeLists.txt
42 |  |  |  ├── LICENSE
43 |  |  |  ├── package.xml
44 |  |  |  └── README.md
45 |  |  └── ...
...
|  |  ├── keyframe_scan_map/
|  |  |  ├── config/
|  |  |  ├── keyframe_scan_map
|  |  |  ├── launch/
|  |  |  ├── resource/
|  |  |  ├── package.xml
|  |  |  ├── README.md
|  |  |  ├── setup.cfg
|  |  |  └── setup.py
|  |  ├── lidar_odometry/
|  |  |  ├── launch/
|  |  |  ├── lidar_odometry/
|  |  |  ├── resource/
|  |  |  ├── src/
|  |  |  ├── CMakeLists.txt
|  |  |  ├── package.xml
|  |  |  └── README.md
|  |  ├── lio_bringup/
|  |  |  ├── config/
|  |  |  ├── launch/
|  |  |  ├── lio_bringup/
|  |  |  ├── resource/
|  |  |  ├── package.xml
|  |  |  ├── setup.cfg
|  |  |  └── setup.py
|  |  ├── livox_ros_driver2/
|  |  |  ├── 3rdparty/
|  |  |  ├── cmake/
|  |  |  ├── config/
|  |  |  ├── launch_ROS1/
|  |  |  ├── launch_ROS2/
|  |  |  ├── msg/
|  |  |  ├── src/
|  |  |  ├── .gitignore
|  |  |  ├── build.sh
|  |  |  ├── CHANGELOG.md
|  |  |  ├── CMakeLists.txt
|  |  |  ├── LICENSE.txt
|  |  |  ├── package.xml
|  |  |  ├── package_ROS1.xml
|  |  |  └── README.md
...
|  |  ├── localisation_ekf/
|  |  |  ├── config/
|  |  |  ├── localisation_ekf/
|  |  |  ├── resource/
|  |  |  ├── src/
|  |  |  ├── test/
|  |  |  ├── package.xml
|  |  |  ├── setup.cfg
|  |  |  └── setup.py
|  |  ├── ros_project_bringup/
|  |  |  ├── config/
|  |  |  ├── docs/
|  |  |  ├── launch/
|  |  |  ├── resource/
|  |  |  ├── ros_project_bringup/
|  |  |  ├── rviz/
|  |  |  ├── scripts/
|  |  |  ├── package.xml
|  |  |  └── setup.py
|  |  └── LiDAR-Instructions.md
|  ├── .gitignore
|  ├── autorun.md
|  ├── CHANGES-CONFIGS
|  ├── deployment.md
|  ├── fastlio_odom_base.record_meta.json
|  ├── Function
|  ├── lidar_type
|  └── README.md

```

```

Type: tf2_msgs/msg/TFMessage
Publisher count: 2

Node name: ekf_node
Node namespace: /
Topic type: tf2_msgs/msg/TFMessage
Endpoint type: PUBLISHER
GID: 01.0f.af.08.71.9c.96.f4.00.00.00.00.00.00.18.03.00.00.00.00.00.00.00
QoS profile:
  Reliability: RELIABLE
  History (Depth): UNKNOWN
  Durability: VOLATILE
  Lifespan: Infinite
  Deadline: Infinite
  Liveliness: AUTOMATIC
  Liveliness lease duration: Infinite

Node name: fastlio_mapping
Node namespace: /
Topic type: tf2_msgs/msg/TFMessage
Endpoint type: PUBLISHER
GID: 01.0f.af.08.73.9c.4d.ea.00.00.00.00.00.00.1b.03.00.00.00.00.00.00.00
QoS profile:
  Reliability: RELIABLE
  History (Depth): UNKNOWN
  Durability: VOLATILE
  Lifespan: Infinite
  Deadline: Infinite
  Liveliness: AUTOMATIC
  Liveliness lease duration: Infinite

Subscription count: 3

Node name: ekf_node
Node namespace: /
Topic type: tf2_msgs/msg/TFMessage
Endpoint type: SUBSCRIPTION
GID: 01.0f.af.08.71.9c.96.f4.00.00.00.00.00.00.12.04.00.00.00.00.00.00.00
QoS profile:
  Reliability: RELIABLE
  History (Depth): UNKNOWN
  Durability: VOLATILE
  Lifespan: Infinite
  Deadline: Infinite
  Liveliness: AUTOMATIC
  Liveliness lease duration: Infinite

Node name: keyframe_map_node
Node namespace: /
Topic type: tf2_msgs/msg/TFMessage
Endpoint type: SUBSCRIPTION
GID: 01.0f.af.08.8a.9c.20.6b.00.00.00.00.00.00.12.04.00.00.00.00.00.00.00
QoS profile:
  Reliability: RELIABLE
  History (Depth): UNKNOWN
  Durability: VOLATILE
  Lifespan: Infinite
  Deadline: Infinite
  Liveliness: AUTOMATIC
  Liveliness lease duration: Infinite

Node name: transform_listener_impl_60660f1003c0
Node namespace: /
Topic type: tf2_msgs/msg/TFMessage
Endpoint type: SUBSCRIPTION
GID: 01.0f.af.08.ab.9c.6a.51.00.00.00.00.00.00.16.04.00.00.00.00.00.00.00
QoS profile:
  Reliability: RELIABLE
  History (Depth): UNKNOWN
  Durability: VOLATILE
  Lifespan: Infinite
  Deadline: Infinite
  Liveliness: AUTOMATIC
  Liveliness lease duration: Infinite

```

Type: nav_msgs/msg/Odometry

Publisher count: 1

```

Node name: ekf_node
Node namespace: /
Topic type: nav_msgs/msg/Odometry
Endpoint type: PUBLISHER
GID: 01.0f.af.08.71.9c.96.f4.00.00.00.00.00.00.14.03.00.00.00.00.00.00.00
QoS profile:
  Reliability: RELIABLE
  History (Depth): UNKNOWN
  Durability: VOLATILE
  Lifespan: Infinite
  Deadline: Infinite
  Liveliness: AUTOMATIC
  Liveliness lease duration: Infinite

```

Subscription count: 1

```

Node name: rviz2
Node namespace: /
Topic type: nav_msgs/msg/Odometry
Endpoint type: SUBSCRIPTION
GID: 01.0f.af.08.ab.9c.6a.51.00.00.00.00.00.00.1a.04.00.00.00.00.00.00.00
QoS profile:
  Reliability: RELIABLE
  History (Depth): UNKNOWN
  Durability: VOLATILE
  Lifespan: Infinite
  Deadline: Infinite
  Liveliness: AUTOMATIC
  Liveliness lease duration: Infinite

```

Tf_static info

```

Type: tf2_msgs/msg/TFMessage
Publisher count: 6

Node name: map_to_odom
Node namespace: /
Topic type: tf2_msgs/msg/TFMessage
Endpoint type: PUBLISHER
GID: 01.0f.af.08.67.9c.34.66.00.00.00.00.00.13.03.00.00.00.00.00.00.00
QoS profile:
  Reliability: RELIABLE
  History (Depth): UNKNOWN
  Durability: TRANSIENT_LOCAL
  Lifespan: Infinite
  Deadline: Infinite
  Liveliness: AUTOMATIC
  Liveliness lease duration: Infinite

Node name: base_link_to_imu_link
Node namespace: /
Topic type: tf2_msgs/msg/TFMessage
Endpoint type: PUBLISHER
GID: 01.0f.af.08.69.9c.76.f5.00.00.00.00.00.13.03.00.00.00.00.00.00.00
QoS profile:
  Reliability: RELIABLE
  History (Depth): UNKNOWN
  Durability: TRANSIENT_LOCAL
  Lifespan: Infinite
  Deadline: Infinite
  Liveliness: AUTOMATIC
  Liveliness lease duration: Infinite

Node name: base_link_to_livox
Node namespace: /
Topic type: tf2_msgs/msg/TFMessage
Endpoint type: PUBLISHER
GID: 01.0f.af.08.6b.9c.4b.ec.00.00.00.00.00.13.03.00.00.00.00.00.00.00
QoS profile:
  Reliability: RELIABLE
  History (Depth): UNKNOWN
  Durability: TRANSIENT_LOCAL
  Lifespan: Infinite
  Deadline: Infinite
  Liveliness: AUTOMATIC
  Liveliness lease duration: Infinite

Node name: livox_frame_to_livox_imu_bridge
Node namespace: /
Topic type: tf2_msgs/msg/TFMessage
Endpoint type: PUBLISHER
GID: 01.0f.af.08.6d.9c.ae.0e.00.00.00.00.00.13.03.00.00.00.00.00.00.00
QoS profile:
  Reliability: RELIABLE
  History (Depth): UNKNOWN
  Durability: TRANSIENT_LOCAL
  Lifespan: Infinite
  Deadline: Infinite
  Liveliness: AUTOMATIC
  Liveliness lease duration: Infinite

Node namespace: /
Topic type: tf2_msgs/msg/TFMessage
Endpoint type: PUBLISHER
GID: 01.0f.af.08.6f.9c.d5.32.00.00.00.00.00.13.03.00.00.00.00.00.00.00
QoS profile:
  Reliability: RELIABLE
  History (Depth): UNKNOWN
  Durability: TRANSIENT_LOCAL
  Lifespan: Infinite
  Deadline: Infinite
  Liveliness: AUTOMATIC
  Liveliness lease duration: Infinite

Node name: odom_to_camera_init_for_fastlio_record
Node namespace: /
Topic type: tf2_msgs/msg/TFMessage
Endpoint type: PUBLISHER
GID: 01.0f.af.08.7a.9c.ec.1b.00.00.00.00.00.13.03.00.00.00.00.00.00.00
QoS profile:
  Reliability: RELIABLE
  History (Depth): UNKNOWN
  Durability: TRANSIENT_LOCAL
  Lifespan: Infinite
  Deadline: Infinite
  Liveliness: AUTOMATIC
  Liveliness lease duration: Infinite

Subscription count: 3

Node name: ekf_node
Node namespace: /
Topic type: tf2_msgs/msg/TFMessage
Endpoint type: SUBSCRIPTION
GID: 01.0f.af.08.71.9c.96.f4.00.00.00.00.00.13.04.00.00.00.00.00.00.00
QoS profile:
  Reliability: RELIABLE
  History (Depth): UNKNOWN
  Durability: TRANSIENT_LOCAL
  Lifespan: Infinite
  Deadline: Infinite
  Liveliness: AUTOMATIC
  Liveliness lease duration: Infinite

Node name: keyframe_map_node
Node namespace: /
Topic type: tf2_msgs/msg/TFMessage
Endpoint type: SUBSCRIPTION
GID: 01.0f.af.08.8a.9c.20.6b.00.00.00.00.00.13.04.00.00.00.00.00.00.00
QoS profile:
  Reliability: RELIABLE
  History (Depth): UNKNOWN
  Durability: TRANSIENT_LOCAL
  Lifespan: Infinite
  Deadline: Infinite
  Liveliness: AUTOMATIC
  Liveliness lease duration: Infinite

Node name: transform_listener_impl_60660f1003c0
Node namespace: /
Topic type: tf2_msgs/msg/TFMessage
Endpoint type: SUBSCRIPTION
GID: 01.0f.af.08.ab.9c.6a.51.00.00.00.00.00.17.04.00.00.00.00.00.00.00
QoS profile:
  Reliability: RELIABLE
  History (Depth): UNKNOWN
  Durability: TRANSIENT_LOCAL

```

Topic	Message Type	Publisher Node	Subscriber Node(s)	Reliability	Durability	Purpose
/livox/lidar	sensor_msgs/msg/PointCloud2	livox_lidar_publisher	lidar_odometry_node, keyframe_map_node	Reliable	Volatile	Publishes raw LiDAR point cloud data used for odometry estimation and mapping
/livox/imu	sensor_msgs/msg/Imu	livox_lidar_publisher	fastlio_mapping / optional estimator inputs	Reliable	Volatile	Publishes embedded Livox IMU measurements for motion compensation and estimation
/imu/data	sensor_msgs/msg/Imu	microstrain_inertial_driver	ekf_node	Reliable	Volatile	Publishes inertial measurements from the

						Microstrain GX5-25 IMU
/lidar/odom	nav_msgs/msg/Odometry	lidar_odometry_node	ekf_node	Reliable	Volatile	Publishes LiDAR-based odometry estimates used for EKF correction
/lidar/odom_raw	nav_msgs/msg/Odometry	lidar_odometry_node	lidar_odom_ema_smooth, optional debug nodes	Reliable	Volatile	Publishes unsmoothed LiDAR odometry data prior to filtering
/ekf/odom	nav_msgs/msg/Odometry	ekf_node	rviz2, keyframe_map_node	Reliable	Volatile	Publishes fused robot odometry estimates generated by EKF sensor fusion
/ekf/pose	geometry_msgs/msg/PoseStamped	ekf_node	rviz2	Reliable	Volatile	Publishes fused robot pose estimates for visualisation
/ekf/path	nav_msgs/msg/Path	ekf_node	rviz2	Reliable	Volatile	Publishes the estimated robot trajectory over time
/keyframe_map	sensor_msgs/msg/PointCloud2	keyframe_map_node	rviz2	Reliable	Volatile	Publishes the accumulated keyframe-based map representation
/keyframe_map/keyframes	nav_msgs/msg/Path	keyframe_map_node	pose_graph_node, rviz2	Reliable	Volatile	Publishes stored keyframe trajectory information
/pose_graph/corrected_keyframes	nav_msgs/msg/Path	pose_graph_node	rviz2	Reliable	Volatile	Publishes globally corrected trajectories after loop closure optimisation
/tf	tf2_msgs/msg/TFMessage	Multiple TF broadcasters	All TF listeners	Reliable	Volatile	Publishes dynamic coordinate frame transforms throughout the ROS graph

/tf_static	tf2_msgs/msg/TFMessage	Static transform publishers	All TF listeners	Reliable	Transient Local	Publishes persistent static coordinate transforms between fixed frames
/rosout	rcl_interfaces/msg/Log	Multiple ROS nodes	ROS logging tools	Reliable	Volatile	Publishes runtime logging and debugging information
/parameter_events	rcl_interfaces/msg/ParameterEvent	ROS parameter system	ROS nodes/tools	Reliable	Volatile	Publishes runtime parameter update events

APPENDIX C: LINKS

[ROS 2 Humble](#)

[ROS 2 Foxy \(Custom\)](#)

[Project Location](#)